

Aligned and oblique dynamics in recurrent neural networks

Reviewed Preprint

v2 • October 11, 2024

Revised by authors

Reviewed Preprint

v1 • March 13, 2024

Friedrich Schuessler , Francesca Mastrogiuseppe, Srdjan Ostojic, Omri Barak

Faculty of Electrical Engineering and Computer Science, Technical University Berlin, Germany • Science of Intelligence, Research Cluster of Excellence, Berlin, Germany • Champalimaud Foundation, Lisbon, Portugal • Laboratoire de Neurosciences Cognitives et Computationnelles, INSERM U960, Ecole Normale Supérieure - PSL Research University, Paris, France • Rappaport Faculty of Medicine and Network Biology Research Laboratories, Technion - Israeli Institute of Technology, Haifa, Israel

 https://en.wikipedia.org/wiki/Open_access
 Copyright information

Abstract

The relation between neural activity and behaviorally relevant variables is at the heart of neuroscience research. When strong, this relation is termed a neural representation. There is increasing evidence, however, for partial dissociations between activity in an area and relevant external variables. While many explanations have been proposed, a theoretical framework for the relationship between external and internal variables is lacking. Here, we utilize recurrent neural networks (RNNs) to explore the question of when and how neural dynamics and the network's output are related from a geometrical point of view. We find that training RNNs can lead to two dynamical regimes: dynamics can either be aligned with the directions that generate output variables, or oblique to them. We show that the choice of readout weight magnitude before training can serve as a control knob between the regimes, similar to recent findings in feedforward networks. These regimes are functionally distinct. Oblique networks are more heterogeneous and suppress noise in their output directions. They are furthermore more robust to perturbations along the output directions. Crucially, the oblique regime is specific to recurrent (but not feedforward) networks, arising from dynamical stability considerations. Finally, we show that tendencies towards the aligned or the oblique regime can be dissociated in neural recordings. Altogether, our results open a new perspective for interpreting neural activity by relating network dynamics and their output.

eLife Assessment

This work provides an **important** and novel framework for interpreting the interactions between recurrent dynamics across stages of neural processing. The authors report that two different kinds of dynamics exist in recurrent networks differing in the extent to which they align with the output weights. The authors also present **convincing** evidence that both types of dynamics exist in the brain.

<https://doi.org/10.7554/eLife.93060.2.sa3>

1. Introduction

The relation between neural activity and behavioral variables is often expressed in terms of neural representations. Sensory input and motor output have been related to the tuning curves of single neurons [23, 26, 45] and, since the advent of large-scale recordings, to population activity [7, 64, 80]. Both input and output can be decoded from population activity, [11, 41], even in real-time, closed-loop settings [63, 82]. However, neural activity is often not fully explained by observable behavioral variables. Some components of the unexplained neural activity have been interpreted as random trial-to-trial fluctuations [16], potentially linked to unobserved behavior [44, 72, 81]. Activity may further be due to other ongoing computations not immediately related to behavior, such as preparatory motor activity in a null space of the motor readout [24, 31]. Finally, neural activity may partially be due to other constraints, for example related to the underlying connectivity [2, 48], the process of learning [63], or stability, i.e., the robustness of the neural dynamics to perturbations [62].

Here we aim for a theoretical understanding of neural representations: Which factors might determine how strongly activity and behavioral output variables are related? To this end, we use trained recurrent neural networks (RNNs). In this setting, output variables are determined by the task at hand, and neural activity can be described by its projection onto the principal components (PCs). We show that these networks can operate between two extremes: an “aligned” regime in which the output weights and the largest PCs are strongly correlated; and a second, “oblique” regime, where the output weights and the largest PCs are poorly correlated.

What determines the regime in which a network operates? We show that quite general considerations lead to a link between the magnitude of output weights and the regime of the network. As a consequence, we can use output magnitude as a control knob for trained RNNs. Indeed, when we trained RNN models on different neuroscience tasks, large output weights led to oblique dynamics, and small output weights to aligned dynamics. Recent results in feedforward networks identified two regimes - rich and lazy - that can also arise from choices of output weights [9, 27]. In an extensive Methods section, we further analyze in detail how the oblique and aligned regimes arise during learning. There we show that the dynamical nature of RNNs, in particular demanding stable dynamics, leads to the replacement of unstable, lazy, solutions by oblique ones.

We then considered the functional consequences of the two regimes. Building on the concept of feedback loops driving the network dynamics [57, 75], we show that, in the aligned regime, the largest PCs and the output are qualitatively similar. In the oblique regime, in contrast, the two may be qualitatively different. This functional decoupling in oblique networks leads to a large freedom for neural dynamics. Different networks with oblique dynamics thus tend to employ different dynamics for the same tasks. Aligned dynamics, in contrast, are much more stereotypical. Furthermore, as a result of how neural dynamics and output are coupled, oblique and aligned networks react differently to perturbations of the neural activity along the output direction. In particular, oblique (but not aligned) networks develop an additional negative feedback loop that suppresses output noise. We finally show that neural recordings from different experiments can have different degrees of alignment, which indicates that our theoretical results may be useful in identifying different regimes for different experiments, tasks, or brain regions.

Altogether, our work opens a new perspective relating network dynamics and their output, yielding important insights for modeling brain dynamics as well as experimentally accessible questions about learning and dynamics in the brain.

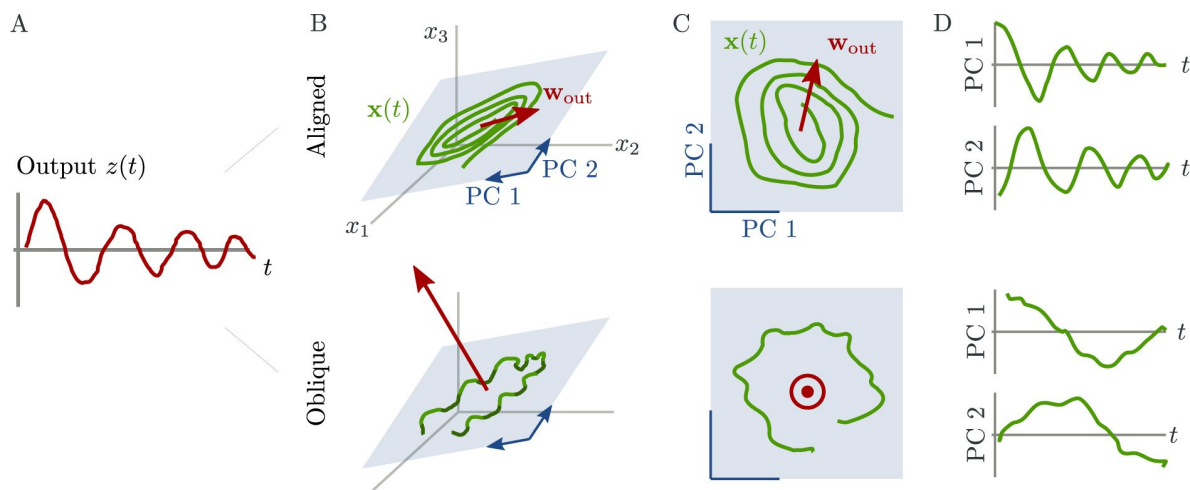


Fig. 1

Schematic of aligned and oblique dynamics in recurrent neural networks. **A** Output generated by both networks. **B** Neural activity of aligned (top) and oblique (bottom) dynamics, visualized in the space spanned by three neurons. Here, the activity (green) is three-dimensional, but most of the variance is concentrated along the two largest PCs (blue). For aligned dynamics, the output weights (red) are small and lie in the subspace spanned by the largest PCs; they are hence correlated to the activity. For oblique dynamics, the output weights are large and lie outside of the subspace spanned by the largest PCs; they are hence poorly correlated to the activity. **C** Projection of activity onto the two largest PCs. For oblique dynamics, the output weights are orthogonal to the leading PCs. **D** Evolution of PC projections over time. For aligned dynamics, the projection on the PCs resembles the output $z(t)$, and reconstructing the output from the largest two components is possible. For the oblique dynamics, such reconstruction is not possible, because the projections oscillate much more slowly than the output.

2. Results

2.1. Aligned and oblique population dynamics

We consider an animal performing a task while both behavior and neural activity are recorded. For example, the task might be to produce a periodic motion, described by the output $z(t)$ of **Fig. 1A**. For simplicity, we assume that the behavioral output can be decoded linearly from the neural activity [19, 41, 61, 63, 82]. We can thus write

$$z(t) = \sum_{i=1}^N w_{\text{out},i} x_i(t) = \mathbf{w}_{\text{out}}^T \mathbf{x}(t), \quad (1)$$

with readout weights $\mathbf{w}_{\text{out}}$. The activity of neuron $i \in \{1, \dots, N\}$ is given by $x_i(t)$, and we refer to the vector $\mathbf{x}$ as the state of the network.

Neural activity has to generate the output in some subspace of the state space, where each axis represents the activity of one neuron. In the simplest case (**Fig. 1B**, top), the output is produced along the largest PCs of activity, as shown by the fact that projecting the neural activity $\mathbf{x}(t)$ onto the largest PCs returns the target oscillation (**Fig. 1D**, top). We call such dynamics “aligned” because of the alignment between the subspace spanned by the largest PCs and the output vector (red).

There is, however, another possibility. Neural activity may have many other components not directly related to the output and these other components may even dominate the overall activity. In this case (**Fig. 1B-D**, bottom), the two largest PCs are not enough to read out the output, and smaller PCs are needed. We call such dynamics “oblique” because the subspace spanned by the largest PCs and the output vector are poorly aligned.

We consider these two possibilities as distinct dynamical regimes, noting that intermediate situations are also possible. The actual regime of neural dynamics has important consequences for how one interprets neural recordings. For aligned dynamics, analyzing the dynamics within the largest PCs may lead to insights about the computations generating the output [80]. For oblique dynamics, such an analysis is hampered by the dissociation between the large PCs and the components generating the output [61].

2.2. Magnitude of output weights controls regime in trained RNNs

What determines which regime a neural network operates in? Given that behavior is the same, but representations differ, we study this question using trained RNNs. This framework constrains what networks do, but not how they do it [3, 74]. The specific property of representation we are interested in is the alignment, or correlation, between output weights and states:

$$\rho(t) = \mathbf{w}_{\text{out}}^T \mathbf{x}(t) / (\|\mathbf{w}_{\text{out}}\| \|\mathbf{x}(t)\|), \quad (2)$$

where the vector norms $\|\mathbf{w}_{\text{out}}\|$ and $\|\mathbf{x}\|$ quantify the magnitude of each vector.

For aligned dynamics, the correlation is large, corresponding to the alignment between the leading PCs of the neural activity and the output weights (**Fig. 1B** top). In contrast, for oblique dynamics, this correlation is small (**Fig. 1B** bottom). Note that the concept of correlation can be generalized to accommodate multiple time points and multidimensional output (see Section 4.3).

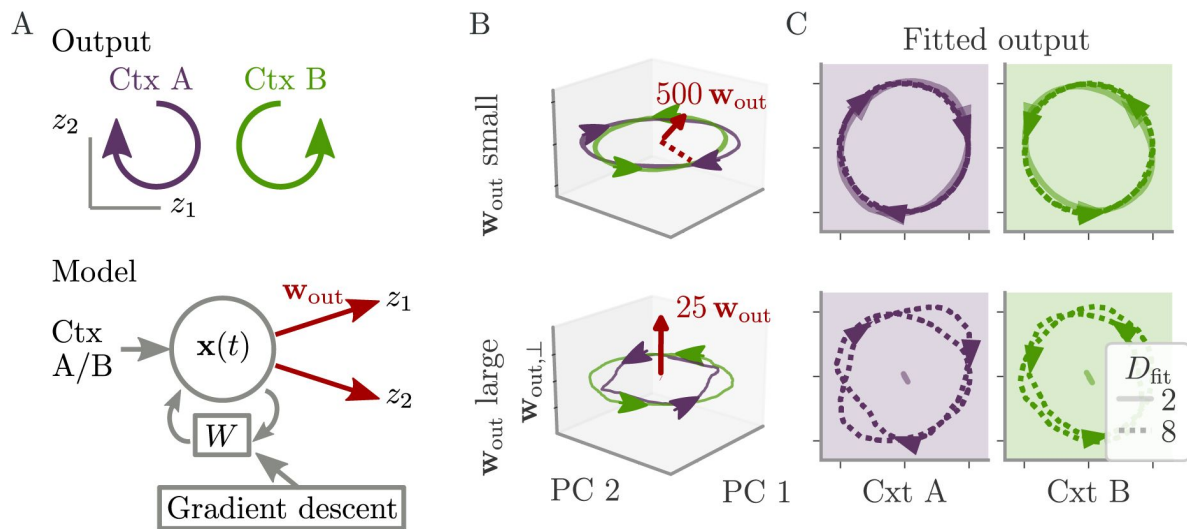


Fig. 2

Aligned and oblique dynamics for a cycling task [61]. **A** A network with two outputs was trained to generate either clockwise or anticlockwise rotations, depending on the context (top). Our model RNN (bottom) received a context input pulse, generated dynamics $\mathbf{x}(t)$ via recurrent weights W , and yielded the output as linear projections of the states. We trained the recurrent weights W with gradient descent. **B-C** Resulting internal dynamics for two networks with small (top) and large (bottom) output weights, corresponding to aligned and oblique dynamics, respectively. **B** Dynamics projected on the first 2 PCs and the remaining direction $\mathbf{w}_{out,\perp}$ of the first output vector (for z_1). The output weights are amplified to be visible. Arrowheads indicate the direction of the dynamics. Note that for the large output weights, the dynamics in the first two PCs co-rotated, despite the counter-rotating output. **C** Output reconstructed from the largest PCs, with dimension $D = 2$ (full lines) or 8 (dotted). Two dimensions already yield a fit with $R^2 = 0.99$ for aligned dynamics (top), but almost no output for oblique (bottom, $R^2 = 0.005$, no arrows shown). For the latter, a good fit with $R^2 > 90\%$ is only reached with $D = 8$.

Studying the same task means that the output z is the same, so it is instructive to express it in terms of the correlation ρ :

$$z(t) = \rho(t) \|\mathbf{w}_{\text{out}}\| \|\mathbf{x}(t)\|. \quad (3)$$

Recent work on feed-forward networks showed that $\|\mathbf{w}_{\text{out}}\|$ can have a large effect on the resulting representations [9, 27]. Equation (3) shows that ρ is indeed linked to $\|\mathbf{w}_{\text{out}}\|$, but $\|\mathbf{x}(t)\|$ can also vary. In a detailed analysis in the Methods (Sections 4.7 to 4.9), we show that for recurrent networks, stability considerations preclude $\|\mathbf{x}(t)\|$ from being small. This implies that if we choose a readout norm $\|\mathbf{w}_{\text{out}}\|$ and then train the RNN on a given task, the correlation must compensate.

If we choose small output weights, we expect aligned dynamics, because a large correlation is necessary to generate sufficiently large output (**Fig. 1B** top). If instead we choose large output weights, we expect oblique dynamics, because only a small correlation keeps the output magnitude from growing too large.

We tested whether output weights can serve as a control knob to select dynamical regimes using an RNN model trained on an abstract version of the cycling task introduced in Ref. [61]. The networks were trained to generate a 2D signal that rotated in the plane spanned by two outputs $z_1(t)$ and $z_2(t)$ (**Fig. 2A**). An input pulse at the beginning of each trial indicated the desired direction of rotation. We set up two models with either small or large output weights and trained the recurrent weights of each with gradient descent (Section 4.1).

After both models learned the task, we projected the network activity into a three-dimensional space spanned by the two largest PCs of the dynamics $\mathbf{x}(t)$. A third direction, $\mathbf{w}_{\text{out},\perp}$, spanned the remaining part of the first output vector $\mathbf{w}_{\text{out},1}$. The resulting plots, **Fig. 2B**, corroborate our hypothesis: Small output weights led to aligned dynamics with a large correlation between the largest PCs and the output weights. In contrast, the large output weights of the second network were almost orthogonal, or oblique, to the two leading PCs. Further qualitative differences between the two solutions in terms of the direction of trajectories will be discussed below.

Another way to quantify these regimes is by the ability to reconstruct the output from the large PCs of neural activity, as quantified by the coefficient of determination R^2 . For the aligned network, the projection on the two largest PCs (**Fig. 2C**, solid) already led to a good reconstruction. For the oblique networks, the two largest PCs were not sufficient. We needed the first eight dimensions (**Fig. 2C**, dashed) to obtain a good reconstruction ($R^2 > 0.9$). In contrast to the differences in these fits, the neural dynamics themselves were much more similar between the networks. Specifically, 90% of the variance was explained by 4 and 5 dimensions for the aligned and oblique networks, respectively.

Can we use the output weights to induce aligned or oblique dynamics in more general settings? We trained RNN models with small or large initial output weights on five different neuroscience tasks (Section 4.2). All weights (input, recurrent, and output) were trained using the Adam algorithm (Section 4.1). After training, we measured the three quantities of **Eq. (3)**: magnitudes of neural activity and output weights, and the correlation between the two. The results in **Fig. 3A** show that across tasks, initialization with large output weights led to oblique dynamics (small correlation), and with small output weights to aligned dynamics (large correlation). While training could, in principle, change the initially small output weights to large ones (and vice versa), we noticed that this does not happen. Small output weights did increase with training, but the large gap in norms remained. This shows that setting the output weights at initialization can serve to determine their scale after learning under realistic settings. While explaining this observation is

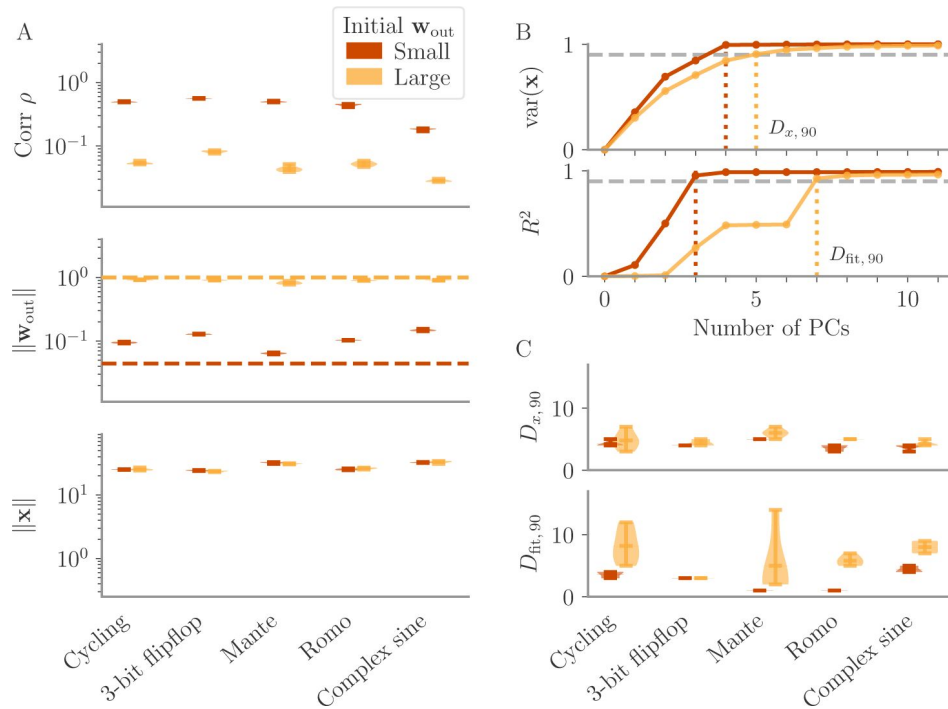


Fig. 3

The magnitude of output weights determines regimes across multiple neuroscience tasks [41, 59, 61, 76]. **A** Correlation and norms of output weights and neural activity. For each task, we initialized networks with small or large output weights (dark vs light orange). The initial norms $\|\mathbf{w}_{\text{out}}\|$ are indicated by the dashed lines. Learning only weakly changes the norm of the output weights. Note that all y-axes are logarithmically scaled. **B** Variance of $\mathbf{x}$ explained and R^2 of reconstructed output for projections of $\mathbf{x}$ on increasing number of PCs. Results from one example network trained on the cycling task are shown for each condition. **C** Number of PCs necessary to reach 90% of the variance of $\mathbf{x}(t)$ or of the R^2 of the output reconstruction (top/bottom; dotted lines in **B**). In **A**, **C**, violin plots show the distribution over 5 sample networks, with vertical bars indicating the mean and the extreme values (where visible).

beyond the scope of this work, we note that (1) changing the internal weights suffices to solve the task, and (2) the extent to which the output weights change during learning depends on the algorithm and specific parametrization [21, 27, 85].

In Fig. 3B-C, we adopted the perspective of Fig. 2C and quantified how well we can reconstruct the output from a projection of $\mathbf{x}$ onto its largest D PCs (Section 4.4). As expected, both the variance of $\mathbf{x}$ explained and the quality of output reconstruction increased, for an increasing number of PCs D (Fig. 3B). How both quantities increased, however, differs between the two regimes. While the variance explained increased similarly in both cases, the quality of the reconstruction increased much more slowly for the model with large output weights. We quantified this phenomenon by comparing the dimensions at which either the variance of $\mathbf{x}$ explained or R^2 reaches 90%, denoted by $D_{x,90}$ and $D_{\text{fit},90}$, respectively.

In Fig. 3C, we compare $D_{x,90}$ and $D_{\text{fit},90}$ across multiple networks and tasks. Generally, larger output weights led to larger numbers for both. However, for large output weights, the number of PCs necessary to obtain a good reconstruction increased much more drastically than the dimension of the data. Thus, the output was less well represented by the large PCs of the dynamics for networks with large output weights, in agreement with our notion of oblique dynamics.

Importantly, reaching the aligned and oblique regimes relies on ensuring robust and stable dynamics, which we achieve by adding noise to the dynamics during training. This yields a similar magnitude of neural activity $\|\mathbf{x}\|$ across networks and tasks (Fig. 3A). We show in Methods, Section 4.7, that learning in simple, noise-free conditions with large output weights can lead to solutions not captured by either aligned or oblique dynamics; those solutions, however, are unstable. Furthermore, we observed that some of the qualitative differences between aligned and oblique dynamics are less pronounced if we initialized networks with small recurrent weights and initially decaying dynamics (Fig. 16).

2.3. Neural dynamics decouple from the output for the oblique regime

What are the functional consequences of the two regimes? A hint might be seen in an intriguing qualitative difference between the aligned and oblique solutions for the cycling task in Fig. 2. For the aligned network, the two trajectories for the two different contexts (green and purple) are counterrotating (Fig. 2B, top). This agrees with the output, which also counter-rotates as demanded by the task (Fig. 2A). In contrast, the neural activity of the oblique network *co-rotates* in the leading two PCs (Fig. 2B, bottom). This is despite the counter-rotating output, since this network also solves the task (not shown). The co-rotation also indicates why reconstructing the output from the leading two PCs is not possible (Fig. 2C). Naturally, the dynamics also contain counter-rotating trajectories for producing the correct output, but these are only present in low-variance PCs. (Note also that for aligned networks, one can also observe co-rotation in low-variance PCs, see Fig. 17.) Taken together, aligned and oblique dynamics differ in the coupling between leading neural dynamics and output. For aligned dynamics, the two are strongly coupled. For oblique dynamics, the two decouple qualitatively.

Such a decoupling for oblique, but not aligned, dynamics leads to a prediction regarding the universality of solutions [39, 49, 79]. For aligned dynamics, the coupling implies that the internal dynamics are strongly constrained by the task. We thus expect different learners to converge to similar solutions, even if their initial connectivity is random and unstructured. In Fig. 4A, we show the dynamics of three randomly initialized aligned networks trained on the cycling task, projected onto the three leading PCs. Apart from global rotations, the dynamics in the three networks are very similar.

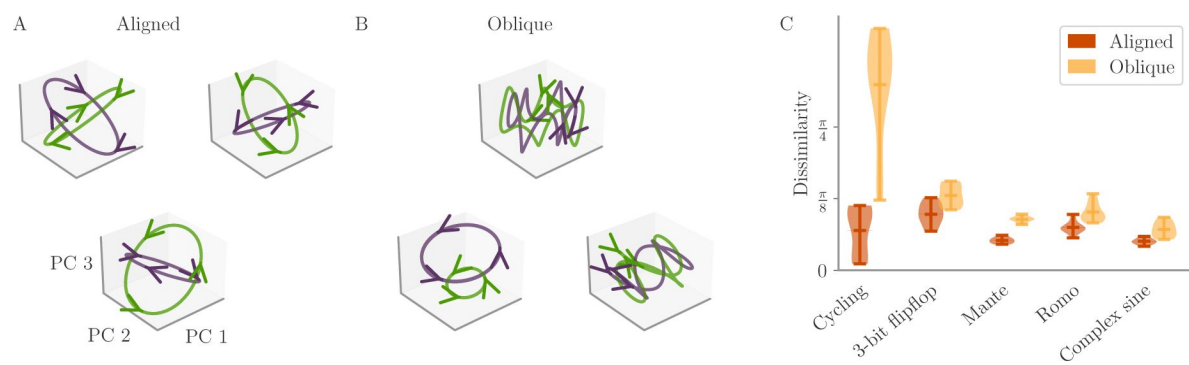


Fig. 4

Variability between learners for the two regimes. **A-B** Examples of networks trained on the cycling task with small (aligned) or large (oblique) output weights. The top left and central networks, respectively, are the same as those plotted in **Fig. 2**. **C** Dissimilarity between solutions across different tasks. Aligned dynamics (red) were less dissimilar to each other than oblique ones (yellow). The violin plots show the distribution over all possible different pairs for five samples (mean and extrema as bars).

For oblique dynamics, the task-defined output exerts weaker constraints on the internal dynamics. Any variability experienced during learning can potentially build up, and eventually create qualitatively different solutions. Three examples of oblique networks solving the cycling tasks indeed show visibly different dynamics (**Fig. 4B**). Further analysis shows that the models also differ in the frequency components in the leading dynamics (**Fig. 18**).

The degree of variability between learners depends on the task. The differences observable in the PC projections were most striking for the cycling task. For the flipflop task, for example, solutions were generally noisier in the oblique regime than in the aligned but did not have observable qualitative differences in either regime (**Fig. 19**). We quantified the difference between models for the different neuroscience tasks considered before. To compare different neural dynamics, we used a dissimilarity measure invariant under rotation (Section 4.5) [83]. The results are shown in **Fig. 4C**. Two observations stand out: First, across tasks, the dissimilarity was higher for networks in the oblique regime than for those in the aligned. Second, both overall dissimilarity and the discrepancy between regimes differed strongly between tasks. The largest dissimilarity (for oblique dynamics) and the largest discrepancy between regimes was found for the cycling. The smallest discrepancy between regimes was found for the flipflop task. Such a difference between tasks is consistent with the differences in the range of possible solutions for different tasks, as reported in Refs. [39, 79].

What are the underlying mechanisms for the qualitative decoupling in oblique, but not aligned networks? For aligned dynamics, we saw that the small output weights demand large activity to generate the output. In other words, the activity along the largest PCs must be coupled to the output. For oblique dynamics, this constraint is not present, which opens the possibility for small components outside the largest PCs to generate the output. If this is the case, we have a decoupling, such as the observed co-rotation in the cycling task, and the possible variability between solutions. We discuss this point in more detail in the Methods, Section 4.10.

In the following two sections, we will explore how the decoupling between neural dynamics and output for oblique, but not aligned, dynamics influences the response to perturbations and the effects of noise during learning.

2.4. Differences in response to perturbations

Understanding how networks respond to external perturbations and internal noise requires some insight into how dynamics are generated. Dynamics of trained networks are mostly generated internally, through recurrent interactions. In robust networks, these internally generated dynamics are a prominent part of the largest PCs (among input-driven components; Sections 4.7 and 4.9). Internally-generated dynamics are sustained by positive feedback loops, through which neurons excite each other. Those loops are low-dimensional, with activity along a few directions of the dynamics being amplified and fed back along the same directions. This results in dynamics being driven by effective feedback loops along the largest PCs (**Fig. 5A**). As shown above, the largest PCs can either be aligned or not aligned, with the output weights. This leads to predictions about how aligned and oblique networks differentiate in their responses to perturbations along different directions.

Our intuition about feedback loops suggests that networks respond strongly to a perturbation that is aligned with the directions contributing to the feedback loop, but weakly to a perturbation that is orthogonal to them. In particular, if a perturbation is applied along the output weights, aligned and oblique dynamics should dissociate, with a strong disruption of dynamics for aligned, but not for oblique dynamics (**Fig. 5A**).

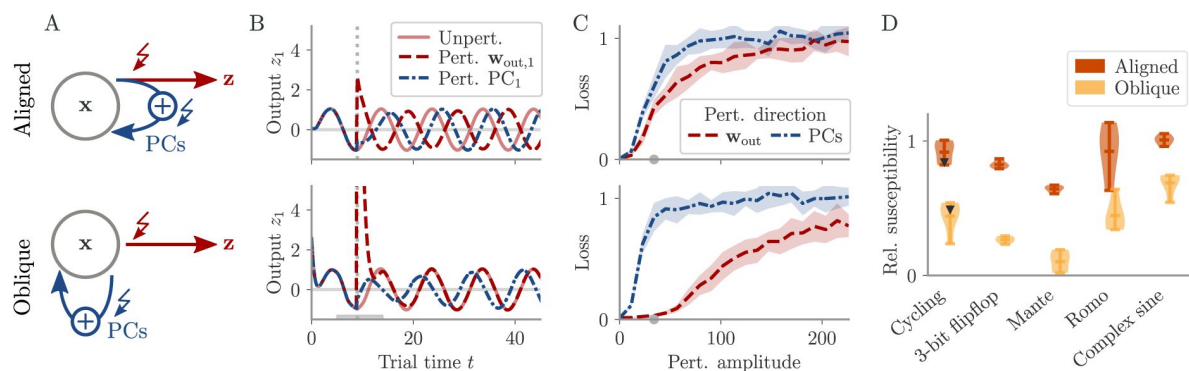


Fig. 5

Perturbations differentially affect dynamics in the aligned and oblique regimes. **A** Cartoon illustrating the relationship between perturbations along output weights or PCs and the feedback loops driving autonomous dynamics. **B** Output after perturbation for aligned (top) and oblique (bottom) networks trained on the cycling task. The unperturbed network (light red line) yields a sine wave along the first output direction z_1 . At $t_p = 9$, a perturbation with amplitude $\|\Delta x\| = 34$ is applied along the output weights (dashed red) or the first PC (dashed-dotted blue). The perturbations only differ in the directions applied. While the immediate response for the oblique network to a perturbation along the output weights is much larger, $z_1(t_p) \approx 80$, the long-term dynamics yield the same output as the unperturbed network. See also [Fig. 20](#) for more details. **C** Loss for perturbations of different amplitudes for the two networks in **B**. Lines and shades are means and std devs over different perturbation times $t_p \in [5, 15]$ and random directions spanned by the output weights (red) or the two largest PCs (blue). The loss is the mean squared error between output and target for $t > 20$. The gray dot indicates an example in **B**. **D** Relative susceptibility of networks to perturbation directions for different tasks and dynamical regimes. We measured the area under the curve (AUC) of loss over perturbation amplitude for perturbations along the output weights of the two largest PCs. The relative susceptibility is the ratio between the two AUCs. The example in **C** is indicated by gray triangles.

To test this, we compare the response to perturbations along the output direction and the largest PCs. We apply perturbations to the neural activity at a single point in time: $\mathbf{x}(t)$ evolves undisturbed until time t_p . At that point, it is shifted to $\mathbf{x}(t_p) + \Delta\mathbf{x}$. After the perturbation, we let the network evolve freely and compare this evolution to that of an unperturbed copy. Such a perturbation mimics a very short optogenetic perturbation applied to a selected neural population [14, 46]. In Fig. 5B, we show the output after such perturbations for an aligned (top) and an oblique network (bottom) trained on the cycling task. The time point and amplitude are the same for both directions and networks. For each network and type of perturbation, there is an immediate deflection and a long-term response. For both networks, perturbing along the PCs (blue) leads to a long-term phase shift. Only in the aligned network, however, perturbation along the output direction (red) leads to a visible long-term response. In the oblique network, the amplitude of the immediate response is larger, but the long-term response is *smaller*. Our results for the oblique network, but not for the aligned, agree with simulations of networks generating EMG data from the cycling experiments [65].

To quantify the relative long-term susceptibility of networks to perturbations along output weights or PCs, we sampled from different times t_p and different directions in the 2D subspaces spanned either by the two output vectors or the two largest PCs. For each perturbation, we measured the loss of the perturbed networks on the original task (excluding the immediate deflection after the perturbation by starting to compute the loss at $t_p + 5$). Fig. 5C shows that the aligned network is almost equally susceptible to perturbations along the PCs and the output weights. In contrast, the oblique network is much more susceptible to perturbations along the PCs.

We repeated this analysis for oblique and aligned networks trained on the five different tasks. We computed the area under the curve (AUC) for both loss profiles in Fig. 5C. We then defined the “relative susceptibility” as the ratio $AUC_{\text{wout}}/AUC_{\text{PC}}$, Fig. 5D. For aligned networks (red), the relative susceptibility was close to 1 indicating similarly strong responses to both types of perturbations. For oblique networks (yellow), it was much smaller than 1, indicating that long-term responses to perturbations along the output direction were weaker than those to perturbations along the PCs.

2.5. Noise suppression for oblique dynamics

In the oblique regime, the output weights are large. To produce the correct output (and not a too large one), the large PCs of the dynamics are almost orthogonal to the output weights. The large output weights, however, pose a robustness problem: Small noise in the direction of the output weights is also amplified at the level of the readout. We show that learning leads to a slow process of sculpting noise statistics to avoid this effect (Fig. 11). Specifically, a negative feedback loop is generated that suppresses fluctuations along the output direction (Fig. 6A, Fig. 10) [29]. Because the positive feedback loop that gives rise to the large PCs is mostly orthogonal to the output direction, it remains unaffected by this additional negative feedback loop. A detailed analysis of how learning is affected by noise shows that, for large output weights, the network first learns a solution that is not robust to noise. This solution is then transformed to increasingly stable and oblique dynamics over longer time scales (Sections 4.8 and 4.9).

To illustrate the effect of the negative feedback loop, we consider the fluctuations around trial averages. We take a collection of states $\mathbf{x}(t)$ and then subtract the task-conditioned averages

$\bar{\mathbf{X}}(t)$ to compute $\delta\mathbf{x}(t) = \mathbf{x}(t) - \bar{\mathbf{X}}(t)$. We then project $\delta\mathbf{x}(t)$ onto three different direction

categories: the largest PCs of the averaged data $\bar{\mathbf{X}}(t)$, the output directions, or randomly drawn directions.

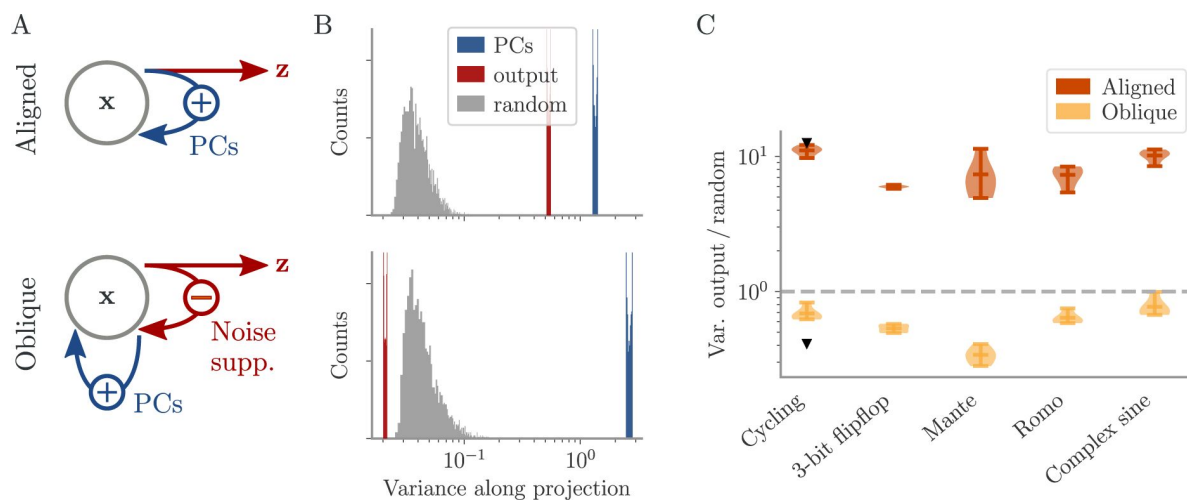


Fig. 6

Noise suppression along the output direction in the oblique regime. **A** A cartoon of the feedback loop structure for aligned (top) and oblique (bottom) dynamics. The latter develops a negative feedback loop which suppresses fluctuations along the output direction. **B** Comparing the distribution of variance of mean-subtracted activity along different directions for networks trained on the cycling task (see Fig. 21): PCs of trial-averaged activity (blue), readout (red), and random (grey) directions. For the PCs and output weights, we sampled 100 normalized combinations of either the first two PCs or the two output vectors. For the random directions, we drew 1000 random vectors in the full, N -dimensional space. **C** Noise compression across tasks as measured by the ratio between variance along output and random directions. The dashed line indicates neither compression nor expansion. Black markers indicate the values for the two examples in B-C. Note the log-scales in B-C.

How strongly the activity fluctuates along each direction is quantified by the variance of the projections (**Fig. 6B**). For both aligned and oblique dynamics, the variance is much larger along the PCs than along random directions. This is not necessarily expected, because the PCA was performed on the *averaged* activity, without the fluctuations. Instead, it is a dynamical effect: the same positive feedback that generates the autonomous dynamics also amplifies the noise (Section 4.9).

The two network regimes, however, dissociate when considering the variance along the output direction. For aligned dynamics, there is no negative feedback loop, and *wout* is correlated with the PCs. The variance along the output direction is hence similar to that along the PCs, and larger than along random directions. For oblique dynamics, the negative feedback loop suppresses the fluctuations along the output direction, so that they become weaker than along random directions.

In **Fig. 6C**, we quantify this dissociation across different tasks. We measured the ratio between variance along output and random directions. Aligned networks have a ratio much larger than one, indicating that the fluctuations along the output direction are increased due to the autonomous dynamics along the PCs. In contrast, oblique networks have a ratio smaller than 1 for all tasks, which indicates noise compression along the output.

2.6. Different degrees of alignment in experimental settings

For the cycling task, we observed that dynamics were qualitatively different for the two regimes, with trajectories either counter- or co-rotating (**Fig. 2B**). Interestingly, the experimental results of Russo et al. [61] matched the oblique, but not the aligned dynamics. The authors observed co-rotating dynamics in the leading PCs of motor cortex activity despite counter-rotating activity of simultaneously recorded muscle activity. Here we test whether our theory can help to more clearly and quantitatively distinguish between the two regimes in experimental settings.

In typical experimental settings, we do not have direct access to output weights. We can, however, approximate these by fitting neural data to simultaneously recorded behavioral output, such as hand velocity in a motor control experiment (**Fig. 7A** top). Following the model above, where the output is a weighted average of the states, we reconstruct the output from the neural activity with linear regression. To quantify the dynamical regime, we then compute the correlation ρ between the weights from fitting and the neural data. Additionally, we can also quantify the alignment by the “relative fitting dimension” $D_{\text{fit},90}/D_{x,90}$, where $D_{\text{fit},90}$ is the number of PCs necessary to recover the output and $D_{x,90}$ number to the number of PCs necessary to represent 90% of the variance of the neural data. We computed both the correlation and the relative fitting dimension for different publicly available data sets (**Fig. 7B**). For details, see Section 4.6.

We started with data sets from two monkeys performing the cycling task [61]. The data contained motor cortex activity, hand movement, and EMG from the arms, all averaged over multiple trials of the same condition. In **Fig. 7B**, we show results for reconstructing the hand velocity. The correlation was small, $\rho \in [0.04, 0.07]$. To obtain a good reconstruction, we needed a substantial fraction of the dimension of the neural data: The relative fitting dimension was $D_{\text{fit},90}/D_{x,90} \in [0.7, 0.8]$. Our results agree with previous studies, showing that the best decoding directions are only weakly correlated with the leading PCs of motor cortex activity [66].

We also analyzed data made available through the Neural Latents Benchmark (NLB) [52]. In two different tasks, monkeys needed to perform movements along a screen. In a random target task (RTT), the monkeys had to point at a randomly generated target position on a screen, with a successive target point generated once the previous one was reached [40]. In a maze task, the monkeys were trained to follow a trajectory through a maze with their hand [10]. In both cases, we reconstructed the finger or hand velocity from neural activity on single trials. The correlation was higher than in the cycling task, $\rho = 0.13$. The relative fitting dimension was lower than in the trial-averaged cycling data, albeit still on the same order: $D_{\text{fit},90}/D_{x,90} \in [0.4, 0.5]$.

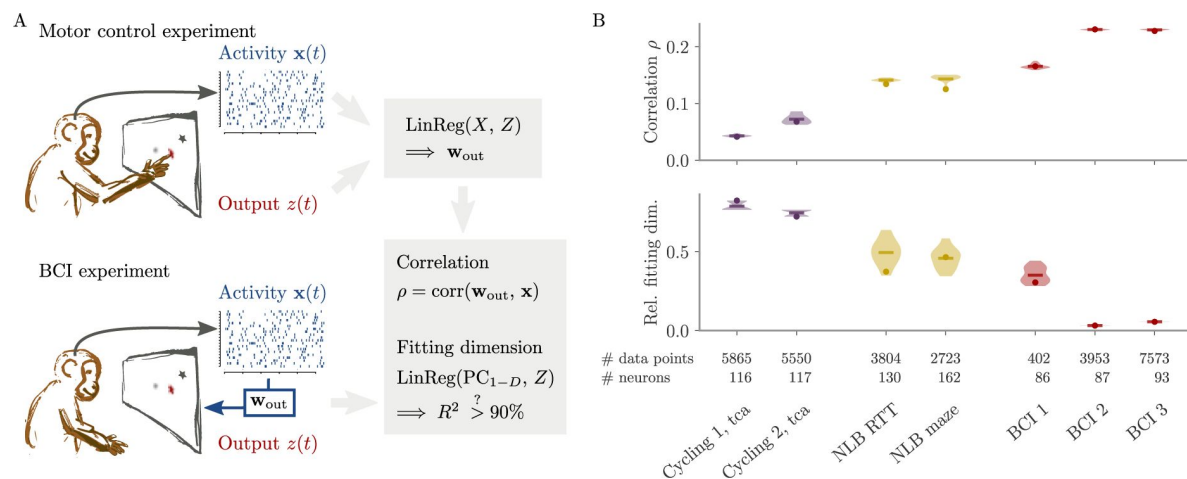


Fig. 7

Quantifying aligned and oblique dynamics in experimental data [12, 22, 25, 52, 61]. **A** Diagram of the two types of experimental data considered. Here we always took the velocity as the output (hand, finger, or cursor). In motor control experiments (top), we first needed to obtain the output weights $\mathbf{w}_{\text{out}}$ via linear regression. We then computed the correlation ρ and the reconstruction dimension $D_{\text{fit},90}$, i.e. the number of PCs of $\mathbf{x}$ necessary to obtain a coefficient of determination $R^2 > 90\%$. In BCI experiments (bottom), the output (cursor velocity) is generated from neural activity $\mathbf{x}(t)$ via output weights $\mathbf{w}_{\text{out}}$ defined by the experimenter. This allowed us to directly compute correlations and fitting dimensions. **B** Correlation ρ (top) and relative fitting dimension $D_{\text{fit},90}/D_{x,90}$ (bottom) for several publicly available data sets. The cycling task data (purple) were trial-conditioned averages, the BCI experiments (red) and NLB tasks (yellow) single-trial data. Results for the full data sets are shown as dots. Violin plots indicate results for 20 random subsets of 25% of the data points in each data set (bars indicate mean). See small text below x-axis for the number of time points and neurons.

Finally, we considered brain-computer interface (BCI) experiments [63]. In these experiments, monkeys were trained to control a cursor on a screen via activity read out from their motor cortex (Fig. 7A bottom). The output weights generating the cursor velocity were set by the experimenter (so we don't need to fit). Importantly, the output weights were typically chosen to be spanned by the largest PCs (the “neural manifold”), suggesting aligned dynamics. For three example data sets [12, 22, 25], we obtained higher correlation values, $\rho \in [0.17, 0.23]$ (Fig. 7B). The relative fitting dimension was much smaller than for the non-BCI data sets, especially for the two largest data sets, where $D_{\text{fit},90}/D_{x,90} \in [0.03, 0.06]$.

The higher correlation and much smaller relative fitting dimension suggest that, indeed, the neural dynamics arising in BCI experiments are more aligned, and those in non-BCI settings are more oblique. These trends also hold when decoding other behavioral outputs for the cycling task and the NLB tasks (position, acceleration, or EMG), even if the ability to decode and the numerical values for correlation and fitting dimension may fluctuate considerably (Fig. 22). Thus, while we do not observe strongly different regimes as in the simulations, we do see an ordering between different data sets according to the alignment between outputs and neural dynamics. It would be interesting to test the differences between BCI and non-BCI data on larger data sets, and different experiments with different dimensions of neural data [20, 71].

3. Discussion

We analyzed the relationship between neural dynamics and behavior, asking to which extent a network's output is represented in its dynamics. We identified two different limiting regimes: aligned dynamics, in which the dominant activity in a network is related to its output, and oblique dynamics, where the output is only a small modulation on top of the dominating dynamics. We demonstrated that these two regimes have different functional implications. We also examined how they arise through learning, and how they relate to experimental findings.

Linking neural activity to external variables is one of the core challenges of neuroscience [26]. In most cases, however, such links are far from perfect. The activity of single neurons can be related in a nonlinear, mixed manner, to task variables [56]. Even when considering populations of neurons, a large fraction of neural activity is not easily accounted for by external variables [1]. Various explanations have been proposed for this disconnect. In the visual cortex, activity has been shown to be related to “irrelevant” external variables, such as body movements [72]. Follow-up work showed that, in primates, some of these effects can be explained by the induced changes on retinal images [78], but this study still explained only half of the neural variability. An alternative explanation hinges on the redundancy of the neural code, which allows “null spaces” in which activity can visit without affecting behavior [30, 31, 58]. Through the oblique regime, our study offers a simple explanation for this phenomenon: in the presence of large output weights, resistance to noise or perturbations requires large, potentially task-unrelated neural dynamics. Conversely, generating task-related output in the presence of large, task-unrelated dynamics requires large readout weights.

We showed theoretically and in simulations that, when training recurrent neural networks, the magnitude of output weights is a central parameter that controls which regime is reached. This finding is vital for the use of RNNs as hypothesis generators [3, 74, 80], where it is often implicitly assumed that training results in universal solutions [39] (even though biases in the distribution of solutions have been discussed [77]). Here, we show that a specific control knob allows one to move between qualitatively different solutions of the same task, thereby expanding the control over the hypothesis space [49, 79]. Note in particular that the default initialization in standard learning frameworks has large output weights, which results in oblique dynamics (or unstable solutions if training without noise, see Methods, Section 4.7).

The role of the magnitude of output weights is also discussed in machine learning settings, where different learning regimes have been found [9, 27, 28, 43]. In particular, “lazy” solutions were observed for large output weights in feed-forward networks. We show in Methods, Section 4.7, that these are unstable for recurrent networks and are replaced in a second phase of learning by oblique solutions. This second, slower, phase is reminiscent of implicit regularization in overparameterized networks [5, 35, 55, 86]. On a broader scale, which learning regime is relevant when modeling biological learning is an open question that has only just begun to be explored [15, 37].

The particular control knob we studied has an analog in the biological circuit - the synaptic weights. We can thus use experimental data to study whether the brain might rely on oblique or aligned dynamics. Existing experimental work has partially addressed this question. In particular, the work by Russo et al. [61] has been a major inspiration for our study. Our results share some of the key findings from that paper - the importance of stability leading to “untangled” dynamics [73] and a dissociation between hidden dynamics and output. In addition, we suggest a specific mechanism to reach oblique dynamics - training networks with large output weights. Furthermore, we characterize the aligned and oblique regimes along experimentally accessible axes.

We see three avenues for exploring our results experimentally. First, simultaneous measurements of neural dynamics and muscle activity could be used to quantify noise along the output direction. This would allow checking whether noise is compressed in this direction, and in particular, whether such compression occurs on a slow timescale after initial task acquisition. We suggest how to test this in Fig. 6C. Second, we show how the dynamical regimes dissociate under perturbations along specific directions. Experiments along these lines have recently become possible [8, 14, 60]. Future work is left to combine our model with biological constraints that induce additional effects during perturbations, e.g., through non-normal synaptic connectivity [6, 32, 38, 46]. Third, our work connects to the setting of brain-computer interfaces, where the experimenter chooses the output weights at the beginning of learning [22, 54, 63, 82]. Typically, the output weights are set to lie “within the manifold” of the leading PCs so that we expect aligned dynamics [63]. In experiments where the output weights were rotated out of the manifold (without changing the norm), learning took longer and led to a rotation of the manifold, i.e. at least a partial alignment [47]. Our theory suggests directly comparing the degree of alignment between dynamics obtained from within- and out-of-manifold initializations. Furthermore, it would be interesting to systematically change the norm of the output weights (in particular for out-of-manifold initializations) to see whether larger output weights lead to more oblique solutions. If this is the case, we suggest testing whether such more oblique solutions meet our predictions, e.g. higher variability between individuals and noise suppression.

Overall, our results provide an explanation for the plethora of relationships between neural activity and external variables. It will be interesting to see whether future studies will find hallmarks of either regime for different experiments, tasks, or brain regions.

4. Methods

4.1. Details on RNN models and training

We consider rate-based RNNs with N neurons. The states $\mathbf{x}(t) \in \mathbb{R}^N$ are governed by

$$\dot{\mathbf{x}} = -\mathbf{x} + W\phi(\mathbf{x}) + W_{\text{in}}\mathbf{s}(t) + \boldsymbol{\xi}(t), \quad (4)$$

Parameter	Symbol	Cycling	Flip-flop	Mante	Romo	Complex Sine
# inputs	N_{in}	2	3	4	1	1
# outputs	N_{out}	2	3	1	1	1
Trial duration	T	72	25	48	29	50
Fixation duration	t_{fix}	0	$\mathcal{U}(0, 1)$	3	$\mathcal{U}(1.3)$	0
Stimulus duration	t_{stim}	1	1	20	1	50
Stimulus delay	t_{sd}	-	$\mathcal{U}(3, 10)$	-	$\mathcal{U}(2, 12)$	-
Decision delay	t_{delay}	1	2	5	4	0
Decision duration	t_{dec}	71	t_{sd}	20	8	50
Simulation time step	Δt			- 0.2 -		
Target time step	Δt_{target}	1.0	1.0	1.0	1.0	0.2
Activation noise	$\sqrt{2}\sigma_{\text{noise}}$	0.2	0.2	0.05	0.2	0.2
Initial state noise	σ_{init}			- 1.0 -		
Network size	N			- 512 -		
# training epochs		1000	4000	4000	6000	6000
Learning rate aligned	η_0	0.02	0.005	0.002	0.005	0.005
Learning rate oblique	η_0	0.02	0.01	0.02	0.01	0.005
Batch size				- 32 -		

Table 1

Task, simulation, and network parameters for **Figs. 3** to **6**

where W is a recurrent weight matrix and $\phi = \tanh$ a nonlinearity applied element-wise. The network receives a low-dimensional input $\mathbf{s}(t) \in \mathbb{R}^{N_{\text{in}}}$ via input weights W_{in} . It is also driven by white, isotropic noise with zero mean and covariance $E[\xi_i(t)\xi_j(t')] = 2\sigma_{\text{noise}}^2\delta_{ij}\delta(t-t')$. The initial states $\mathbf{x}(0)$ are drawn from a centered normal distribution with variance σ_{init}^2 at each trial. This serves as additional noise. The output is a low-dimensional, linear projection of the states: $\mathbf{z}(t) = W_{\text{out}}\mathbf{x}(t)$ with $W_{\text{out}} = [\mathbf{w}_{\text{out},1}, \dots, \mathbf{w}_{\text{out},N_{\text{out}}}]^T$.

The initial output weights are drawn from centered normal distributions with variance σ_{out}^2/N . “Small” output weights refer to $\sigma_{\text{out}} = 1/\sqrt{N}$, and “large” ones to $\sigma_{\text{out}} = 1$. We have $\|\mathbf{w}_{\text{out},i}\| = \sigma_{\text{out}}[1 + O(1/\sqrt{N})]$ at initialization. Note that large initial output weights are the current default in standard learning environments [50, 85]. The recurrent weights were initialized from centered normal distributions with variance g^2/N . We chose $g = 1.5$ so that dynamics were chaotic before learning [70].

To simulate the noisy RNN dynamics numerically, we used the Euler-Maruyama method [34] with a time step of Δt . We used the Adam algorithm [33] implemented in PyTorch [50]. Apart from the learning rate, we kept the parameters for Adam at the default (some filtering, no weight decay). We selected learning rates and the number of training steps such that learning was relatively smooth and converged sufficiently within the given number of trials. Learning rates were set to $\eta = \eta_0/N$. Details for all simulation parameters can be found in **Table 1**.

For the comparisons over different tasks (Figs. 3 to 6), we trained 5 networks for each task. All weights (W_{out} , W , W_{in}) were adapted. For the example networks trained on the cycling task (Figs. 2, 5 and 6), we used networks with $N = 256$ neurons and only changed the recurrent weights W . We also trained for longer (5000 training steps) and with a higher learning rate ($\eta = 0.1/N$).

4.2. Task details

The tasks the networks were trained on are taken from the neuroscience literature: a cycling task [61], a 3-bit flip-flop task and a “complex sine” task (with input-dependent frequencies) [76], a context-dependent decision making task (“Mante”) [41, 68], and a working memory task comparing the amplitudes of two pulse stimuli (“Romo”) [59, 68]. All tasks have similar structure [69]: A trial of length T starts with a fixation period (length t_{fix}). This is followed by an input for t_{stim} . For the cycling and flip-flop task, the inputs are pulses of amplitude 1; else see below. After a delay t_{delay} , the output of the network is required to reach an input-dependent value during a time period t_{dec} . During this decision period, we set target points t_i every Δt_{target} time steps. The loss was defined as the mean squared error between network output and target at these time points. Below we provide further details for each task.

Cycling task

The network receives an initial pulse, whose direction ($[1, 0]^T$ or $[0, 1]^T$) determines the sense of direction of the target. The target is given by a rotation in 2D, $\hat{\mathbf{z}}(t) = [\sin(2\pi ft), \cos(2\pi ft)]^T$, with frequency $f = 0.1$ and $a = \pm 1$ for the two directions (clockwise or anticlockwise).

Flip-flop task

The network repeatedly receives input pulses along one of 3 directions, followed by decision periods. In each decision period, the output coordinate corresponding to the last input should reach ± 1 depending on the sign of the input. All other coordinates should remain at ± 1 as defined by the last time they were triggered. To make sure that this is well-defined, we trigger all inputs at time steps $k\Delta t$ for $k \in [N_{\text{in}}]$ with random signs.

Mante task

Input channels for this task are split into two groups of size $N_{\text{in}}/2$: half of the channels for the signal and the other half for the context, indicating which of the signal channels is relevant. All signal channels $s_i(t)$ deliver a constant mean $\hat{s}_i$ plus additional white noise: $s_i(t) = \hat{s}_i + a_{\text{noise}}\eta_i(t)$. The mean is drawn uniformly from $\{\pm 1, \pm \frac{1}{2}, \pm \frac{1}{4}, \pm \frac{1}{8}\}$, and the noise amplitude is $a_{\text{noise}} = 0.05$. For simulations, we draw a standard normal variable $n_{i,k} \sim N(0, 1)$ at time step k , and set $\eta_{i,k} = n_{i,k}/\sqrt{\Delta t}$. Only a single contextual input is active at each trial, $s_{i+N_{\text{in}}/2} = \delta_{ij}$, with j chosen uniformly from the number of context $N_{\text{in}}/2$. The target during the decision period is the sign of the relevant input, $\hat{z}(t) = \text{sign}(\hat{s}_j)$.

Romo task

For the Romo task, the input consists of two input pulses separated by random delays t_{sd} . The amplitude of the inputs is drawn independently from $U(0.5, 1.5)$ with the condition of being at least 0.2 apart (else both are redrawn). During the decision period, the network needs to yield $\hat{z}(t) = \pm 1$, depending on which of the two pulses was larger.

Complex Sine

The target is $\hat{z}(t) = \sin(2\pi ft)$, with frequency $f = (1 - a)f_{\text{min}} + af_{\text{max}}$, and boundaries $f_{\text{min}} = 0.04, f_{\text{max}} = 0.2$, and where $a \sim U(0, 1)$. The input is a constant input of amplitude $s = a + 0.25$.

4.3. Generalized correlation

For [Fig. 3A](#), we used a generalized correlation measure which allows for multiple output dimensions, multiple time points, and noisy data. Consider neural activity of N neurons at P time points stacked into the matrix $X = [\mathbf{x}(1), \dots, \mathbf{x}(P)] \in \mathbb{R}^{N \times P}$. We assume the states to be centered in time, $\frac{1}{P} \sum_{t=1}^P x_i(t) = 0$ for $i \in [N]$. The corresponding D -dimensional output is summarized in the $D \times P$ matrix

$$Z = W_{\text{out}}^T X, \quad (5)$$

with weights $W_{\text{out}} \in \mathbb{R}^{N \times D}$. We define the generalized correlation as

$$\rho = \frac{\|W_{\text{out}}^T X\|}{\|W_{\text{out}}\| \|X\|}. \quad (6)$$

The norm is the Frobenius norm, $\|X\| = \sqrt{\sum_{ij} X_{ij}^2}$. In particular, we have

$$\|Z\| = \rho \|W_{\text{out}}\| \|X\|. \quad (7)$$

The case of one-dimensional output and a single time step discussed in the main text, [Eq. \(3\)](#), is recovered up to the sign, which we discard. Note that in that case, the vectors $\mathbf{w}_{\text{out}}$ and $\mathbf{x}(t)$ should be centered along coordinates to receive a valid correlation. Our numerical results did not change qualitatively when centering across coordinates only or both coordinates and time.

For trajectories with multiple conditions, we stack these instances in a matrix $X \in \mathbb{R}^{N \times N_c N_t}$, with N_c the number of conditions, and N_t the number of time points per trajectory. For noisy trajectories, we first average over multiple instances per condition and time point to obtain a

similar matrix $\bar{X}$.

4.4. Regression

In **Fig. 3B-C** we computed the number of PCs necessary to either represent the dynamics or fit the output. We simulated the trained networks again on their corresponding tasks. We did not apply noise during these simulations, since keeping the same noise as during training would reduce the quality of the output for large output weights; trial averaging yielded similar results to the ones obtained without noise (not shown).

The simulations yielded neural states $X \in \mathbb{R}^{N \times P}$ and outputs $Z \in \mathbb{R}^{N_{\text{out}} \times P}$, where P is the number data points (batch size times number of time points T). We applied PCA to the states X . The cumulative explained variance ratio obtained from PCA is plotted in **Fig. 3B**. We then projected X onto the first k PCs and fitted these projections to the output with ridge regression (cross-validated, using scikit-learn's RidgeCV [51]).

4.5. Dissimilarity measure

For measuring the dissimilarity between learners in **Fig. 4**, we apply a measure following Williams et al. [83]. We define the distance between two sets with P data points $X, Y \in \mathbb{R}^{N \times P}$ as

$$d(X, Y) = \arg \min_{Q \in \mathcal{O}} \arccos \frac{\text{Tr}(\hat{X} \hat{Y}^T Q)}{\|\hat{X}\| \|\hat{Y}\|}, \quad (8)$$

where the hat corresponds to centering along the rows, and Q is an orthogonal matrix. The solution to this so-called orthogonal Procrustes' problem is found via the singular value decomposition $\hat{X} \hat{Y}^T = U \Sigma V^T$. The optimal transformation is $Q^* = V U^T$, and the numerator in Eq. (8) is then $\text{Tr}(\hat{X} \hat{Y}^T Q^*) = \text{Tr}(\Sigma)$.

Note that this is more restricted than canonical correlation analysis (CCA), which is also commonly used [17, 18]. In particular, CCA involves whitening the matrices X and Y before applying a rotation [83]. This sets all singular values to 1. For originally low-D data, this mostly means amplifying the noise, unless the data was previously projected onto a small number of PCs. In the latter case, the procedure still removes the information about how much each PC contributes.

4.6. Experimental data

We detail the analyses of neural data in Section 2.6. We made use of publicly available data sets: data from the cycling task of Ref. [61], two data sets available through the Neural Latents Benchmark (NLB) [52], and data from monkeys trained on a center-out reaching task with a BCI [12, 22, 25]. For all data sets, we first obtain firing rates $X \in \mathbb{R}^{N \times T}$ where N is the number of measured neurons, and T the number of data points, (see **Fig. 7B** for these numbers). We also collect the simultaneously measured behavior in the matrix $Z \in \mathbb{R}^{D \times T}$. In **Fig. 7**, we only analyzed cursor or hand velocity for behavior, so that the output dimension is $D = 2$. See **Fig. 22** for similar results for hand position and acceleration or the largest two PCs of the EMG data recorded for the cycling task.

For the cycling task, the firing rates were binned in 1 ms bins and convolved with a 25 ms Gaussian filter. The mean firing rate was 22 and 18 Hz for the two monkeys, respectively. For the NLB data, spikes came in 1 ms bins. We binned data to 45 ms bins and applied a Gaussian filter with 45 ms width. This increased the quality of the fit, as firing rates were much lower (mean of 5 Hz for both) than in the cycling data set. For the BCI experiments, firing rates came as spike counts

in 45 ms bins. The mean firing rate was (45, 45, 55) Hz for the data of Refs. [12, 22, 25], respectively. In agreement with the original BCI experiments, we did not apply a filter to the neural data.

For fitting, we centered both firing rates X and output Z across time (but not coordinates). We also added a delay of 100 ms between firing rates and output for the cycling and NLB data sets, which increased the quality of the fits. We then fitted the output Z to the firing rates X with ridge regression, with regularization obtained from cross-validation. We treated the coefficients as output weights W_{out} . The trial average data of the cycling tasks was very well fitted for both monkeys, $R^2 = [0.97, 0.98]$. For the NLB tasks with single-trial data, the fits were not as good, $R^2 = [0.73, 0.69]$. For two of the BCI data sets [22, 25], the output weights were also given, and we checked that the fit recovers these. For the third BCI data set [12], we did not have access to the output weights, and only access to the cursor velocity after Kalman filtering. Here, fitting yielded $R^2 = 0.83$.

For the fitting dimension $D_{\text{fit},90}/D_{x,90}$ in Fig. 7B bottom, we used an adapted definition of $D_{\text{fit},90}$: Because $R^2 = 90\%$ is not reached for all data sets, we asked for the number of PCs necessary to obtain 90% of the R^2 value obtained for the full data set.

We also considered whether the correlation ρ scales with the number of neurons N . In our model, oblique and aligned dynamics can be defined in terms of such a scaling: aligned dynamics have highly correlated output weights and low-dimensional dynamics, so that $\rho \sim N^0 = 1$, i.e. independent of the network size. For oblique dynamics, large output weights with norm $\mathbf{w}_{\text{out}} \sim 1$ lead to vanishing correlation, $\rho \sim 1/\sqrt{N}$. This is indeed similar to the relation between

two random vectors, for which the correlation is precisely $1/\sqrt{N}$ (in the limit of large N). In

Fig. 8, we show the scaling of R^2 and ρ with the number of subsampled neurons. For the cycling task and NLB data, the correlation scaled slightly weaker than $\rho \sim N^{1/2}$. For the BCI data, the scaling was closer to $\rho \sim N^{-1/4}$ which is in between the aligned and oblique regimes of the model. These insights, however, are limited due to the trial averaging for the cycling task and the limited number of time points for the NLB tasks (not enough to reach $R^2 = 1$). Applying these measures to larger data sets could yield more definitive insights.

4.7. Analysis of solutions under noiseless conditions

In the sections below, we explore in detail under which conditions aligned and oblique solutions arise, and which other solutions arise if these conditions are not met.

We first consider small output weights and show that these lead to aligned solutions. Then, for large output weights, we show that without noise, two different, unstable solutions arise. Finally, we consider how adding noise affects learning dynamics. For a linear model, we can solve the dynamics of learning analytically and show how a negative feedback loop arises, that suppresses noise along the output direction. However, the linear model does not yield an oblique solution, so we also consider a nonlinear model for which we show in detail why oblique solutions arise.

We start by analyzing a simplified version of the network dynamics Eq. (4): autonomous dynamics without noise,

$$\dot{\mathbf{x}} = -\mathbf{x} + W\phi(\mathbf{x}), \quad (9)$$

with fixed initial condition $\mathbf{x}(0)$. We assume a one-dimensional output $z(t)$ and a target $\hat{z}(t_i)$ defined on a finite set of time points t_i .

We illustrate the theory with an example of a simple sine wave task (Fig. 9). We demand the network to autonomously produce a sine wave with fixed frequency $f = 0.1$. At the beginning of the task, the network receives an input pulse that sets the starting point of the trajectory. We set the noise σ_{init} on the initial state $\mathbf{x}(0)$ to zero. We define the target as 20 target points in the interval $t \in [1, 21]$ (two cycles; purple dots in Fig. 9).

4.7.1. Small weights lead to aligned solutions

For small output weights, $\|\mathbf{w}_{\text{out}}\| = 1/\sqrt{N}$, gradient-based learning in such a noise-less system has been analyzed by Schuessler et al. [69]. Learning changes the dynamics qualitatively through low-rank weight changes ΔW . These weight changes are spanned by existing directions such as the output weights. The resulting dynamics $\mathbf{x}(t)$ are thus aligned to the output weights. This means that the correlation between the two is large, independently of the network size, $\rho = O(1)$. The target of the task is also independent of N , so that after learning we have $z = O(1)$. Given the small output weights, we can thus infer the size of the states:

$$\underbrace{z}_{O(1)} = \underbrace{\rho}_{O(1)} \underbrace{\|\mathbf{w}_{\text{out}}\|}_{O(1/\sqrt{N})} \underbrace{\|\mathbf{x}\|}_{O(\sqrt{N})}, \quad (10)$$

so that $\|\mathbf{x}\| = O(\sqrt{N})$, or equivalently single neuron activations $x_i = O(1)$. This scaling corresponds to the aligned regime.

For the sine wave task, training with small output weights converges to an intuitive solution (Fig. 9A). Neural activity evolves on a limit cycle, and the output is a sine wave that extrapolates beyond the training interval. Plotting activity and output weights along the largest two PCs and the remaining direction $\mathbf{w}_{\text{out},\perp}$ confirms substantial correlation, $\rho = O(1)$, as expected. The solution was robust to adding noise after training (not shown). Changes in the initial dynamics or the presence of noise during training did not lead to qualitatively different solutions (Fig. 24). A further look at the eigenvalue spectrum of the trained recurrent weights revealed a pair of complex conjugate outliers corresponding to the limit cycle [42, 68], and a bulk of remaining eigenvalues concentrated on a disk with radius g , Fig. 23.

4.7.2. Large weights, no noise: linearization of dynamics

We now consider learning with large output weights, $\|\mathbf{w}_{\text{out}}\| = 1$, for noise-less dynamics, Eq. (9). We start with the assumption that the activity changes for each neuron are small,

$\Delta x_i(t) = O(1/\sqrt{N})$, or $\|\Delta \mathbf{x}\| = O(1)$. Here, $\Delta \mathbf{x}(t) = \mathbf{x}(t) - \mathbf{x}_0(t)$, where $\mathbf{x}_0(t)$ is the activity before learning. To perform a task, learning needs to induce output changes $\Delta z = O(1)$ to reach the target $\hat{z}(t)$. Note that a possible order-one initial output z_0 must also be compensated. Together with the output weight scale, we arrive at

$$\underbrace{\Delta z}_{O(1)} = \underbrace{\rho_{\Delta}}_{O(1)} \underbrace{\|\mathbf{w}_{\text{out}}\|}_{O(1)} \underbrace{\|\Delta \mathbf{x}\|}_{O(1)}, \quad (11)$$

where $\rho_{\Delta} = \text{corr}(\mathbf{w}_{\text{out}}, \Delta \mathbf{x})$. This shows that our assumption of small state changes $\Delta \mathbf{x}$ is consistent - it allows for a solution - and that such small changes need to be strongly correlated to the output weights. Note that we make the distinction between the changes $\Delta \mathbf{x}$ and the final activity $\mathbf{x} = \mathbf{x}_0 + \Delta \mathbf{x}$, because the latter may be dominated by $\mathbf{x}_0$. In the main text, we only consider the correlation between $\mathbf{x}$ and $\mathbf{w}_{\text{out}}$, because (as we show below) solutions with small $\Delta \mathbf{x}$ are not robust, and the final $\mathbf{x}$ will be dominated by $\Delta \mathbf{x}$.

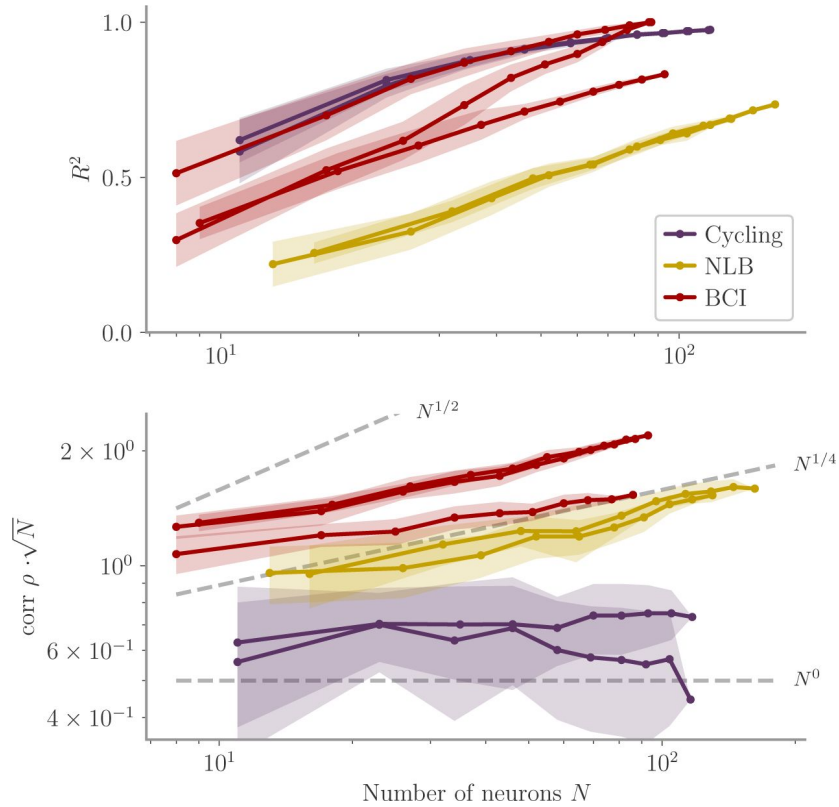


Fig. 8

Scaling of the correlation p with the number of neurons N in experimental data. We fitted the output weights to subsets of N neurons and computed the quality of fit (top) and the correlation between the resulting output weight and firing rates (bottom). To compare with random vectors, the correlation is scaled by $\sqrt{N}$. Dashed lines are $N^p/2$, for $p \in \{1/2, 1/4, 0\}$ for comparison. The aligned regime corresponds to $p = 1/2$, and the oblique one to $p = 0$.

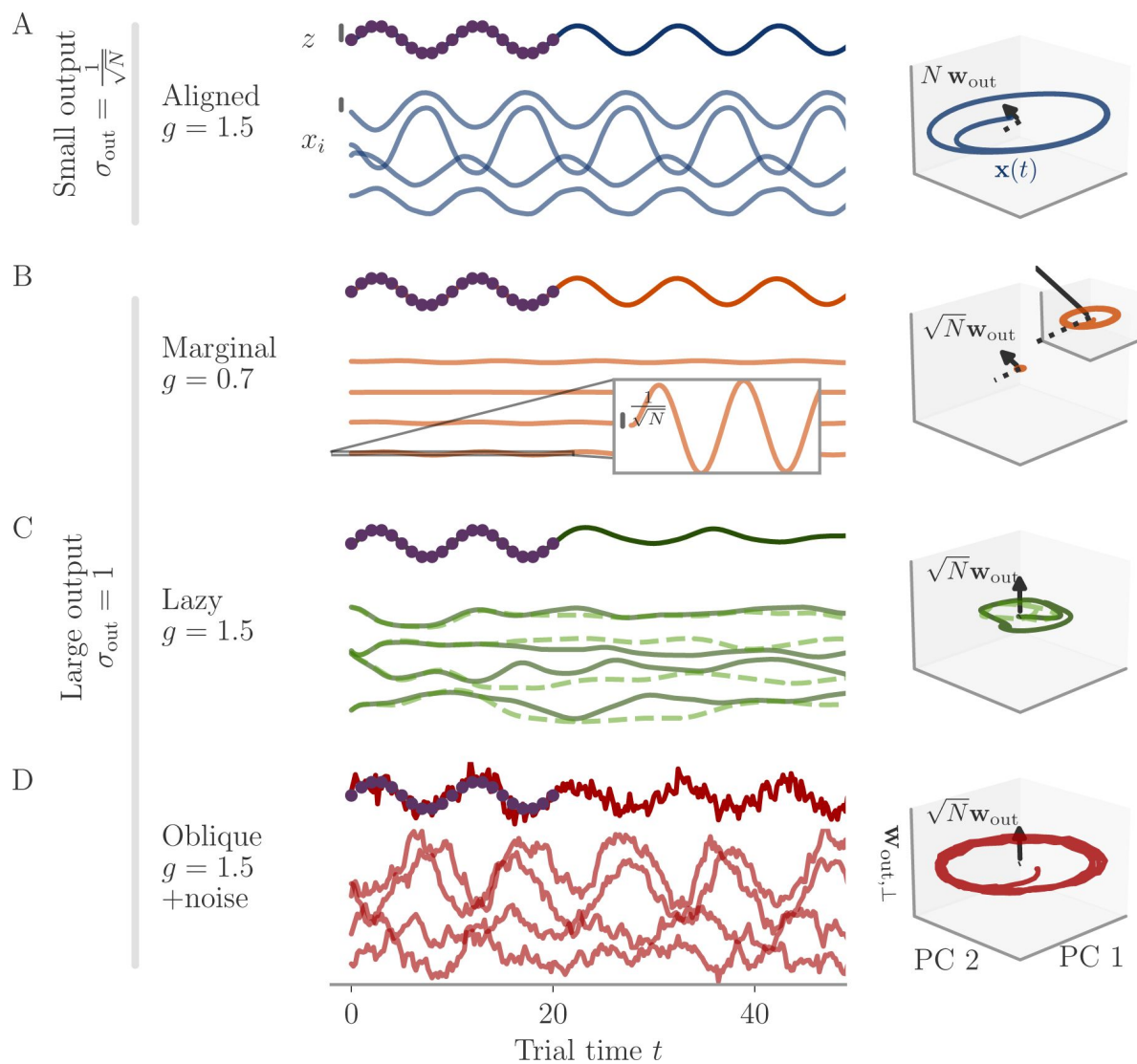


Fig. 9

Different solutions for networks trained on a sine wave task. All networks have $N = 512$ neurons. Four regimes: **A**: aligned for small output weights, **B**: marginal for large output weights, small recurrent weights, **C**: lazy for both large output and recurrent weights, **D**: oblique for large output weights and noise added during training. Left: Output (dark), target (purple dots), and four states (light) of the network after training. Black bars indicate the scales for output and states (length = 1; same for all regimes). The output beyond the target interval $t \in [1, 21]$ can be considered as extrapolation. The network in the oblique regime, **D**, receives white noise during training, and the evaluation is shown with the same noise. Without noise, this network still produces a sine wave (not shown). Right: Projection of states on the first 2 PCs and the orthogonal component $\mathbf{w}_{\text{out}, \perp}$ of the output vector. All axes have the same scale, which allows for comparison between the dynamics. Vectors show the (amplified) output weights, dotted lines the projection on the PCs (not visible for lazy and oblique). The insets for the marginal solution (B, left and right) show the dynamics magnified by $\sqrt{N}$.

For now, however, we ignore robustness and continue with the assumption of small $\Delta \mathbf{x}$. Given this assumption, we linearize the dynamics around the initial trajectory $\mathbf{x}_0(t)$:

$$\frac{d\Delta \mathbf{x}}{dt} = \underbrace{\Delta W \phi(\mathbf{x}_0)}_{\mathbf{a}} + \underbrace{[-I + (W_0 + \Delta W)R'(\mathbf{x}_0)] \Delta \mathbf{x}}_{\mathbf{b}} + O(\Delta \mathbf{x}^2), \quad (12)$$

with diagonal matrix $R'(\mathbf{x})_{ij} = \delta_{ij} \phi'(x_i)$ and the weights changes $\Delta W = W - W_0$ that induce $\Delta \mathbf{x}$. Note that we haven't yet constrained the weight changes ΔW so we cannot discard the terms of the kind $\Delta W \Delta \mathbf{x}$. The next steps depend on the initial trajectories $\mathbf{x}_0(t)$.

4.7.3. Initially decaying dynamics lead to a marginal regime

We first consider networks with decaying dynamics before learning. This is obtained by drawing the initial recurrent weights independently from $W_{ij} \sim \mathcal{N}(0, \frac{g^2}{N})$, with $g < 1$ [70]. With such dynamics, $\mathbf{x}_0(t)$ vanishes exponentially in time. In Eq. (12), we disregard the term $\mathbf{a}$ and have $R' = I$, so that

$$\frac{d\Delta \mathbf{x}}{dt} = (-I + W_0 + \Delta W) \Delta \mathbf{x} + O(\Delta \mathbf{x}^2). \quad (13)$$

To have self-sustained dynamics, the matrix $W_0 + \Delta W$ must have a leading eigenvalue λ_+ with real part above the stability line: $\Re \lambda_+ = 1 + \epsilon$.

The distance $\epsilon > 0$ must be small, else the states would become large. To understand how ϵ needs to scale with N , we turn to a simple model studied before [42, 68]: an autonomously generated fixed point and rank-one connectivity $W = \frac{1}{N} \lambda_+ \mathbf{u} \mathbf{u}^T$. The vector $\mathbf{u}$ has entries $u_i \sim \mathcal{N}(0, 1)$. A fixed point of the dynamics Eq. (9) fulfills $\mathbf{x} = \frac{1}{N} \lambda_+ \mathbf{u} \mathbf{u}^T \phi(\mathbf{x})$. Projecting on $\mathbf{u}$ and applying partial integration in the limit $N \rightarrow \infty$, we obtain

$$\lambda_+ = \frac{1}{\langle \phi' \rangle}, \quad (14)$$

where $\langle \phi' \rangle = \int \mathcal{D}u \phi'(\sigma_x u)$, with standard normal measure

$\mathcal{D}u = du \frac{1}{\sqrt{2\pi}} e^{-u^2/2}$. Here, σ_x is the scale of the states, $\sigma_x = \|\mathbf{x}\|/\sqrt{N}$ or $x_i = O(\sigma_x)$. The

fixed point is situated along the vector $\mathbf{u}$. To have the smallest possible fixed point generate some output, we set the output weights to $\mathbf{W}_{\text{out}} = \frac{1}{\sqrt{N}} \mathbf{u}$. Then we have correlation $\rho = 1$ and

$z = \mathbf{W}_{\text{out}}^T \mathbf{x} = \rho \|\mathbf{W}_{\text{out}}\| \|\mathbf{x}\| = 1 \cdot 1 \cdot \sqrt{N} \sigma_x$. In other words, a small fixed point with $\sigma_x \sim \frac{1}{\sqrt{N}}$. We expand ϕ' in Eq. (14) around zero. For even ϕ , we have $\phi''(0) = 0$ and

$$\lambda_+ = \frac{1}{1 + \frac{1}{2} \phi'''(0) \sigma_x^2} = 1 - \frac{1}{2} \phi'''(0) \sigma_x^2. \quad (15)$$

Sigmoidal functions have $\phi''' < 0$, e.g. $\phi'''(0) = -2$ for $\phi = \tanh$. Hence $\lambda = 1 + \epsilon$, with

$$\epsilon = \sigma_x^2 \sim \frac{1}{N}. \text{ Remarkably, the perturbation leading to states with}$$

$$\sigma_x = O(1/\sqrt{N}) \text{ only needs to have a distance } \epsilon = O(1/N) \text{ away from the stability line.}$$

The insights from this simplified setting extend to the example of the sine wave task (Fig. 9B). The model with large output weights, $g = 0.7$, and no noise yields a limit cycle. The output

extrapolates in time, but the states are very small, scaling as $x_i = O(1/\sqrt{N})$ (analysis

over different N not shown). Such a solution is only marginally stable – adding a white noise with $\sigma_{\text{noise}} = 0.2$ after training destroyed the rotation (not shown). The eigenvalues were again split into two outliers and a bulk (Fig. 23). However, the two outliers now had a real part $1 + \epsilon$, that is, they were very close to the stability line of the fixed point at zero. To better illustrate the marginal solution, we also set the initial state $\mathbf{x}(t = 0)$ to small values, $x_i(t = 0) \sim N(0, 1/N)$. For $x_i(t = 0) \sim N(0, 1)$, there would be an initial decay much larger than the limit cycle.

4.7.4. Initially chaotic dynamics lead to a lazy regime

In contrast to the situation before, initially chaotic dynamics (for $g > 1$) imply order-one initial states, $(x_0(t))_i = O(1)$ for all trial times t . The driving term $\mathbf{a}$ in Eq. (12) can thus not be ignored and we expect it to be on the same scale as $\Delta \mathbf{x}$:

$$1 \sim \|\Delta \mathbf{x}\| \sim \|\Delta W \phi(\mathbf{x}_0)\|. \quad (16)$$

The smallest possible weight changes ΔW will be those for which $\phi(\mathbf{x}_0)$ yields a maximal response, but other vectors do not yield a strong response. This is captured by the operator norm, $\|\Delta W\|_2 = \max\{\|\Delta W \mathbf{x}\| : \mathbf{x} \in \mathbb{R}^N \text{ with } \|\mathbf{x}\| = 1\}$. We can then write $\|\Delta W \phi(\mathbf{x}_0)\| \sim \|\Delta W\|_2 \|\mathbf{x}_0\|$, and hence

$$\|\Delta W\|_2 \sim \frac{1}{\sqrt{N}}. \text{ The operator norm also bounds the eigenvalues } \Delta W \text{ and hence the effect of}$$

the matrix on the dynamics of the system. For large N , this implies that the changes ΔW are too small to change the dynamics qualitatively, and the latter remain chaotic. Note that because the network dynamics are chaotic, the term $\mathbf{b}$ in Eq. (12) diverges, so that our discussion is only valid for short times. Numerically, we find small weight changes and chaotic solutions even for large target times t_i (not shown).

For the sine wave task, the network with initially chaotic dynamics indeed converges to such a solution (Fig. 9C). The output does not extrapolate beyond the training interval $t \in [1, 21]$, and dynamics remain qualitatively similar to those before training. During the training interval, the dynamics also remain close to the initial trajectories (dashed line). Testing the response to small perturbations in $\mathbf{x}(0)$ indicated that the dynamics remain chaotic (not shown). No limit cycle was formed, and the spectrum of eigenvalues did not show outliers (Fig. 23).

We called this regime “lazy”, following similar settings in feedforward networks [9, 27]. Note that there, the output $z(t)$ is linearized around the weights at initialization (as opposed to the dynamics, Eq. (9)). This can be done in our case as well:

$$z(t) = z_0(t) + \sum_{ij} \mathbf{w}_{\text{out}}^T \frac{d\mathbf{x}(t)}{dW_{ij}} \bigg|_{W_0} \Delta W_{ij} + O(\Delta W^2). \quad (17)$$

Demanding $\hat{z}(t) = z(t)$ yields a linear equation for each time point t . As we have N^2 parameters, this system is typically underconstrained. Gradient descent for this linear system leads to the minimal norm solution, which can also be found directly using the Moore-Penrose pseudo-inverse.

Numerically, we found that the weights ΔW_{lin} obtained by this linearization are very close to those found by gradient descent (GD) on the nonlinear system, ΔW_{GD} , with Frobenius norm

$$\|\Delta W_{\text{GD}} - \Delta W_{\text{lin}}\| \sim \frac{1}{N} \text{ (Appendix A.1).}$$

4.7.5. Marginal and lazy solutions disappear with noise during training

The deduction above hinges on the assumption that $\Delta \mathbf{x}$ is small. This assumption does not hold if dynamics are noisy, Eq. (4). For marginal dynamics, the noise would push solutions to different attractors or different positions along the limit cycle. For lazy dynamics, the chaotic dynamics would amplify any perturbations along the trajectory (if chaos persists under noise [67]).

We will explore how learning is affected by noise in the sections below. Here we only show that adding noise for our example task abolishes the marginal or lazy solutions and leads to oblique ones (Fig. 9D). We added white noise with amplitude $\sigma_{\text{noise}} = 1/\sqrt{2}$ to the dynamics during learning. After training, the output was a noisy sine wave. States were order one, and the 3D projection showed dynamics along a limit cycle that was almost orthogonal to the output vector. The noise in the 2D subspace of the first two PCs was small, $O(\frac{1}{\sqrt{N}})$, and thus did not disrupt the dynamics (e.g. very little phase shift). The eigenvalue spectrum had two outliers whose real part was increased in comparison to those in the aligned regime (Fig. 23). Note that for the chosen values $g = 1.5$ and $\sigma_{\text{noise}} = 1/\sqrt{2}$, the network actually was not chaotic at initialization [67]. However, the choice of g does not influence the solution in the oblique regime qualitatively, so both marginal and lazy solutions cease to exist (Fig. 25).

4.8. Learning with noise for linear RNNs

In the next two sections, we aim to understand how adding noise affects dynamics and training. We start with a simple setting of a linear RNN which allows us to track the learning dynamics analytically. Despite its simplicity, this setting already captures a range of observations: different time scales for learning the bias and variance part, and the rise of a negative feedback loop for noise suppression. Oblique dynamics, however, do not arise, showing that these need autonomously generated, nonlinear dynamics, covered in Section 4.9.

We consider a linear network driven by a constant input and additional white noise. The dynamics read

$$\dot{\mathbf{x}} = (-I + W)\mathbf{x} + \mathbf{w}_{\text{in}} + \boldsymbol{\xi}, \quad (18)$$

with noise $\boldsymbol{\xi}$ as in Eq. (4). We focus on a simplified task, which is to produce a constant nonzero output $\hat{z}$ once the average dynamics converged, i.e., for large trial times. The output is

$$z = \mathbf{w}_{\text{out}}^T \mathbf{x} = \bar{z} + \delta z, \text{ where the bar denotes average over the noise, and}$$

the delta fluctuations around the average. The average is given by $\bar{z} = \mathbf{w}_{\text{out}}^T (I - W)^{-1} \mathbf{w}_{\text{in}}$. We train the network by changing only the recurrent weights W via gradient descent. For small output weights, the fluctuations are too small to affect training: $\delta z = O(1/\sqrt{N})$. Apart from a small correction, learning dynamics are then the same as for small output weights and no noise, a setting analyzed in Ref. [69]. Here we only consider the case of large output weights.

The loss separates into two parts, $L = L_{\text{bias}} + L_{\text{var}}$ with $L_{\text{bias}} = (\bar{z} - \hat{z})^2$ and $L_{\text{var}} = \overline{\delta z^2} = \text{var}(\delta z)$. Learning aims to minimize the sum. We first consider learning based on each part alone and then join both to describe the full learning dynamics.

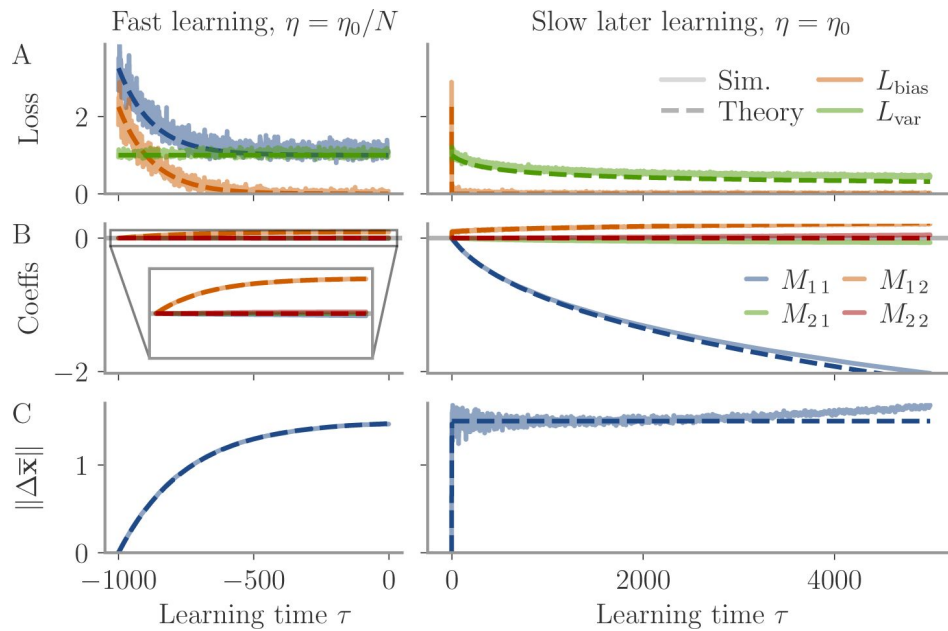


Fig. 10

Noise-induced learning for a linear network with input-driven fixed point. Learning separates into fast learning of the bias part of the loss (left), and slow learning reducing the variance part (right). Learning rates are $\eta = \eta_0/N$ and $\eta = \eta_0$, respectively, with $\eta_0 = 0.002$ and network size $N = 256$. Learning epochs in the first phase are counted from -1000, so that the second phase starts at 0. In the right column, the initial learning phase with learning time steps multiplied by $1/N$ is shown for comparison. In all plots, simulations (full lines) are compared with theory (dashed lines). **A** Loss $L = L_{\text{bias}} + L_{\text{var}}$. The two components are obtained by averaging over a batch with 32 examples at each learning step. The full loss is not plotted in the slow phase, because it is indistinguishable from L_{var} . **B** Coefficients of the 2-by-2 coupling matrix M . $M_{11} = \lambda_-$ is the feedback loop along the output weights, $M_{12} = b/\sqrt{N}$ a feedforward coupling from input to output. The theory predicts $M_{21} \sim M_{22} \sim O(1/N)$. **C** Norm of state changes during training. The theory predicts that it remains constant during the second phase and small compared to $\|\bar{x}_0\| = \sqrt{N}$. Other parameters: target $\hat{z} = 1$, $\sigma_{\text{noise}} = 1$, overlap between input and output vectors $\rho_{io} = -0.5/\sqrt{N}$.

Learning based on the bias part alone converges to a lazy solution (see Appendix A.2). For no initial weights, $W_0 = 0$, we have to leading order

$$\Delta W(\tau) = \frac{b_1(\tau)}{\sqrt{N}} \hat{\mathbf{w}}_{\text{out}} \hat{\mathbf{w}}_{\text{in},\perp}^T, \quad (19)$$

with

$$b_1(\tau) = (1 - e^{-2N\eta\tau})(\hat{z} - \sqrt{N}\rho_{io}). \quad (20)$$

Thus, ΔW is rank one with norm $\|\Delta W\|_2 \sim \frac{1}{\sqrt{N}}$. Furthermore, for a learning rate η , it converges in $O(\frac{1}{\eta N})$ time steps. We will see that this is very fast compared to learning the variance part.

4.8.1. Learning to reduce noise alone slowly produces a negative feedback loop

Next, we consider learning based on the variance part L_{var} alone, that is, to reduce fluctuations in the output while ignoring the mean. The network dynamics are linear, so that $\delta \mathbf{x}$ is an Ornstein-Uhlenbeck process. Its stationary variance Σ is the solution to the Lyapunov equation

$$0 = A\Sigma + \Sigma A^T + 2\sigma_{\text{noise}}^2 I, \quad (21)$$

where $A = -I + W$. The variance part of the loss is then

$$L_{\text{var}} = \mathbf{w}_{\text{out}}^T \Sigma \mathbf{w}_{\text{out}}. \quad (22)$$

One can state the gradient of this loss in terms of a second Lyapunov equation [84] [\[84\]](#):

$$G_{\text{var}} = \frac{dL_{\text{var}}}{dW} = 2\Omega\Sigma, \quad (23)$$

where Ω is the solution to

$$0 = A\Omega + \Omega A^T + \mathbf{w}_{\text{out}} \mathbf{w}_{\text{out}}^T. \quad (24)$$

Generally, solving both Lyapunov equations analytically is not possible, and even results for random matrices are still sparse (e.g. for symmetric Wigner matrices W [53] [\[53\]](#)). To gain intuition, we thus restrict ourselves to the case of no initial connectivity, which leads to the connectivity spanned by input and output weights only. We start with the simplest case of a rank-one matrix only spanned by the output weights, $W = \lambda_- \mathbf{w}_{\text{out}} \mathbf{w}_{\text{out}}^T$, and extend to rank two in the following section. The Lyapunov equations then become one-dimensional, and we obtain

$$\Sigma = \sigma_{\text{noise}}^2 \left(I + \frac{\lambda_-}{1 - \lambda_-} \mathbf{w}_{\text{out}} \mathbf{w}_{\text{out}}^T \right), \quad (25)$$

$$\Omega = \frac{1}{2(1 - \lambda_-)} \mathbf{w}_{\text{out}} \mathbf{w}_{\text{out}}^T. \quad (26)$$

The gradient $G_{var} = 2\Sigma\Omega$ is therefore in the same subspace as W , and we can evaluate the onedimensional dynamics

$$\frac{d\lambda_-(\tau)}{d\tau} = \frac{-\eta\sigma_{noise}^2}{[1 - \lambda_-(\tau)]^2}, \quad (27)$$

where τ is the number of update steps. We assume τ to be continuous, that is, we assume a sufficiently small learning rate and approximate the discrete dynamics of gradient descent with gradient flow. With initial condition $\lambda_-(0) = 0$, the solution is

$$\lambda_-(\tau) = 1 - (3\eta\sigma_{noise}^2\tau + 1)^{\frac{1}{3}}, \quad (28)$$

which is negative for $\tau > 0$. The loss then decays as

$$L_{var} = \sigma_{noise}^2 (3\eta\sigma_{noise}^2\tau + 1)^{-\frac{1}{3}}, \quad (29)$$

namely at order $O(\tau^{-1/3})$ in learning time. We thus obtained that, during the variance phase of learning, connectivity develops a negative feedback loop aligned with the output weights, which serves to suppress output noise. For very long learning times, $\tau \sim N^3/\eta$, learning can in principle reduce the output fluctuations to $\frac{1}{\sqrt{N}}$. Note, however, that this implies a huge negative feedback loop, $\lambda_- = O(N)$, which potentially leads to instability in a system with delays or discretized dynamics [29].

4.8.2. Optimizing mean and fluctuations occurs on different time scales

We now consider learning both the mean and the variance part. For zero initial recurrent weights, $W0 = 0$, the input and output vectors make up the only relevant directions in space. We thus express the recurrent weights as a rank-two matrix, $W = \hat{U}M\hat{U}^T$, with orthonormal basis $\hat{U} = [\hat{\mathbf{w}}_{out}, \hat{\mathbf{w}}_{in, \perp}]$. The hats indicate normalized vectors. For large networks and large output weights, the first vector is already normalized, $\hat{\mathbf{w}}_{out} = \mathbf{w}_{out}$. The second vector is the input weights after Gram-Schmidt. Assuming that $\mathbf{w}_{out}$ and $\mathbf{w}_{in}$ are drawn independently, we have a small, random correlation $\rho_{io} = O(1/\sqrt{N})$, and can write $\hat{\mathbf{w}}_{in, \perp} = \hat{\mathbf{w}}_{in} - \mathbf{w}_{out}\rho_{io} + O(\frac{1}{N})$.

We computed the learning dynamics in terms of the coefficient matrix M , using the same tools introduced above, and the insight that learning the mean is much faster than learning to reduce the variance. The details are relegated to Appendix A.3, here we summarize the results. We obtained

$$M(\tau) = \begin{bmatrix} \lambda_-(\tau) & \frac{b(\tau)}{\sqrt{N}} \\ 0 & 0 \end{bmatrix}. \quad (30)$$

Before discussing the temporal evolution of the two components, we analyze the structure of the matrix. The eigenvalue $M_{11} = \lambda_-$ is a negative feedback loop along the eigenvector $\mathbf{w}_{out}$, and $M_{21} = b/\sqrt{N}$ is a small feedforward component b which maps the input to the output. Along the input direction $\hat{\mathbf{w}}_{in}$, learning does not change the dynamics: The second eigenvalue, corresponding to this direction, is zero.

The dynamics unfold on two time scales. First, there is a very fast learning of the bias via the feedforward coefficient

$$b(\tau) = (1 - e^{-2N\eta\tau})(\hat{z} - \sqrt{N}\rho_{io}). \quad (31)$$

During this phase, the eigenvalue $M_{11} = \lambda_-$ remains at zero, so that overall weight changes remain small, $\|W\| \sim 1/\sqrt{N}$. The average fixed point also does not change much, $\|\Delta\bar{x}(\tau)\| = b_1(\tau) = O(1)$, in comparison to the fixed point before learning, $\|\bar{x}_0\| = \|\bar{w}_{in}\| = \sqrt{N}$. The loss evolves as

$$L_{\text{bias}} = e^{-4N\eta\tau} (\hat{z} - \sqrt{N}\rho_{io})^2. \quad (32)$$

The variance part of the loss does not change during this phase.

In a second, slower learning phase, the eigenvalue λ_- evolves like above, Eq. (28), the case where only the variance part is learned. The second coefficient compensates for the resulting change in the output. This compensation happens at a much faster time scale (N^3 times faster than λ_-), so we consider its steady state:

$$b(\tau) = \hat{z}[1 - \lambda_-(\tau)] - \sqrt{N}\rho_{io}. \quad (33)$$

Because of this compensation, the bias part of the loss always remains at zero, and the full loss $L(\tau) = L_{\text{var}}(\tau)$ evolves as before, Eq. (29). Meanwhile, the average fixed point does not change anymore; we have $\|\Delta\bar{x}\| = \hat{z} - \rho_{io}\sqrt{N}$.

We compare our theoretical predictions against numerical simulations in Fig. 10. For the task, we let the linear network dynamics converge from $\mathbf{x}(0) = \mathbf{0}$ until $t = 15$, and demand the output $z(t)$ to be at the target $\hat{z}$ during the interval $t \in [15, 20]$. Because the first learning phase converges N times faster than the second one (with $N = 256$), using a single learning rate η is problematic. One can either observe the first phase only (for small η) or risk unstable learning during the first phase (for large η). We thus split learning into two parts with adapted learning rates. For the initial phase, we set a learning rate to $\eta = \eta_0/N$, with $\eta_0 = 0.002$ (Fig. 10 left column). For the second phase, we set $\eta = \eta_0$ (Fig. 10 right column). Theory and simulation agree well for both phases. Small deviations can be observed for the second phase and long learning times: nonzero coefficients M_{21} and M_{22} and a corresponding increase in the norm $\|\Delta\bar{x}\|$. Testing with larger network sizes showed that these errors decreased as $O(1/N)$, which is consistent with our theory above (not shown).

Our results show that learning in this linear system is a hybrid between oblique and aligned during the second phase. We have a large term that compresses the output noise, but a very small, “lazy” correction in the feedforward component that corrects the output, and the states are only marginally changed. Although this system is a somewhat degenerate limiting case, we can still derive important insights. The two most striking features – the different time scales of the two learning processes, and the slow emergence of negative feedback, $\lambda_-(\tau) \sim -\tau^{1/3}$, along the output – are found robustly also for nonlinear networks and other tasks.

4.9. Oblique solutions arise for noisy, nonlinear systems

We now examine the origin of oblique solutions. The linear system did not yield oblique solutions, so we turn to a nonlinear model. We consider a 1D flip-flop task, where the network has to yield a constant, nonzero output $\tilde{k}$ depending on the sign of the last input pulse. We further simplify the analysis by only considering the steady state of a network, not how the input pulse mediates the transition. At the output, we thus only consider the average $\bar{z}$ and the fluctuations δz . As for the linear network, the loss splits into a bias part $L_{\text{bias}} = (\bar{z} - \tilde{z})^2$ and variance part $L_{\text{var}} = \overline{\delta z^2}$. As before, we assume that learning only acts on a low-dimensional parameter matrix M .

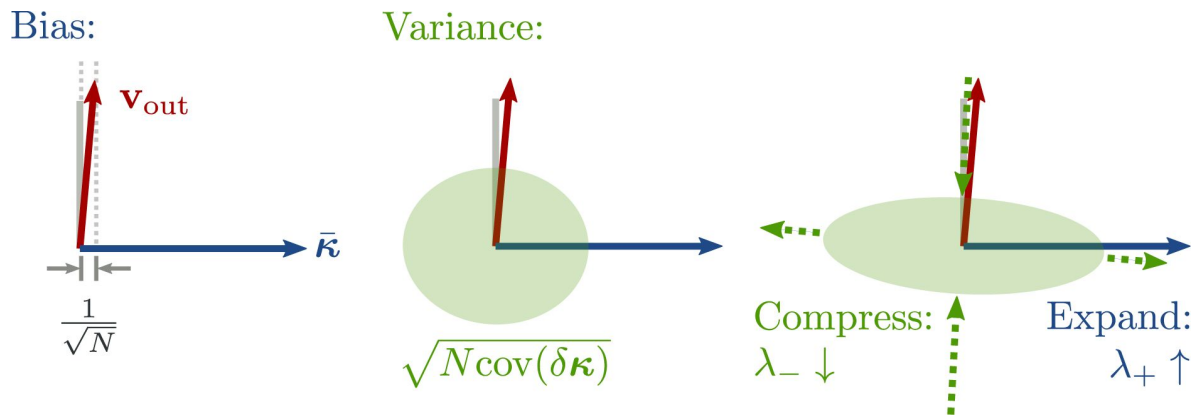


Fig. 11

Cartoon illustrating the split into bias and variance components of the loss, and noise suppression along the output direction. The two-dimensional subspace spanned by U illustrates the main directions under consideration: the PCs of the average trajectories (here only a fixed point $\bar{x} = U\bar{\kappa}$), and the direction of output weights $w_{\text{out}} = \frac{1}{\sqrt{N}}Uv_{\text{out}}$. Left: During learning, a fast process keeps the average output close to the target so that $L_{\text{bias}} = 0$. Center: The variance component, L_{var} , is determined by the projection of the fluctuations $\delta\kappa$ onto the output vector. Note that the noise in the low-D subspace is very small, $\delta\kappa = O(1/\sqrt{N})$, but the output is still affected due to the large output weights. Right: During training, the noise becomes non-isotropic. Along the average direction $\bar{\kappa}$, the fluctuations are increased as a byproduct of the positive feedback λ_+ . Meanwhile, a slow learning process suppresses the output variance via a negative feedback λ_- .

Because following the learning dynamics in nonlinear networks is difficult, we take a different approach. We develop a mean field theory to show how noise affects the dynamics of a nonlinear network with autonomous fixed points. This allows us to compute the loss components L_{bias} and L_{var} in terms of M . We then show that the minimum of the loss function corresponds to oblique solutions. This leads to a clear interpretation of the mechanisms pushing for oblique solutions. Finally, we show that the theory quantitatively predicts the outcome of learning with gradient descent.

4.9.1. Rank-two connectivity model with fixed point

We first introduce the connectivity model and compute the latent dynamics using mean field theory. We constrain the recurrent connectivity to a rank-two model of the form $W = \frac{1}{N}UMU^T$, where M is a 2×2 coefficient matrix to be learned. The randomly drawn projection matrix $U \in \mathbb{R}^{N \times 2}$ has entries U_{ia} drawn independently from a standard normal distribution. This implies orthogonality to leading order, $\frac{1}{N}U^TU = I_2 + O(1/\sqrt{N})$. We will discard the correction term, as it does not change our results apart from a constant bias. We further assume that the input and output vectors are spanned by U , although not necessarily identified with the components of U as in the previous section. Note also that for clarity we do not normalize U as $\tilde{U}$ before. The assumption of Gaussian connectivity greatly simplifies the math, while its restrictions are irrelevant to the task considered here [4, 13, 68].

To understand the dynamics Eq. (4) analytically, we make use of the low-rank connectivity. Following previous work [29, 57], we split the dynamics into two parts: a parallel part $\mathbf{x}_{\parallel}$ in the subspace spanned by U , and an orthogonal part $\mathbf{x}_{\perp}$. This yields

$$\dot{\mathbf{x}}_{\parallel} = -\mathbf{x}_{\parallel} + \frac{1}{N}UMU^T\phi(\mathbf{x}_{\parallel} + \mathbf{x}_{\perp}) + \frac{1}{N}UU^T\xi, \quad (34)$$

$$\dot{\mathbf{x}}_{\perp} = -\mathbf{x}_{\perp} + (I - \frac{1}{N}UU^T)\xi. \quad (35)$$

Notice that the parallel part is partially driven by the orthogonal one, but not vice versa. The parallel part can be expressed in terms of the latent variable

$$\kappa = \frac{1}{N}U^T\mathbf{x} = \frac{1}{N}U^T\mathbf{x}_{\parallel}. \quad (36)$$

The scaling here ensures that κ is order one if $\mathbf{x}_{\parallel}$ has order-one states. Because the readout is assumed to be spanned by U , the output is fully determined by the parallel part. We can write

$$z = \mathbf{w}_{\text{out}}^T\mathbf{x}_{\parallel} = \sqrt{N}\mathbf{v}_{\text{out}}^T\kappa, \quad (37)$$

with projected output weights $\mathbf{v}_{\text{out}} = \frac{1}{\sqrt{N}}U^T\mathbf{w}_{\text{out}}$. Note that we assume large output weights, $\|\mathbf{w}_{\text{out}}\| = 1$, so that $\mathbf{v}_{\text{out}}$ is also normalized.

We split the latent state into its average over the noise ξ and fluctuations, $\kappa = \bar{\kappa} + \delta\kappa$. Similarly, the output splits into $z = \bar{z} + \delta z$. The loss then has two components, $L = L_{\text{bias}} + L_{\text{var}}$, with

$$L_{\text{bias}} = (\bar{z} - \hat{z})^2 = \left(\sqrt{N}\mathbf{v}_{\text{out}}^T\bar{\kappa} - \hat{z}\right)^2, \quad (38)$$

$$L_{\text{var}} = \overline{\delta z^2} = N\mathbf{v}_{\text{out}}^T\overline{\delta\kappa\delta\kappa^T}\mathbf{v}_{\text{out}}. \quad (39)$$

We want to understand situations with small loss. For the bias term, the average $\bar{\kappa}$ must be either small or oblique to the readout weights. For the variance term, the covariance of the fluctuations, $\text{cov}(\delta\kappa) = \overline{\delta\kappa\delta\kappa^T}$, must be compressed along the output direction: Even though the covariance is

$O(1/N)$, it still projects to the output at $O(1)$, as reflected by the factor N in Eq. (39). Fig. 11 illustrates the situation in a cartoon from this high-level perspective.

To understand the underlying mechanisms in detail, we next explore how the relevant variables $\bar{\kappa}$ and $\delta\kappa$ are determined by the coupling matrix M . We do so by applying mean field theory, following previous works [29, 42, 67, 68]. Detailed derivations can be found in Appendix A.4. Here we present the high-level results. The average dynamics converge to a fixed point determined by the equation for the latent variable,

$$\bar{\kappa} = \langle \phi' \rangle M \bar{\kappa} + O\left(\frac{1}{\sqrt{N}}\right). \quad (40)$$

The $O(1/\sqrt{N})$ term is a constant offset for any given network that we ignore without loss of generality. The average slope is

$$\langle \phi' \rangle = \langle \phi'(\bar{\sigma}_x u) \rangle = \int \mathcal{D}u \phi'(\bar{\sigma}_x u), \quad (41)$$

with variance

$$\overline{\sigma_x^2} = \|\bar{\kappa}\|^2 + \sigma_{\perp}^2. \quad (42)$$

(We have $\bar{\sigma}_x = \sqrt{\overline{\sigma_x^2}}$ because the fluctuations are small.) The orthogonal variance is simply the variance of the noise, $\overline{\sigma_{\perp}^2} = \sigma_{\text{noise}}^2$. The fixed point Eq. (40), implies that for a nonzero fixed point, the matrix M must have an eigenvalue

$$\lambda_+ = \frac{1}{\langle \phi' \rangle}. \quad (43)$$

This is very similar to the noiseless situation discussed briefly above, Eq. (14). However, here the additional variance $\overline{\sigma_{\perp}^2}$ decreases the average slope due to saturation of the nonlinearity. This in turn increases the minimal eigenvalue for a nonzero fixed point, which can be found by setting $\bar{\kappa} = 0$, and hence $\bar{\sigma}_x = \sigma_{\perp}$:

$$\lambda_{+, \text{min}} = \frac{1}{\langle \phi'(\sigma_{\perp} u) \rangle}. \quad (44)$$

In other words, the noise decreases the effective gain $\langle \phi' \rangle$, and thus the connectivity eigenvalue λ_+ needs to compensate. From the point of view of the spectrum, we are thus pushed away from the margin $1 + \epsilon$. These considerations, however, do not exclude the possibility that dynamics converge to an average fixed point that is small and correlated, which is at odds with oblique dynamics. To understand why learning leads to oblique dynamics, we need to move beyond the average $\bar{\kappa}$ and take into account the fluctuations $\delta\kappa$.

4.9.2. Fluctuations of the latent variable

The fluctuations $\delta\kappa$ around a fixed point $\bar{\kappa}$ are driven by the noise, both directly and indirectly via the dynamics. The direct contribution is a white noise term of order $1/\sqrt{N}$ because ξ is isotropic and independent of U . A detailed analysis (Appendix A.4) shows that the indirect contribution is given by a colored noise term which originates from the finite size fluctuations in

the variance of the orthogonal part. This second term is also $O(1/\sqrt{N})$, which implies that the fluctuations are small, $\delta\kappa = O(1/\sqrt{N})$. We can thus linearize their dynamics around the mean $\bar{\kappa}$, which yields

$$\frac{d\delta\kappa(t)}{dt} = A\delta\kappa(t) + \frac{1}{\sqrt{N}}\zeta(t), \quad (45)$$

where the order-one term ζ contains both the white and the colored noise term. The Jacobian A depends on M and $\bar{\kappa}$:

$$A = -I + \langle\phi'\rangle M + \frac{\langle\phi'''\rangle}{\langle\phi'\rangle} \bar{\kappa} \bar{\kappa}^T. \quad (46)$$

The averages are again evaluated at the joint variance $\sigma_x^2 = \|\bar{\kappa}\|^2 + \sigma_{\perp}^2$. Apart from the increased variance, the stability analysis yields the same results as in the noise-free case [68]: The Jacobian has the eigenvalues $\gamma_+ = \frac{\langle\phi'''\rangle}{\langle\phi'\rangle} \|\bar{\kappa}\|^2$ and $\gamma_- = \frac{\lambda_-}{\lambda_+} - 1$. The average over the third derivative $\langle\phi'''\rangle$ is negative, so that $\gamma_+ < 0$. We assume that the second eigenvalue is smaller than the first, $\gamma_- < \gamma_+$, so that $\gamma_- < 0$. The fixed point under consideration is hence stable.

Next, we compute the covariance of the fluctuations at steady state, see Appendix A.4. The outcome is

$$\overline{\delta\kappa\delta\kappa^T} = \frac{1}{N} \left[\sigma_{\text{noise}}^2 \Sigma_A + \frac{\sigma_{\text{noise}}^4}{\|\bar{\kappa}\|^2} \frac{-\gamma_+}{2(2 - \gamma_+)} \mathbf{v}_+ \mathbf{v}_+^T \right], \quad (47)$$

where $\mathbf{v}_+ = \bar{\kappa}/\|\bar{\kappa}\|$ is the normalized eigenvector of M corresponding to eigenvalue γ_+ . The 2×2 matrix Σ_A is the covariance introduced by the white noise part alone and obeys the Lyapunov equation

$$0 = A\Sigma_A + \Sigma_A A^T + 2I_2. \quad (48)$$

The second term in Eq. (47) stems from the colored noise component of ζ .

The loss Eq. (39) is obtained by projecting the covariance on the output weights. Importantly, the factor $1/N$ in the covariance is compensated by the factor N in the loss. Hence, even if the covariance shrinks with increasing network size, the output is still affected at order one. We next explore the implications of minimizing this loss.

4.9.3. Minimizing the loss by balancing saturation and negative feedback loop

To gain an understanding of how the output fluctuations responsible for L_{bias} can be reduced, we first consider the case of a symmetric coefficient matrix M . Simulations of networks trained with gradient descent below show that this approximation is reasonable. For symmetric M , the orthogonal eigenvectors $\mathbf{v}_{\pm}$ with eigenvalues $\lambda_{\pm}$ of the recurrent weights M are also eigenvectors of A , in that case corresponding to the eigenvalues $\gamma_{\pm}$. This allows to diagonalize the Lyapunov Eq. (48) and yields the solution

$$\Sigma_A = \frac{\mathbf{v}_+ \mathbf{v}_+^T}{-\gamma_+} + \frac{\mathbf{v}_- \mathbf{v}_-^T}{-\gamma_-}. \quad (49)$$

For the loss Eq. (39), we further need the relation between the eigenvectors $\mathbf{v}_{\pm}$ and the output weights. Because the fixed point $\bar{\kappa}$ is parallel to the eigenvector, we have $\mathbf{v}_{\text{out}}^T \mathbf{v}_{+} = \rho$. For the other eigenvector, orthogonality yields $\mathbf{v}_{\text{out}}^T \mathbf{v}_{-} = \sqrt{1 - \rho^2}$. Inserting this into Eqs. (39) and (47) yields an expression in terms of the Jacobian eigenvalues $\gamma_{\pm}$, the correlation ρ , and the norm of the fixed point $\|\bar{\kappa}\|$:

$$L_{\text{var}} = \rho^2 \left[\frac{\sigma_{\text{noise}}^2}{-\gamma_{+}} + \frac{\sigma_{\text{noise}}^4}{\|\bar{\kappa}\|^2} \frac{\gamma_{+}}{2(\gamma_{+} - 2)} \right] + (1 - \rho^2) \frac{\sigma_{\text{noise}}^2}{-\gamma_{-}}. \quad (50)$$

For more explicit insight, we choose the nonlinearity $\phi(x) = \text{erf}(\alpha x)$ with $\alpha = \sqrt{\pi}/2$. Similar to tanh, this function is bounded between ± 1 and has slope $\phi'(0) = 1$ at the origin. For this function, we can explicitly compute the relevant Gaussian integrals. The minimal eigenvalue of M to produce a fixed point, Eq. (44), is then given by $\lambda_{+, \text{min}} = 1 + 2\alpha^2 \sigma_{\text{noise}}^2$. For $\lambda_{+} > \lambda_{+, \text{min}}$, the resulting fixed point has norm

$$\|\bar{\kappa}\|^2 = \frac{\lambda_{+}^2 - \lambda_{+, \text{min}}^2}{2\alpha^2}, \quad (51)$$

as shown in Fig. 12A. The larger eigenvalue of the Jacobian is

$\gamma_{+} = -(\lambda_{+}^2 - \lambda_{+, \text{min}}^2)/\lambda_{+}^2$. We next obtain the correlation between fixed point and output weights by assuming that the bias part of the loss Eq. (38), is kept at zero. This is reasonable because it requires only a small adaptation to the weights that leaves the variance part mostly untouched. The resulting correlation is

$$\rho^2 = \frac{1}{N} \frac{\hat{z}^2}{\|\bar{\kappa}\|^2}, \quad (52)$$

so that increasing λ_{+} also decreases the correlation (Fig. 12B). Finally, we obtain an expression for the variance part of the loss only in terms of the eigenvalues of M :

$$L_{\text{var}} = \rho^2 \left(\frac{\sigma_{\text{noise}}^2 \lambda_{+}^2}{\lambda_{+}^2 - \lambda_{+, \text{min}}^2} + \frac{\sigma_{\text{noise}}^4 \alpha^2}{3\lambda_{+}^2 - \lambda_{+, \text{min}}^2} \right) + (1 - \rho^2) \frac{\sigma_{\text{noise}}^2}{1 - \frac{\lambda_{-}}{\lambda_{+}}}. \quad (53)$$

We show L_{var} over λ_{+} for different negative feedback loop sizes λ_{-} (Fig. 12C) and different network sizes N (Fig. 12D). The first term diverges at the phase transition where the fixed point appears, $\lambda_{+} \searrow \lambda_{+, \text{min}}$. Learning will thus push the weights away from the phase transition towards larger λ_{+} . With such increasing λ_{+} , the fixed point norm increases, and the fixed point rotates away from the output, decreasing the correlation. This in turn emphasizes the last term, scaled by $1 - \rho^2$. The last term is reduced with increasingly negative λ_{-} , corresponding to the negative feedback loop that suppresses noise.

Learning can in principle strengthen this feedback further and further, $\lambda_{-} \rightarrow -\infty$ (apart from possible stability issues [29]). However, as for the linear network, Section 4.8, this process takes time. We thus assume λ_{-} to be fixed and search for a minimum across λ_{+} . For $\lambda_{-} < 0$, the last term in Eq. (53) increases with increasing λ_{+} : the effective feedback loop in the full, nonlinear system is weakened by saturation. The loss L_{var} thus has a minimum at some moderate λ_{+} .

To illustrate the mechanisms described above, we simulated networks at different λ_{+} and with $\lambda_{-} = -5$. For each λ_{+} , we compute ρ according to Eq. (52). Setting $\mathbf{v}_{\text{out}} = [1, 0]^T$, we then set the resulting symmetric $M = V \Lambda V^T$. For each network sample, we then draw independent random projections $U \in \mathbb{R}^{N \times 2}$. We started simulations at either one of the two nonzero fixed points. For λ_{+}

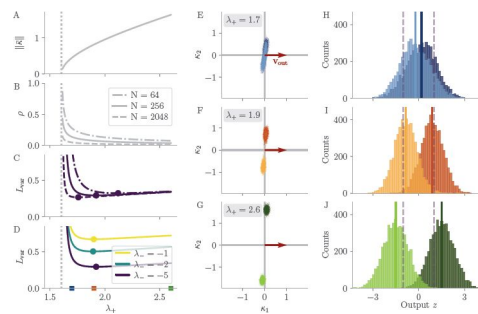


Fig. 12

Mechanisms behind oblique solutions predicted by mean field theory. **A-D:** Mean-field theory predictions as a function of positive feedback strength λ_+ . The dotted lines indicate $\lambda_{+,min}$, the minimal eigenvalue necessary to generate fixed points. **A:** Norm of fixed point $\|\kappa\|$. **B:** Correlation ρ so that $L_{bias} = 0$. **C,D:** Loss due to fluctuations for different λ_- or networks sizes N . Dots indicate minima. **E-G:** Latent states κ of simulated networks for randomly drawn projections U . The symmetric matrix M is fixed by setting λ_+ as noted, $\lambda_- = -5$, and demanding $L_{bias} = 0$ (for the mean field prediction). Dots are samples from the simulation interval $t \in [20, 100]$. **H-J:** Histogram for the corresponding output z . Mean is indicated by full lines, the dashed lines indicate the target $\hat{z}$. Other parameters: $N = 256$, $\sigma_{noise} = 1$, $\hat{z} = 1$

just above $\lambda_{+,min}$, the noise pushes activity from one basin of attraction to the next (Fig. 12C). The resulting output becomes centered around zero and independent of the initial condition for long simulation times (Fig. 12F). For the optimal λ_+ , the trajectories remain close to either one fixed point (Fig. 12F). The output forms two overlapping distributions, each closely matching the target on average (Fig. 12I). For larger λ_+ , the fixed points become increasingly larger (Fig. 12G). While this decreases the probability of leaving the basin of attraction even further, the variance along the output weights becomes larger (slightly wider histograms in Fig. 12J). Note that the mean starts to deviate from the prediction. This is not covered by our theory and is potentially due to the linearization of the fluctuations.

All-in-all, this section revealed a potential path to oblique solutions, initiated by the large fluctuations close to a phase transition, and the interplay between the negative feedback loop and saturation. In the following section, we show that learning via gradient descent actually follows this path and that the parameters predicted by the minimum of L_{var} quantitatively predict solutions from learning.

4.9.4. Oblique solutions from learning are predicted by the mean field theory

We trained neural networks on the fixed points task described above by applying gradient descent to the 2×2 matrix M , initialized at $M_0 = 0$. The gradients G_M with respect to M are equivalent to those with respect to $\tilde{W} = \frac{1}{N} U M U^T$ restricted to the subspace spanned by U , i.e.

$G_M = \frac{1}{N} U U^T G_{\tilde{W}} \frac{1}{N} U U^T$. Such a restriction is exact in the case of linear RNNs without random initial connectivity, and yields qualitative insights even for nonlinear networks with random initial connectivity [69].

The output weights were set to $\mathbf{w}_{out} = \frac{1}{\sqrt{N}} \mathbf{u}_1$, where $\mathbf{u}_1$ is the first of the two projection vectors $U = [\mathbf{u}_1, \mathbf{u}_2]$. We thus have large output weights with norm $\|\mathbf{w}_{out}\| = 1$. Trajectories were initialized at $\mathbf{x}(0) = \mathbf{0}$. At the beginning of a trial with target output $\hat{z} = \pm 1$, the network receives one pulse $s(t) = \pm \delta(t)$ along the input weights $\mathbf{w}_{in} = \mathbf{u}_2$. The input direction is hence the second available direction for the rank-two connectivity. This is a sensible choice as networks without the restriction to rank-two weights would span the recurrent weights from existing directions, and a rank-two connectivity would hence also be spanned by $\mathbf{w}_{out}$ and $\mathbf{w}_{in}$ [69].

The loss over learning time for one network is shown in Fig. 13A. Learning consisted of two phases: a first phase in which the network did not possess a fixed point and hence did not match the target on average, $L_{bias} > 0$. At some point, L_{bias} rapidly decreases, and L_{var} dominates the overall loss. During the second phase, L_{var} slowly decreases, with L_{bias} hovering around zero.

The coefficients of the matrix M indicate the underlying learning dynamics (Fig. 13B). The coefficient along the output weights, M_{11} , is almost identical with λ_- . It continually grows in the negative direction, unaffected by the different phases. Its time course is very similar to the time course observed for the linear system (Fig. 10B). In contrast, the coefficient along the input weights, M_{22} , mirrors the two phases. It grows increasingly fast in the first phase and saturates during the second phase. Its value is very close to the larger eigenvalue, λ_+ . The transition between the two phases of learning happens at the phase transition of the dynamical system when the fixed point emerges for $\lambda_+ = \lambda_{+,min}$. The off-diagonal entries show that M is asymmetric during the first phase and becomes symmetric later on. The coefficient M_{12} corresponds to the feedforward mode mapping the state decaying from $\mathbf{x}(0) = \mathbf{u}_2$ after the pulse to the output weights $\mathbf{w}_{out} = \frac{1}{\sqrt{N}} \mathbf{u}_1$.

Tracing the fixed point norm $\|\bar{\kappa}\|$ and the correlation ρ over learning time shows what we expected (Fig. 13C): The norm grows rapidly at the phase transition, which is accompanied by a decrease in the correlation. The example yields a fixed point with norm $\|\bar{\kappa}\| \approx 0.7$ and correlation $\rho \approx 0.05$ for a network with $N = 1024$. The mere numbers already suggest that we call this an

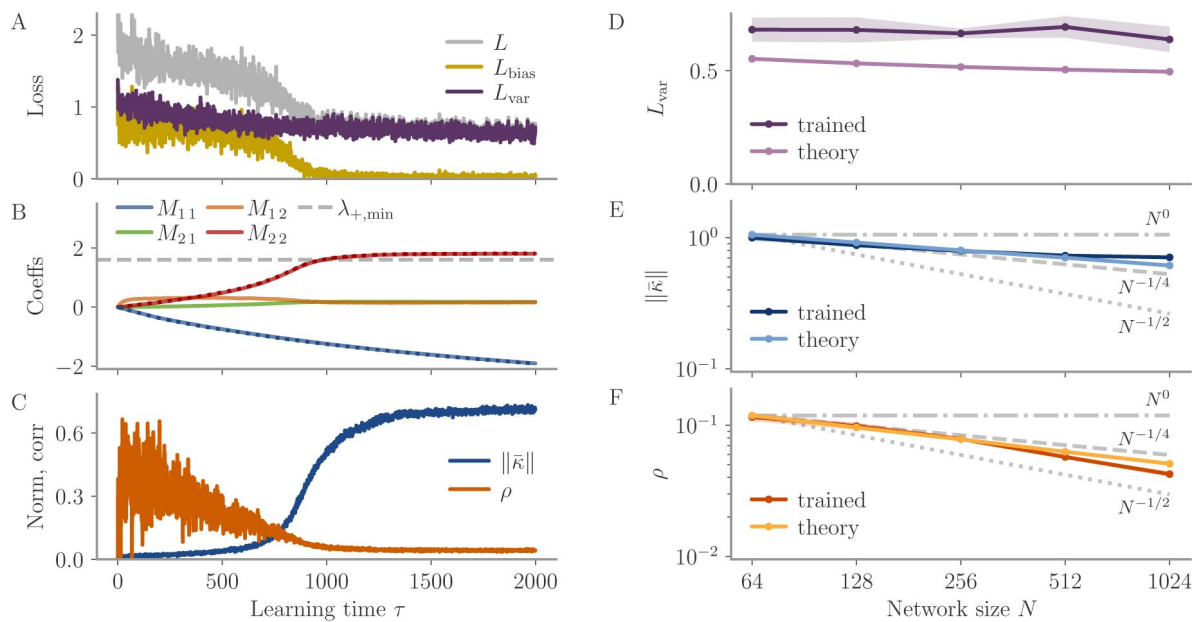


Fig. 13

Mean field theory predicts learning with gradient descent. **A-C**: Learning dynamics with gradient descent for example network with $N = 1024$ neurons and with noise variance $\sigma_{\text{noise}}^2 = 1$. **A**: Loss with separate bias and variance components. **B**: Matrix coefficients M_{ij} . The dotted lines almost identical to M_{22} and M_{11} indicate the eigenvalues λ_+ and λ_- , respectively. The dashed line indicates $\lambda_{+, \text{min}}$. **C**: Fixed point norm and correlation. **D-F**: Final loss, fixed point norm, and correlation for networks of different sizes N . Shown are mean (dots and lines) and std dev (shades) for 5 sample networks, and the prediction by the mean field theory. Grey lines indicate scaling as aN^k , with $k \in \{0, -1/4, -1/2\}$. Note the log-log axes for **E**, **F**.

oblique solution, but the theory description above is based on how these numbers scale with network size N . We trained networks with different N but otherwise the same conditions. They reached the same loss (**Fig. 13A** [↗](#), $L_{\text{bias}} \approx 0$ not shown). The fixed point norm decrease weakly with N , with $\|\bar{\kappa}\| = O(N^k)$, for some $k \geq +1/4$ (**Fig. 13E** [↗](#)). The correlation decreases faster, yet not quite with $1/\sqrt{N}$ (**Fig. 13F** [↗](#)).

We compared the outcome of learning to our mean field theory. Given that M is approximately symmetric at the end of learning, we directly applied our results from the previous sections, again assuming $L_{\text{bias}} = 0$. We fixated λ_- to match the value at the end of training and computed the λ_+ that minimized L_{var} , **Eq. (53)** [↗](#). The results for the norm and correlation match those values obtained with gradient descent very closely.

Our high-level description of oblique and aligned dynamics did not involve scaling with network size (Section 2.1). However, the underlying assumption was that the activity of single neurons x_i is not vanishing for large networks, i.e., $x_i = O(1)$. This would imply $\|\bar{\kappa}\| = O(1)$ and $\rho = O(1/\sqrt{N})$. This is indeed what we observed for the more complex tasks when training networks of different sizes (not shown). We note that our results for the simple fixed point task deviate weakly from this (**Fig. 13** [↗](#) E, F). This hints at other factors pushing solutions to $\|\bar{\kappa}\| = O(1)$. One such factor may be that the loss function $L_{\text{var}}(\lambda_+)$ is very flat for λ_+ larger than the optimum (**Fig. 12C-D** [↗](#)). If learning pushes trajectories beyond the optimum at some point, e.g. due to large updates or if the optimum shifts over learning (**Fig. 26** [↗](#)), then the learning signal to reduce λ_+ afterward may be too small to yield visible effects in finite learning time.

In summary, the last two sections capture the main mechanisms that drive solutions to the oblique regime in nonlinear, noise-driven networks. Although marginally small solutions seem possible for large output weights, such solutions are close to a phase transition, so the resulting system is very susceptible to noise. To accommodate robust solutions, the trajectories (here the fixed points) must increase in magnitude, while rotating away from the output weights. This further allows the implementation of a negative feedback loop that suppresses noise along the output direction. Its efficacy can be reduced by too much saturation, which in turn keeps solutions from growing ever larger. The resulting sweet spot is a network with oblique dynamics.

4.10. Mechanisms behind decoupling of neural dynamics and output

Here we discuss in more detail the underlying mechanisms for the qualitative decoupling in oblique networks. We make a high-level argument that splits into two parts: first the possibility of decoupling in oblique, but not aligned, networks, and second a putative mechanism driving the decoupling.

For the first part, we observe that the output in oblique networks can be obtained from the leading components of the dynamics (along the PCs), but importantly also from the non-leading ones. To see this, we unpack the output in **Eq. (1)** [↗](#) in a slightly different way than before, **Eq. (3)** [↗](#). Namely, we split the activity vector $\mathbf{x}$ into its component along the leading PCs, $\mathbf{x}_{\text{lead}}$, and the remaining, trailing component, $\mathbf{x}_{\text{trail}}$. By definition of the leading PCs as the directions of largest variance, the leading component is expected to be large, and the trailing one small. Inserting this decomposition $\mathbf{x} = \mathbf{x}_{\text{lead}} + \mathbf{x}_{\text{trail}}$ into **Eq. (1)** [↗](#) leads to

$$z = \mathbf{w}_{\text{out}}^T (\mathbf{x}_{\text{lead}} + \mathbf{x}_{\text{trail}}) = \|\mathbf{w}_{\text{out}}\| (\rho_{\text{lead}} \|\mathbf{x}_{\text{lead}}\| + \rho_{\text{trail}} \|\mathbf{x}_{\text{trail}}\|), \quad (54)$$

with separately defined correlations $\rho_{\text{lead}} = \text{corr}(\mathbf{w}_{\text{out}}, \mathbf{x}_{\text{lead}})$ and $\rho_{\text{trail}} = \text{corr}(\mathbf{w}_{\text{out}}, \mathbf{x}_{\text{trail}})$.

For aligned networks, we recover the results from before: $\mathbf{w}_{\text{out}}$ is small so that the output can only be generated by the leading part with large correlation ρ_{lead} . The trailing part is unconstrained but is also not contributing to either the output or the leading dynamics, and hence not of interest.

For oblique networks, $\mathbf{w}_{\text{out}}$ is large so that the output can be generated by either of the two terms in Eq. (54). The correlation ρ_{lead} has to be small because else the output would be too large. The other correlation, ρ_{trail} , can be large, because non-dominant component $\mathbf{x}_{\text{trail}}$ is small. Both terms are potentially of the same magnitude, which means both can potentially contribute to the output. If the dominant part alone generates the output, then neural dynamics and output are coupled and the solution is similar to an aligned one (Fig. 14, right). If, however, the non-dominant part alone generates the output, and the correlation ρ_{lead} is so small that the dominant part does not contribute to the output, then the dominant part is not constrained by the task (Fig. 14, center right). In that case, the dominant dynamics and the output can decouple qualitatively, and we may see the large variability between learners observed above.

The existence of decoupled solutions for oblique dynamics leads to the second question: Why and when do such solutions arise? Understanding this requires more detailed insights into the learning process. Roughly speaking, learning in the oblique regime has two opposite goals: first, to generate the desired output as fast as possible, and hence to induce changes to activity that are as small as possible (Section 4.7); and second, to generate solutions that are robust and stable, and hence to induce changes in activity that are large enough to not be disrupted by noise (Section 4.9). During the process of learning, small, unstable solutions appear first (Fig. 14, center). These may be highly variable, depending strongly on random initialization or other randomness experienced during learning. Such solutions then slowly solidify into stable solutions, that may inherit the variability of the early solutions (Fig. 14, right).

The process of how learning transforms small, unstable solutions to larger, robust ones is analyzed in Sections 4.8 and 4.9. The details of how this process introduces variability between learners, however, are not discussed there and left for future work.

Data availability

The code for all simulations and generation of figures is available on github, https://github.com/frschu/aligned_oblique_in_rnn.

Acknowledgements

FS was supported by the Deutsche Forschungsgemeinschaft (German Research Foundation) under Germany's Excellence Strategy - EXC 2002/1 "Science of Intelligence" - project no. 390523135. OB was supported by the Israeli Science Foundation (grant 1442/21) and an HFSP research grant (RGP0017/2021). SO was supported by the program "Ecoles Universitaires de Recherche" ANR-17-EURE-0017

Author contributions

Conceptualization, F.S., F.M., S.O., and O.B.; Methodology, F.S.; Formal Analysis, F.S.; Investigation, F.S.; Writing - Original Draft, F.S., F.M., S.O., and O.B.; Writing - Review & Editing, F.S., F.M., S.O., and O.B.; Supervision, F.M., S.O., O.B..

Fig. 14

Path to oblique solutions for networks with large output weights. Left: all networks produce the same output (cf. **Fig. 1**). Center: Unstable solutions that arise early in learning. For lazy solutions, initial chaotic activity is slightly adapted, without changing the dynamics qualitatively. For marginal solutions, vanishingly small initial activity is replaced with very small dynamics sufficient to generate the output. Right: With more learning time and noise added during the learning process, stable, oblique solutions arise. The neural dynamics along the largest PCs can be either decoupled from the output (center right), or coupled (right). For decoupled dynamics, the components along the largest PCs (blue subspace) differ qualitatively from those generating the output (same as **Fig. 1B**, bottom). The dynamics along the largest PCs inherit task-unrelated components from the initial dynamics or randomness during learning. Another possibility are oblique, but coupled dynamics (right). Such solutions don't inherit task-unrelated components of the dynamics at initialization. They are qualitatively similar to aligned solutions, and the output is generated by a small projection of the output weights onto the largest PCs (dashed orange arrow).

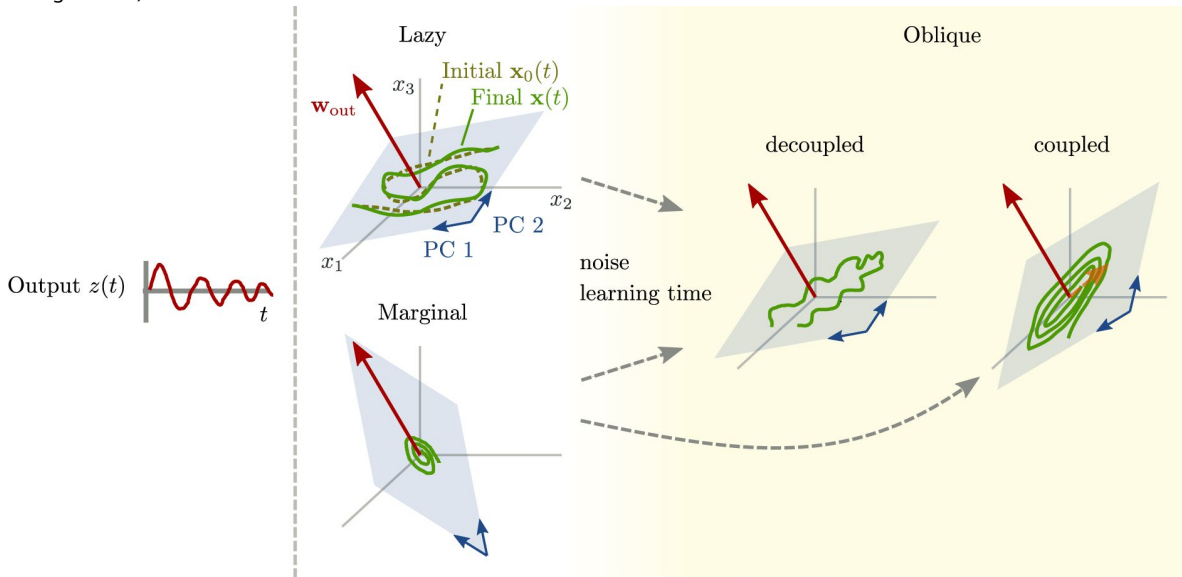
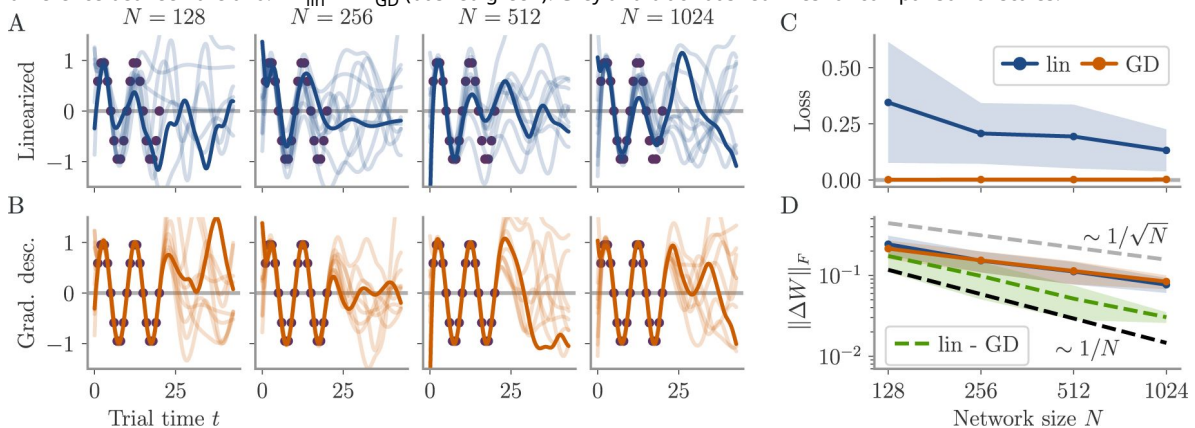


Fig. 15

Solution to linearized network dynamics in the lazy regime. **A** Network output for weight changes ΔW_{lin} obtained from linearized dynamics for different network sizes. Each plot shows 10 different networks (one example in bold). Target points in purple. **B** Output for networks trained with GD from the same initial conditions as those above. **C** Loss on the training set for the linear (lin) and GD solutions. **D** Frobenius norms of weight changes ΔW of the linear and GD solutions, as well as of the difference between the two. $\Delta W_{\text{lin}} - \Delta W_{\text{GD}}$ (dashed green). Grey and black dashed lines for comparison of scales.



Declaration of Interests

The authors declare no competing interests.

A. Supplementary Information

A.1 Linear approximation for lazy learning

We numerically investigated the linearization, Eq. (17), of the output trajectory $z(t)$ in weight changes ΔW , Eq. (17). For this, we used the sine wave task and the same configuration as for the lazy network in Fig. 9(c). In Fig. 15(a), we plot the output of the full network after inserting the linear solution ΔW_{lin} . The fit to the training points got more accurate with increasing network size. However, it did not reach zero error for the networks shown. In general, the error increased with increasing trial time. This indicates that the linear approximation was not valid. This is in line with the nature of the dynamical system: Trajectories become more nonlinear in W with increasing time. The loss on the training set [Fig. 15(c)] summarizes the observations seen before: The linear solution became more accurate with increasing network size.

How close was the linear solution to the one found by GD? To test this, we optimized the network with GD starting the same initial condition W_0 . The output of this network is shown in orange in Fig. 15(b). As seen before [Fig. 9(c)] training points were met, but the dynamics after the last training points remained chaotic. We then quantified the norms of the two solutions and the difference between both. As shown in Fig. 15(d), both solutions had similar norm, which decays as $1/\sqrt{N}$. The difference decayed faster, as $1/N$. This indicates that the linearized system can give a good insight into the solution found by training with GD, despite the inaccuracies of the linear approximation. In particular, the main features (scaling, unstable solutions, no extrapolation) are the same for the linear and the gradient descent solutions.

A.2. Details linear model: Bias only: lazy learning

We consider learning for the linear, input-driven model, Eq. (4). We start with the bias part, which is equivalent to the noise-free model [69]. Disregarding initial transients, the average output is $\bar{z} = \mathbf{w}_{\text{out}}^T B \mathbf{w}_{\text{in}}$, with $B = (1 - W)^{-1}$. The gradient of the loss $L_{\text{bias}} = (\bar{z} - \hat{z})^2$ is then

$$G_{\text{bias}} = \frac{dL_{\text{bias}}}{dW} = 2B^T \mathbf{w}_{\text{out}} (\bar{z} - \hat{z}) \mathbf{w}_{\text{in}}^T B^T. \quad (55)$$

We make the ansatz of small weight changes and hence linearize the output in weights (like in feedforward networks, [36]):

$$\bar{z} = \bar{z}_0 + G_{z,0} \cdot \Delta W + O(\Delta W^2), \quad (56)$$

with $G_{z,0}$ the gradient of the output at initialization,

$$G_{z,0} = \left. \frac{dz}{dW} \right|_0 = B_0^T \mathbf{w}_{\text{out}} \mathbf{w}_{\text{in}}^T B_0^T, \quad (57)$$

and $B_0 = (1 - W_0)^{-1}$. The dot-product signifies sum over all entries,

$A \cdot B = \sum_{ij} A_{ij} B_{ij}$ for two matrices A, B . Because the output is linear, the weight changes are spanned by the gradient, $\Delta W = b(\tau) G_{z,0} / \|G_{z,0}\|$. Insertion into gradient descent

dynamics yields

$$\dot{b}(\tau) = -2\eta m [\bar{z}_0 - \hat{z} - b(\tau)m], \quad (58)$$

with

$$m = \|G_{z,0}\| = \|B_0^T \mathbf{w}_{\text{out}}\| \|B_0^T \mathbf{w}_{\text{in}}\| = \frac{\|\mathbf{w}_{\text{out}}\| \|\mathbf{w}_{\text{in}}\|}{1 - g^2} = \frac{\sqrt{N}}{1 - g^2}. \quad (59)$$

We used that $\|B_0 \mathbf{v}\|^2 = \|B_0^T \mathbf{v}\|^2 = \frac{1}{1 - g^2} \|\mathbf{v}\|^2$ for any vector $\mathbf{v}$ independent of W_0 (which is the case for the input and output vectors) [69]. For initial condition $b(0) = 0$, the solution to Eq. (58) is

$$b(\tau) = \frac{\hat{z} - z_0}{m} (1 - e^{-2m^2 \eta \tau}). \quad (60)$$

The corresponding output changes are $\Delta \bar{z} = \bar{z} - \bar{z}_0 = b(\tau)m$. Because $m^2 \sim N$, the output converges to the target in $O(\eta N)$ learning steps. The resulting weights are

$$\Delta W = \frac{(\hat{z} - \bar{z}_0)(1 - g^2)^2}{\sqrt{N}} B_0^T \hat{\mathbf{w}}_{\text{out}} \hat{\mathbf{w}}_{\text{in}}^T B_0^T, \quad (61)$$

where the hats indicate normalized vectors. Consistent with linearization, the norm (both Frobenius and operator) of the changes is small:

$$\|\Delta W\| = \|\Delta W\|_2 = (\hat{z} - \bar{z}_0)(1 - g^2)/\sqrt{N} \sim 1/\sqrt{N}.$$

To connect with the following paragraph, we consider the case $g = 0$, so that $\Delta W = W$. This is for reference, and Fig. 10; see details below. We express the results in the orthonormalized bases introduced below:

$$\Delta W(\tau) = \frac{b_1(\tau)}{\sqrt{N}} \hat{\mathbf{w}}_{\text{out}} \hat{\mathbf{w}}_{\text{in},\perp}^T + O\left(\frac{1}{N}\right), \quad (62)$$

with

$$b_1(\tau) = (1 - e^{-2N\eta\tau})(\hat{z} - \sqrt{N}\rho_{io}). \quad (63)$$

The corresponding activity changes are $\Delta \bar{\mathbf{x}} = b_1(\tau) \hat{\mathbf{w}}_{\text{out}}$, with norm $\|\Delta \bar{\mathbf{x}}\| = b_1(\tau)$. The output changes are $\Delta z = b_1(\tau)$, so that the loss is $L_{\text{bias}} = [b_1(\tau) + \sqrt{N}\rho_{io} - \hat{z}]^2 = e^{-4N\eta\tau}(\hat{z} - \sqrt{N}\rho_{io})^2$.

A.3. Details linear model:, Bias and variance combined

We now combine results from the previous section, concerning the bias, with results from the Methods, concerning the variance, to derive the full learning dynamics of the linear model. For the case of no initial connectivity, $W_0 = 0$, the gradients for both the bias and the variance component, Eqs. (23) and (55), are spanned by the output and input vector. By orthonormalizing, we define $\hat{U} = [\hat{\mathbf{w}}_{\text{out}}, \hat{\mathbf{w}}_{\text{in},\perp}]$ with $\hat{U}^T \hat{U} = \mathbf{1}_2$. The weights are then constrained to $W = \hat{U} M \hat{U}^T$. For ease of writing, we define the vectors $\mathbf{v}_{\text{out}} = \hat{U}^T \hat{\mathbf{w}}_{\text{out}} = [1, 0]^T$ and

$$\mathbf{v}_{\text{in}} = \hat{U}^T \hat{\mathbf{w}}_{\text{in}} = [\rho_{io}, \sqrt{1 - \rho_{io}^2}]^T, \text{ where}$$

$\rho_{io} = \hat{\mathbf{w}}_{\text{out}}^T \hat{\mathbf{w}}_{\text{in}} \sim \mathcal{N}(0, 1/N)$ is the random, $O(1/\sqrt{N})$ correlation between input and output vector. As before, the norms of input and output vectors are $\|\mathbf{w}_{\text{in}}\| = \sqrt{N}$ and $\|\mathbf{w}_{\text{out}}\| = \sigma_{\text{out}}$ and $\|\mathbf{w}_{\text{out}}\| = \sigma_{\text{out}}$.

The projection of the deterministic gradient is then

$$G_{\text{det}, \hat{U}} = \hat{U}^T G_{\text{det}} \hat{U} = 2(\bar{z} - \hat{z}) \sigma_{\text{out}} B_{\hat{U}}^T \mathbf{v}_{\text{out}} \mathbf{v}_{\text{in}}^T B_{\hat{U}}^T, \quad (64)$$

with deterministic output

$$\bar{z} = \sigma_{\text{out}} \mathbf{v}_{\text{out}}^T B_{\hat{U}} \mathbf{v}_{\text{in}}, \quad (65)$$

and projection $B_{\hat{U}} = \hat{U}^T B \hat{U} = (1 - M)^{-1}$.

For the stochastic part, we need to solve the two Lyapunov **Eqs. (21)** and **(24)**. They reduce to low-D versions after projection. For the first one, we have

$$\Sigma = \sigma_{\text{noise}}^2 \left[\mathbb{1}_N + \hat{U}(-\mathbb{1}_2 + \Sigma_{\hat{U}}) \hat{U}^T \right], \quad (66)$$

and the projection solves the 2-dimensional Lyapunov equation

$$A_{\hat{U}} \Sigma_{\hat{U}} + \Sigma_{\hat{U}} A_{\hat{U}}^T + 2\mathbb{1}_2 = 0, \quad (67)$$

with $A_{\hat{U}} = -\mathbb{1}_2 + M$. Similarly, the low-D version of the second Lyapunov **Eq. (24)** is obtained by defining $\Omega = \sigma_{\text{out}}^2 \hat{U} \Omega_{\hat{U}} \hat{U}^T$, with

$$A_{\hat{U}} \Omega_{\hat{U}} + \Omega_{\hat{U}} A_{\hat{U}}^T + \mathbf{v}_{\text{out}} \mathbf{v}_{\text{out}}^T = 0. \quad (68)$$

The full learning dynamics for gradient flow are then given by

$$\dot{M} = -\eta (G_{\text{det}, \hat{U}} + G_{\text{sto}, \hat{U}}), \quad (69)$$

with $M(0) = 0$ and

$$G_{\text{sto}, \hat{U}} = \hat{U}^T G_{\text{sto}} \hat{U} = 2\sigma_{\text{noise}}^2 \sigma_{\text{out}}^2 \Omega_{\hat{U}} \Sigma_{\hat{U}}. \quad (70)$$

For a small output scale, the deterministic part is order one, while the stochastic one is order $1/N$ (through Ω). Assuming $\eta = O(1)$, learning thus reaches an arbitrary small order-one loss after $O(1)$ updates, with noise adding a $1/N$ stochastic loss, and noise-driven learning equally adding order $1/N$ deviations to the entries of M . Hence, learning remains unaffected by the noise unless it is continued for $O(\eta N^2)$ steps.

For a large output scale, the deterministic part is fast but only leads to $1/\sqrt{N}$ entries. In such a situation, the stochastic part can be expanded in orders of $\epsilon = 1/\sqrt{N}$. To leading order, it yields dynamics along the first entry of M .

In detail: For a large readout, $\sigma_{\text{out}} = 1$, we expand M to first order in $\epsilon = \frac{1}{\sqrt{N}}$:

$$M = \begin{bmatrix} a_0 & b_0 \\ c_0 & d_0 \end{bmatrix} + \epsilon \begin{bmatrix} a_1 & b_1 \\ c_1 & d_1 \end{bmatrix}. \quad (71)$$

The leading order of the output is

$$\bar{z} = \frac{1}{\epsilon} \frac{b_0}{1 - a_0 + a_0 d_0 - b_0 c_0 - d_0} + \mathcal{O}(1), \quad (72)$$

which implies that $b_0 = 0$ throughout learning. Under this assumption, we have

$$\bar{z} = \frac{b_1 + (1 - d_0)\sqrt{N}\rho_{io}}{(1 - a_0)(1 - d_0)} + \mathcal{O}(\epsilon). \quad (73)$$

Furthermore, the deterministic gradient is

$$G_{\text{det}, \hat{U}} = \frac{2}{\epsilon} (\bar{z} - \hat{z}) \begin{bmatrix} 0 & -\frac{\hat{z}}{(1-a_0)(1-d_0)} \\ 0 & 0 \end{bmatrix} + \mathcal{O}(1). \quad (74)$$

This component of learning converges on the order of η/ϵ^2 learning steps due to the prefactor $1/\epsilon$ and the fact that b_1 enters $\bar{z}$ at order one. The higher order terms therefore only lead to $\mathcal{O}(\epsilon^2)$ corrections.

The stochastic part of the gradient simplifies by assuming $b_0 = c_0 = d_0 = 0$. The latter two are justified self-consistently because neither the deterministic nor the stochastic part contribute to c and d at order one if we start with $M = 0$. With this assumption, we have

$$G_{\text{sto}, \hat{U}} = \sigma_{\text{noise}}^2 \frac{1}{(1 - a_0)^2} \left(\begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} + \epsilon \begin{bmatrix} \frac{2a_1}{1-a_0} & \frac{b_1(1-a_0)+c_1(2-a_0)}{2-a_0} \\ \frac{c_1}{2-a_0} & 0 \end{bmatrix} + \mathcal{O}(\epsilon^2) \right). \quad (75)$$

Consistent with our assumption, we only have order-one growth for the entry a . The resulting dynamics are

$$a_0(\tau) = 1 - (3\eta\sigma_{\text{noise}}^2\tau + 1)^{-\frac{1}{3}}, \quad (76)$$

where τ is the number of learning steps. Further, because the deterministic part is so much faster, we can assume that the second entry, $b = b_1$, is always constrained by the target. Namely, from **Eq. (73)**, we have

$$b_1 = \hat{z}(1 - a_0) - \sqrt{N}\rho_{io}. \quad (77)$$

Lastly, the first-order corrections for the other entries all vanish due to vanishing initial conditions:

$$a_1 = c_1 = d_1 = 0. \quad (78)$$

The loss due to the stochastic part is then

$$L_{\text{var}} = \sigma_{\text{noise}}^2 (3\eta\sigma_{\text{noise}}^2\tau + 1)^{-\frac{1}{3}}, \quad (79)$$

while the deterministic part is kept at $O(1/N)$.

During learning, the average states change only very little. To see this, we express the connectivity as $W = \hat{\mathbf{w}}_{\text{out}} \mathbf{w}_{\text{r}}^T$ with

$$\mathbf{w}_{\text{r}} = \lambda_- \hat{\mathbf{w}}_{\text{out}}^T + \frac{b_1}{\sqrt{N}} \hat{\mathbf{w}}_{\text{in},\perp}, \quad (80)$$

and orthonormalized input weights

$$\hat{\mathbf{w}}_{\text{in},\perp} = \frac{\hat{\mathbf{w}}_{\text{in}} - \hat{\mathbf{w}}_{\text{out}} \rho_{io}}{\sqrt{1 - \rho_{io}^2}} = \hat{\mathbf{w}}_{\text{in}} - \hat{\mathbf{w}}_{\text{out}} \rho_{io} + O(1/N). \quad (81)$$

The output is then

$$\begin{aligned} \bar{\mathbf{x}} &= (1 - W)^{-1} \mathbf{w}_{\text{in}} \\ &= \left(\mathbf{1} + \frac{\hat{\mathbf{w}}_{\text{out}} \mathbf{w}_{\text{r}}^T}{1 - a_0} \right) \mathbf{w}_{\text{in}} \\ &= \mathbf{w}_{\text{in}} + \hat{\mathbf{w}}_{\text{out}} \frac{a_0 \rho_{io} \sqrt{N} + b_1}{1 - a_0}. \end{aligned} \quad (82)$$

The initial states are $\bar{\mathbf{x}}_0 = \mathbf{w}_{\text{in}}$, so the changes are

$$\Delta \bar{\mathbf{x}} = \frac{a_0 \rho_{io} \sqrt{N} + b_1}{1 - a_0} \hat{\mathbf{w}}_{\text{out}} = (\hat{z} - \rho_{io} \sqrt{N}) \hat{\mathbf{w}}_{\text{out}}, \quad (83)$$

where the scalar part is also the norm. Thus, $\|\Delta \bar{\mathbf{x}}\| \sim 1$, which is small, because the entries are $O(1/\sqrt{N})$. Paradoxically, the norm does not change with learning time. Essentially, once the lazy learning takes place at the beginning of learning, the norm does not change anymore (to leading order).

A.4. Details nonlinear autonomous system with noise

This section contains detailed derivations for the average and fluctuations of the latent projection κ . The theoretical treatment of the network dynamics is very similar to that for a network with only a negative feedback loop for “balance” by Kadmon et al. [29]. What is added here is a colored noise term that enters via the variance $\sigma_{\perp}^2$ of the orthogonal dynamics. This is dependent on a nonzero fixed point (not present in Ref. [29]).

The dynamics of the latent variable are

$$\dot{\kappa} = -\kappa + M \frac{1}{N} U^T \phi(U \kappa + \mathbf{x}_{\perp}) + \frac{1}{N} U^T \xi. \quad (84)$$

That is, the low-D variable is driven autonomously by itself, but externally by the high-D orthogonal part $\mathbf{x}_{\perp}$. To resolve this into low-D dynamics only, we apply two averages: the trial-conditioned average, which is over the noise ξ , and denoted by the bar, $\bar{\cdot} = \mathbb{E}_{\xi}[\cdot]$; and the average over the statistics of U , which arises naturally from the projection on U (a population average). We start with the latter, writing

$$\frac{1}{N} U^T \phi(\mathbf{x}) = \frac{1}{N} \mathbb{E}_U[U^T \phi(\mathbf{x})] + \epsilon. \quad (85)$$

One can show that the error is small, $\mathbb{E}_U[\epsilon_a] = 0$ and $\mathbb{E}_U[\epsilon_a \epsilon_a] \sim 1/N$. For the average, we apply partial integration as in the noise-free case, Eq. (14), and arrive at

$$\frac{1}{N} \mathbb{E}_U[U^T \phi(\mathbf{x})] = \langle \phi'(\sigma_x u) \rangle \boldsymbol{\kappa}, \quad (86)$$

where the average on the right-hand side is over the standard normal variable u . In contrast to the noise-free case, the variance is now comprised of two terms,

$$\sigma_x(t)^2 = \|\boldsymbol{\kappa}(t)\|^2 + \sigma_{\perp}(t)^2, \quad (87)$$

and $\sigma_{\perp}^2(t) = \frac{1}{N} \mathbf{x}_{\perp}(t)^T \mathbf{x}_{\perp}(t)$ is the population average of $\mathbf{x}_{\perp}(t)$. We included the time index here to emphasize that both variables still contain fluctuations around the trial-conditioned average.

We now split the term Eq. (85) into average and fluctuations around this. We self-consistently assume that all fluctuations, denoted with a δ , are small, which allows us to linearize around the respective averages. We have

$$\frac{1}{N} U^T \phi(\mathbf{x}) = \overline{\langle \phi'(\sigma_x u) \rangle} \bar{\boldsymbol{\kappa}} + \bar{\boldsymbol{\epsilon}} + \overline{\langle \phi'(\sigma_x u) \rangle} \delta \boldsymbol{\kappa} + \delta \langle \phi'(\sigma_x u) \rangle \bar{\boldsymbol{\kappa}}. \quad (88)$$

We only keep terms up to order $\mathcal{O}(1/\sqrt{N})$. The fluctuations of the error are $\delta \boldsymbol{\epsilon} \sim 1/N$ and hence discarded. By linearization, we further have $\overline{\langle \phi'(\sigma_x u) \rangle} = \langle \phi'(\bar{\sigma}_x u) \rangle$, with

$$\overline{\sigma_x^2} = \|\bar{\boldsymbol{\kappa}}\|^2 + \overline{\sigma_{\perp}^2}. \quad (89)$$

The orthogonal part is an Ornstein-Uhlenbeck process with steady-state covariance

$$\overline{x_{\perp i}(t) x_{\perp j}(t+s)} = \delta_{ij} \overline{\sigma_{\perp}^2} e^{-|s|}, \quad (90)$$

where the variance is inherited from the white noise, $\overline{\sigma_{\perp}^2} = \sigma_{\text{noise}}^2$.

We take the average over the latent dynamics Eq. (84). Apart from an initial transient, which we discard, this leaves us with a fixed point

$$\bar{\boldsymbol{\kappa}} = \langle \phi'(\bar{\sigma}_x u) \rangle M \bar{\boldsymbol{\kappa}} + \bar{\boldsymbol{\epsilon}}. \quad (91)$$

We now discuss the fluctuations in Eq. (88). There are two sources of fluctuations, $\delta \boldsymbol{\kappa}$ and $\delta \sigma_{\perp}^2$, so that we have

$$\delta \langle \phi' \rangle = \partial_{\sigma_x} \langle \phi' \rangle \left(\partial_{\boldsymbol{\kappa}}^T \sigma_x \delta \boldsymbol{\kappa} + \partial_{\sigma_{\perp}^2} \sigma_x \delta \sigma_{\perp}^2 \right). \quad (92)$$

All derivatives are evaluated at the averages (we discard the bar in the following lines to keep notation at bay). We further compute

$$\partial_{\sigma_x} \langle \phi'(\sigma_x u) \rangle = \langle \phi''(\sigma_x u) u \rangle = \sigma_x \langle \phi'''(\sigma_x u) \rangle, \quad (93)$$

$$\partial_{\kappa} \sigma_x = \frac{1}{\sigma_x} \kappa, \quad (94)$$

$$\partial_{\sigma_x^2} \sigma_x = \frac{1}{2\sigma_x}. \quad (95)$$

Inserting these terms into the dynamics Eq. (84), we arrive at

$$\frac{d \delta \kappa(t)}{dt} = (-\mathbb{1} + \langle \phi' \rangle M + \langle \phi''' \rangle M \bar{\kappa} \bar{\kappa}^T) \delta \kappa(t) + \frac{\langle \phi''' \rangle}{2} M \bar{\kappa} \delta \sigma_x^2(t) + \frac{1}{N} U^T \xi. \quad (96)$$

The white noise term has variance

$$\frac{1}{N^2} \overline{U^T \xi \xi^T U} = \frac{2\sigma_{\text{noise}}^2}{N} \mathbb{1}_2. \quad (97)$$

For the fluctuations in the population average of the variance, we use Eq. (90) to compute the covariance function between different time points,

$$\begin{aligned} \overline{\delta \sigma_x^2(t) \delta \sigma_x^2(t+s)} &= \frac{1}{N^2} \sum_{ij} \overline{x_{\perp i}(t)^2 x_{\perp j}(t+s)^2} - (\overline{\sigma_x^2})^2 \\ &= (\overline{\sigma_x^2})^2 \left(\frac{N(N-1)}{N^2} - 1 \right) + \frac{1}{N} \sum_i \overline{x_{\perp i}(t)^2 x_{\perp i}(t+s)^2} \\ &= \frac{\sigma_{\text{noise}}^4}{N} \left[-1 + \langle u^2 (e^{-|s|} u + \sqrt{1 - e^{-2|s|}} u')^2 \rangle \right] \\ &= \frac{2\sigma_{\text{noise}}^4}{N} e^{-2|s|}. \end{aligned} \quad (98)$$

For the third line, the average is taken over the independent standard normal variables u, u' . Putting all the pieces together, we have

$$\frac{d \delta \kappa(t)}{dt} = A \delta \kappa(t) + \frac{1}{\sqrt{N}} \zeta(t), \quad (99)$$

with Jacobian

$$A = -\mathbb{1} + \langle \phi' \rangle M + \frac{\langle \phi''' \rangle}{\langle \phi' \rangle} \bar{\kappa} \bar{\kappa}^T, \quad (100)$$

and noise term $\zeta(t)$ with zero average and covariance

$$\overline{\zeta(t) \zeta(t+s)^T} = \left(2\sigma_{\text{noise}}^2 \delta(s) \mathbb{1}_2 + \beta e^{-2|s|} \bar{\kappa} \bar{\kappa}^T \right), \quad (101)$$

with

$$\beta = \frac{\sigma_{\text{noise}}^4}{2} \left(\frac{\langle \phi''' \rangle}{\langle \phi' \rangle} \right)^2. \quad (102)$$

We inserted the eigenvalue equation, $M\bar{\kappa} = \frac{1}{\langle\phi'\rangle}\bar{\kappa}$. We note that the colored noise term did not appear in previous studies such as [29], because there was no positive feedback and corresponding fixed point $\bar{\kappa}$.

We now compute the variance of the fluctuations $\delta\kappa$. The computation is simplified by the fact the $\bar{\kappa}$ is also eigenvector of A , with eigenvalue

$$\gamma_+ = \frac{\langle\phi'''\rangle}{\langle\phi'\rangle} \|\bar{\kappa}\|^2. \quad (103)$$

The eigenvector is also the direction in which the colored noise acts. Because of this, we can write

$$\begin{aligned} \overline{\delta\kappa(t)\delta\kappa(t)^T} &= \frac{1}{N} \int_0^t dt' \int_0^t dt'' e^{A(t-t')} \overline{\zeta(t')\zeta(t'')^T} e^{A^T(t-t')} \\ &= \frac{1}{N} \left[\sigma_{\text{noise}}^2 \Sigma_A(t) + \beta \bar{\kappa} \bar{\kappa}^T \underbrace{\int_0^t dt' \int_0^t dt'' e^{\gamma_+(2t-t'-t'')-2|t'-t''|}}_{I(t)} \right]. \end{aligned} \quad (104)$$

The first summand stems from the white noise term,

$$\Sigma_A(t) = 2 \int_0^t dt' e^{A(t-t')} e^{A^T(t-t')}. \quad (105)$$

The integral $I(t)$ can be computed to yield

$$I(t) = \frac{e^{2\gamma_+ t}}{\gamma_+(2-\gamma_+)} \left[1 - e^{-2\gamma_+ t} - \frac{2\gamma_+}{2+\gamma_+} \left(1 - e^{-(2+\gamma_+)t} \right) \right]. \quad (106)$$

For the steady-state variance, we take $t \rightarrow \infty$. The first term yields $\lim_{t \rightarrow \infty} \Sigma_A(t) = \Sigma_A$, which can be expressed as the solution of the Lyapunov equation

$$0 = A\Sigma_A + \Sigma_A A^T + 2\mathbb{I}_2. \quad (107)$$

The integral converges to

$$\lim_{t \rightarrow \infty} I(t) = \frac{1}{-\gamma_+(2-\gamma_+)}. \quad (108)$$

Joining all the bits and pieces, we have the steady-state covariance

$$\overline{\delta\kappa\delta\kappa^T} = \frac{1}{N} \left[\sigma_{\text{noise}}^2 \Sigma_A + \frac{\sigma_{\text{noise}}^4}{\|\bar{\kappa}\|^2} \frac{-\gamma_+}{2(2-\gamma_+)} \mathbf{v}_+ \mathbf{v}_+^T \right], \quad (109)$$

with normalized eigenvector

$$\mathbf{v}_+ = \frac{\bar{\kappa}}{\|\bar{\kappa}\|}. \quad (110)$$

For the loss, we only need to consider the variance along the output vector. Namely, we have the output fluctuations

$$\delta z = \sqrt{N} \|\mathbf{w}_{\text{out}}\| \mathbf{v}_{\text{out}}^T \delta\kappa, \quad (111)$$

with projected output weights

$$\mathbf{v}_{\text{out}} = \frac{1}{\sqrt{N}} U^T \mathbf{w}_{\text{out}} / \|\mathbf{w}_{\text{out}}\|. \quad (112)$$

Thus,

$$L_{\text{var}} = \overline{\delta z^2} = N \mathbf{v}_{\text{out}}^T \overline{\delta \boldsymbol{\kappa} \delta \boldsymbol{\kappa}^T} \mathbf{v}_{\text{out}}. \quad (113)$$

A.5. Additional figures

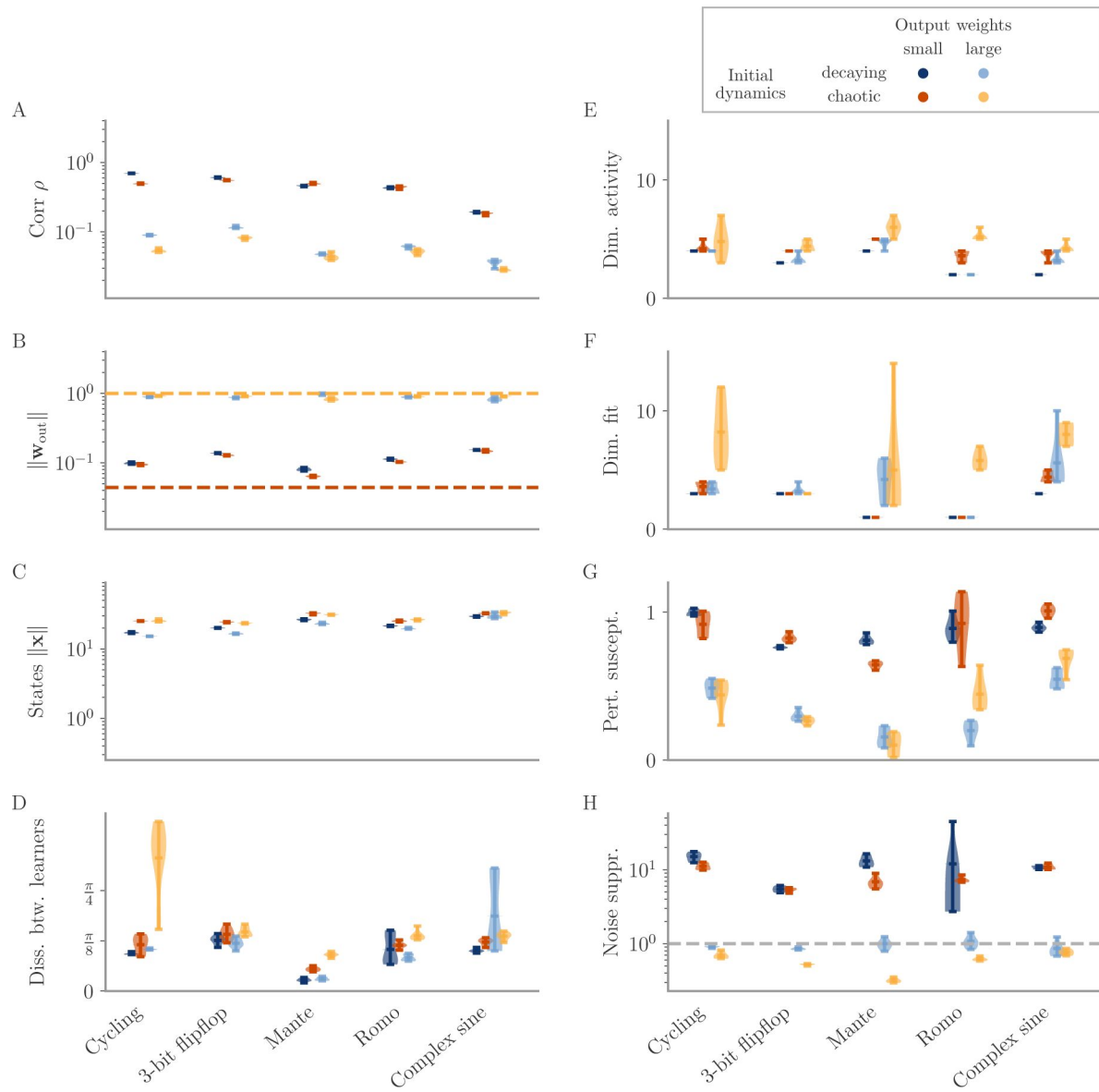


Fig. 16

Summary of all measures for initially decaying or chaotic networks.

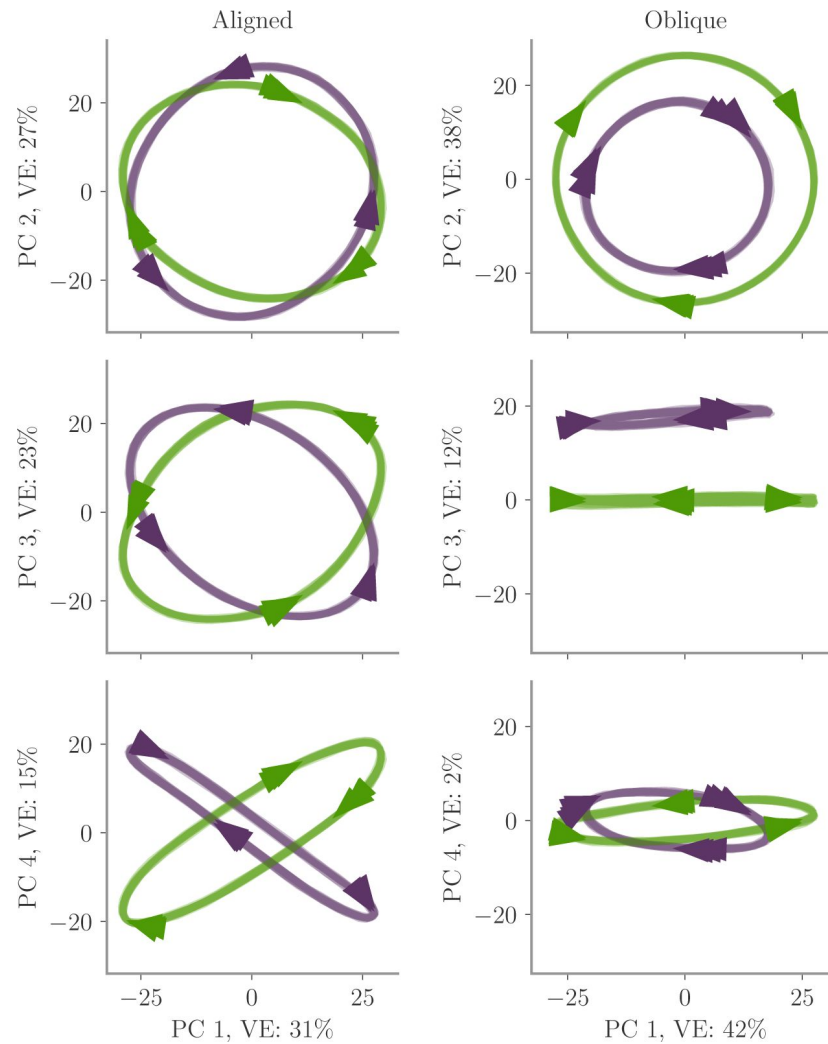


Fig. 17

Projection of neural dynamics onto first 4 PCs for the cycling task, **Fig. 2**. The x-axis for all plots is PC 1 of the respective dynamics (left: aligned, right: oblique). The y-axes are PCs 2 to 4. Axis labels indicate the relative variance explained by each PC. Arrows indicate direction. Note that there is co-rotation for the aligned network for PCs 1 and 3, as well as the counter-rotation for the oblique network for PCs 1 and 4.

Fig. 18

Power spectral densities for the six networks shown in **Fig. 4**. The dashed orange line indicates the output frequency. Note the high power for non-target frequencies in the first PCs in some of the large output solutions.

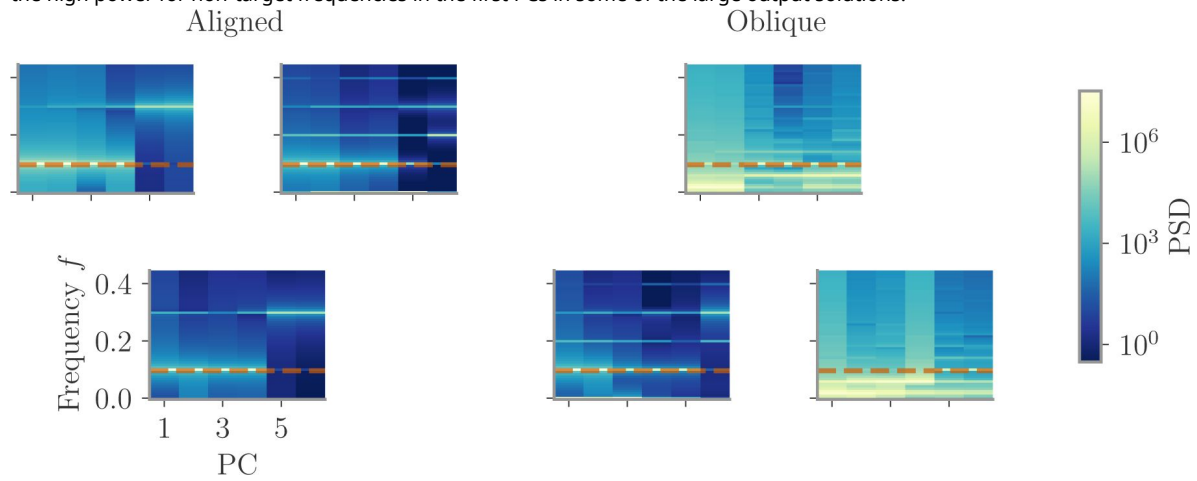
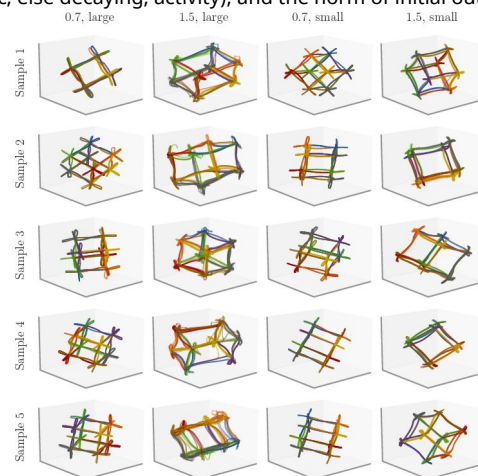


Fig. 19

Example of task variability for the flipflop task. The titles in each row indicate the spectral radius g of the initial recurrent connectivity ($g > 1$ for initially chaotic, else decaying, activity), and the norm of initial output weights.



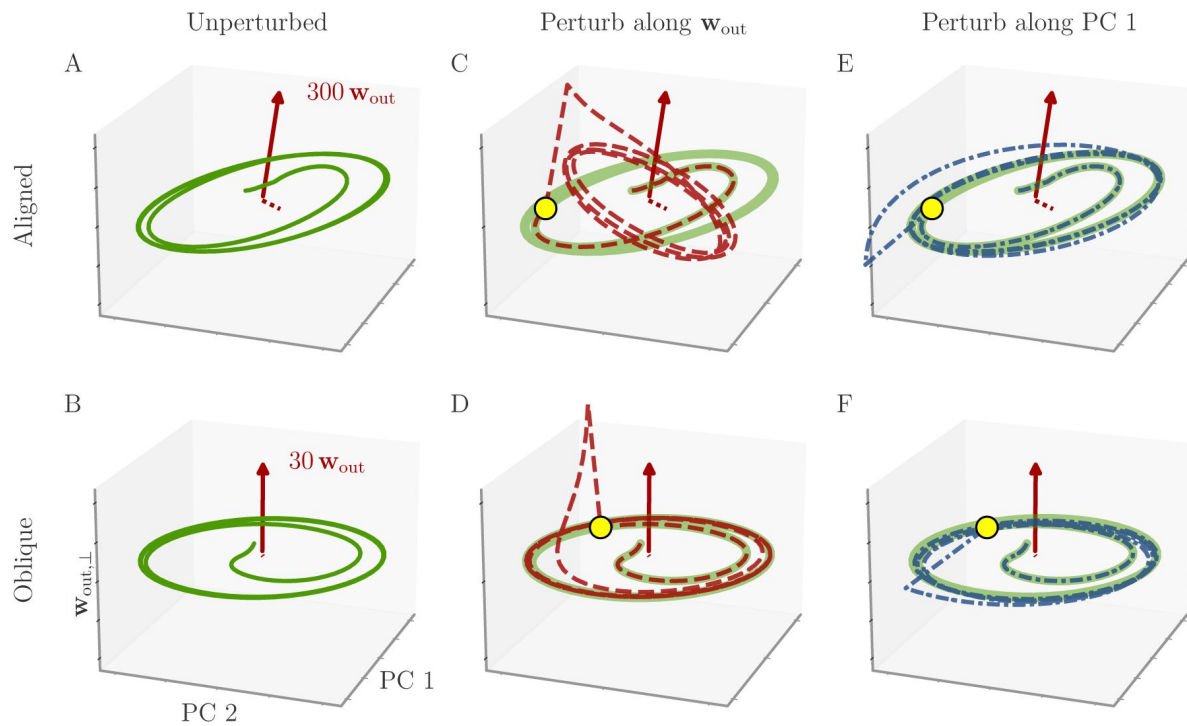


Fig. 20

Neural activity in response to the perturbations applied in [Fig. 5](#). Activity is plotted in the space spanned by the leading two PCs and the output weights $w_{out,1}$. We first show the unperturbed trajectories in each network (left), then the perturbed ones for perturbations along the first output direction (center) and along the first PC (right). The unperturbed trajectories are also plotted for comparison. Yellow dots indicate the point where the perturbation is applied. All perturbations but the one along the output for aligned lead to trajectories on the same attractor, but potentially with a phase shift. Note that in general, perturbations can also lead to the activity converging on a different attractor. Here, we see a specific example of this happening for the cycling task in the aligned regime.

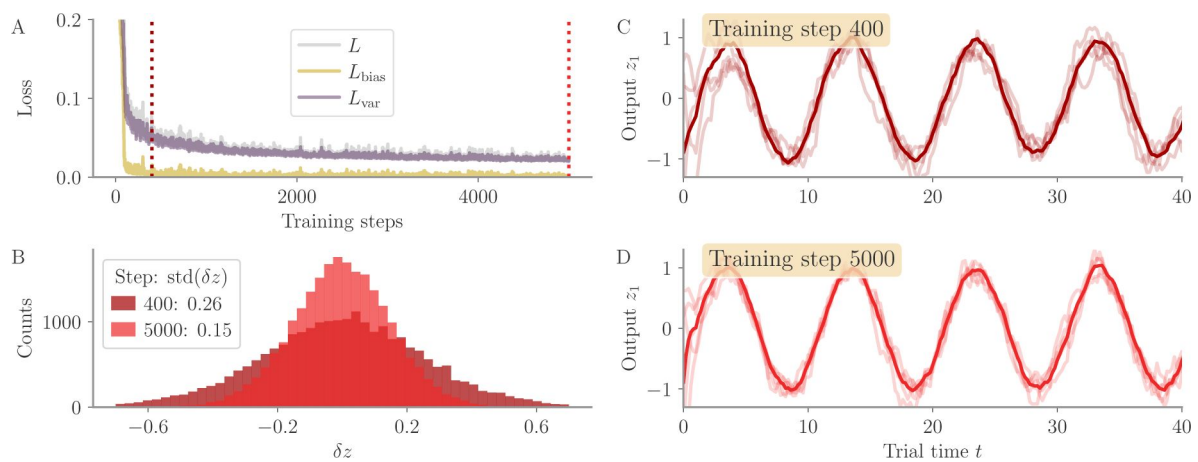


Fig. 21

Noise compression over training time for the cycling task. Example networks trained on the cycling tasks with $\sigma_{\text{noise}} = 0.2$ and $N = 256$. The network at the end of training is analyzed in [Fig. 6B](#). **A** Full loss (grey) and decomposition in bias (golden) and variance (purple) parts over learning time. The bias part decays rapidly (the y-axis is clipped, initial loss $L_0 = 1.4$), whereas the variance part needs many more training steps to decrease. Dotted lines indicate the two examples in **B-D**. **B** Output fluctuations $\delta z_i(t) = z_i(t) - \bar{z}_i(t)$ around the trial-conditioned average $\bar{z}_i$. Mean is over 16 samples for each of the two trial conditions (clockwise and anticlockwise rotation). Because both output dimensions $i \in \{1, 2\}$ are equivalent in scale, we collected both for the histogram. **C-D** Example output trajectories early (**C**) and late (**D**) in learning. Shown are the mean (dark) and 5 samples (light).

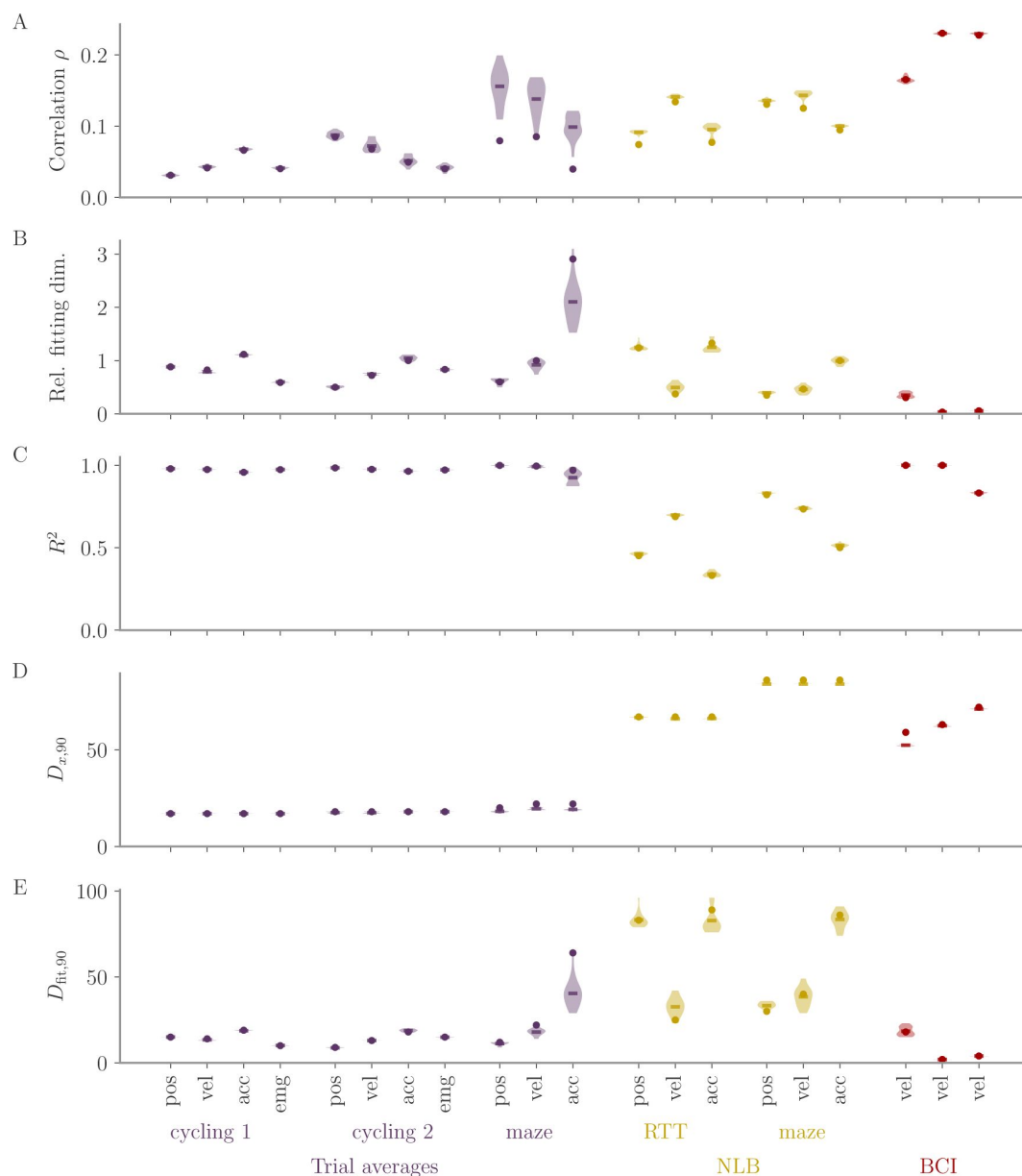


Fig. 22

Fitting neural activity to different output modalities (hand position, velocity, acceleration, EMG). Output modality is indicated by the x-ticks, the corresponding datasets by the color and the labels below. A, B: Correlation and relative fitting dimension. Similar to **Fig. 7B**, where these are shown for velocity alone. C-E: Additional details. C: Coefficient of determination R^2 of the linear regression. D: Number of PCs necessary to reach 90 % of the variance of the neural activity X . E: Number of PCs necessary to reach 90 % of the R^2 value of the full neural activity. For each output modality, the delay between activity and output is optimized. Position decodes earlier (300–200ms) than velocity or acceleration (100–50ms); no delay for EMG. The data X is the same with each dataset apart from a potential shift by the respective delay, so that dimension $D_{x,90}$ in D is almost the same. Note that we also computed trial averages for the NLB maze task to test for the effect of trial averaging. These, however, have a small sample number (189 data points, but 162 neurons), which leads to considerable discrepancy between the dots (full data) and the subsamples with only 25% of the data points, for example in the correlation (A).

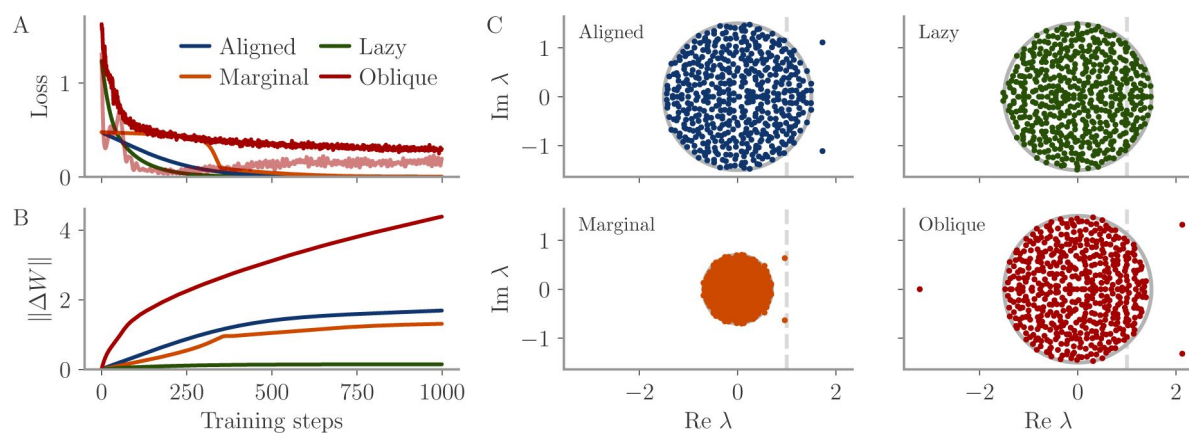


Fig. 23

Learning curves and eigenvalue spectra for sine wave task. **(a)** Loss over training steps for four networks. The light red line is the bias term of the loss for the oblique network. **(b)** Norm of weight changes over learning time. **(c)** Eigenvalue spectra of connectivity matrix W_f after training. The dashed line indicates the stability line for the fixed point at the origin.

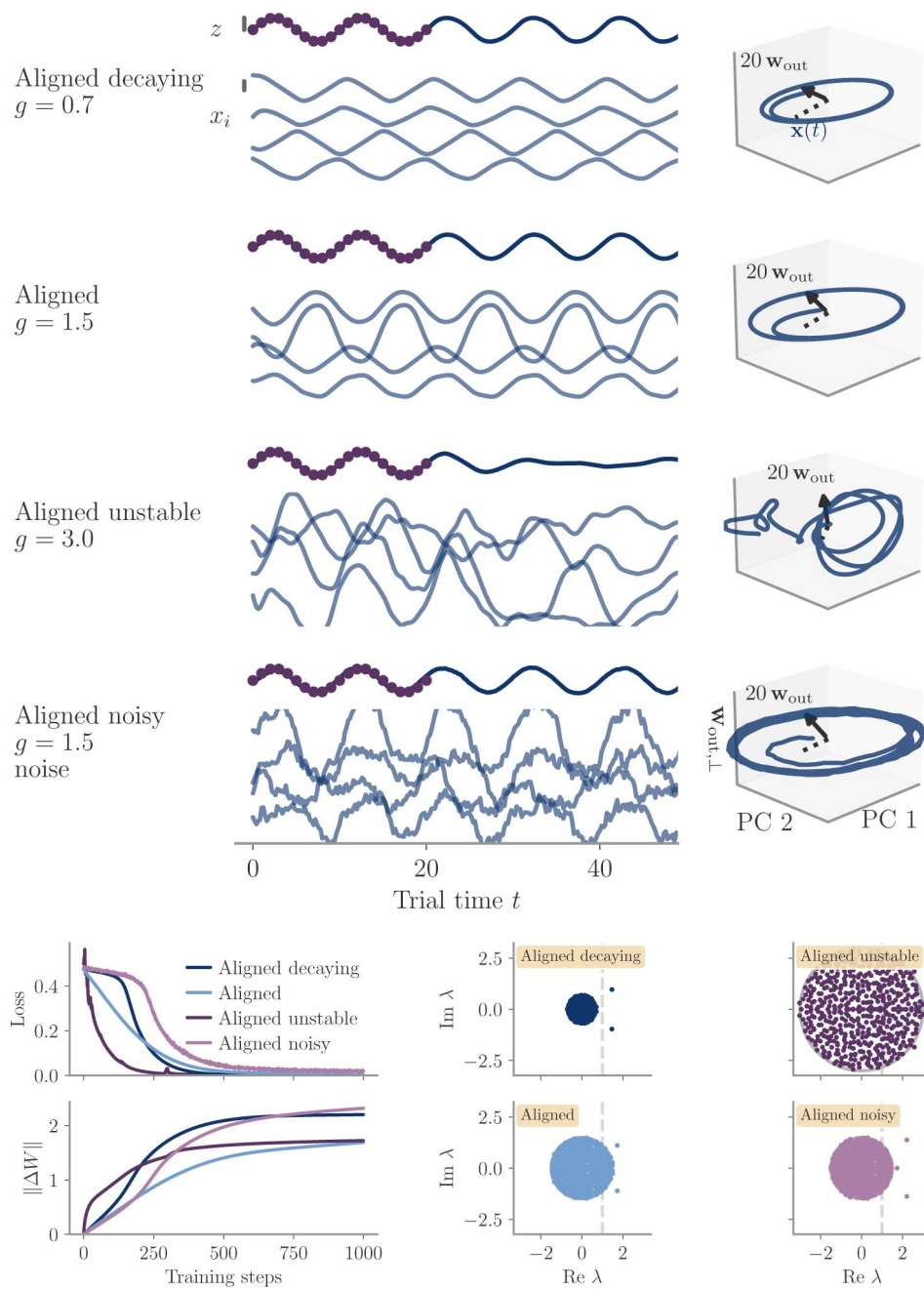


Fig. 24

The aligned regime is robust to the choice of other hyperparameters.

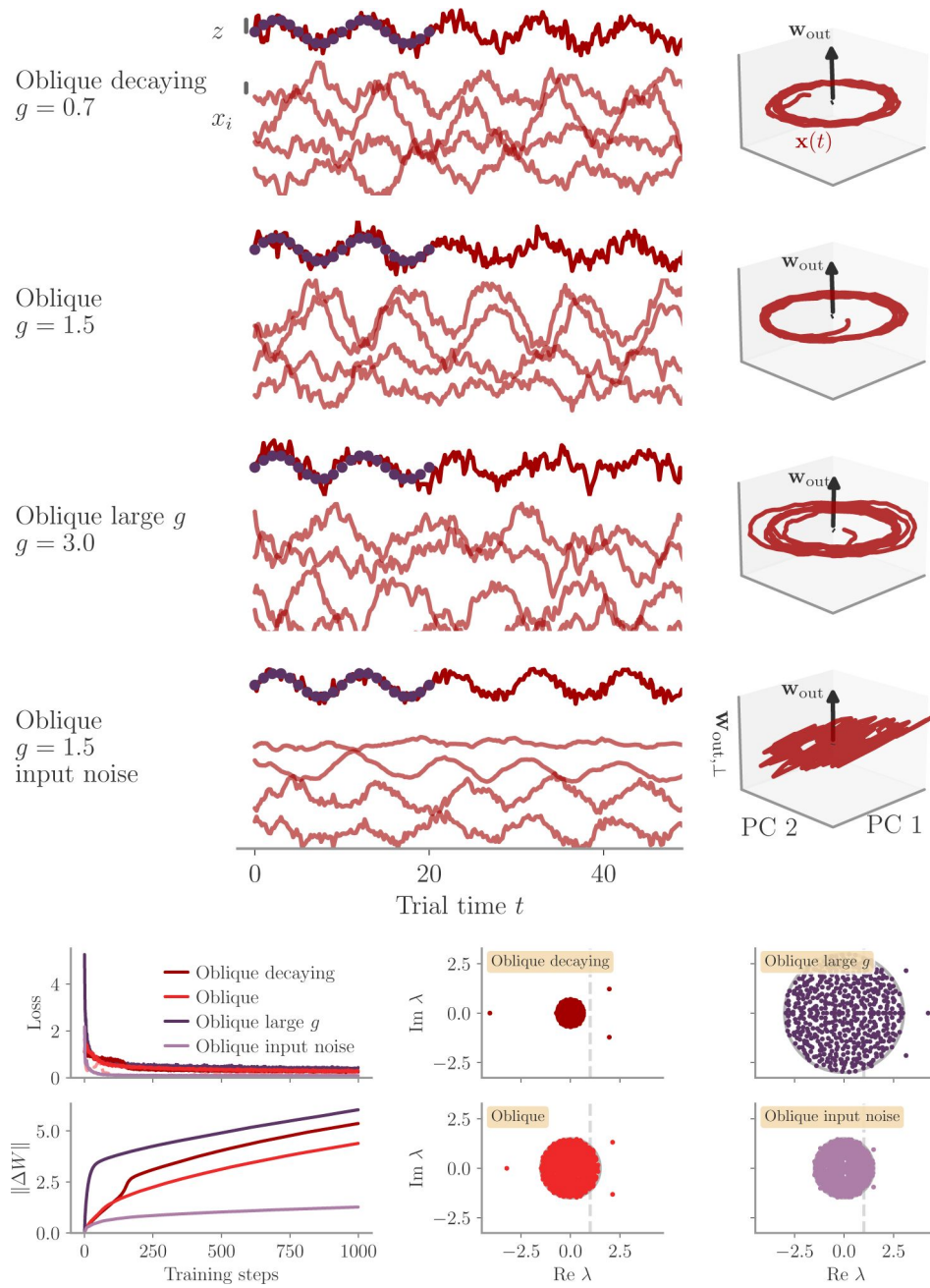


Fig. 25

The oblique regime is robust to the choice of other hyperparameters.

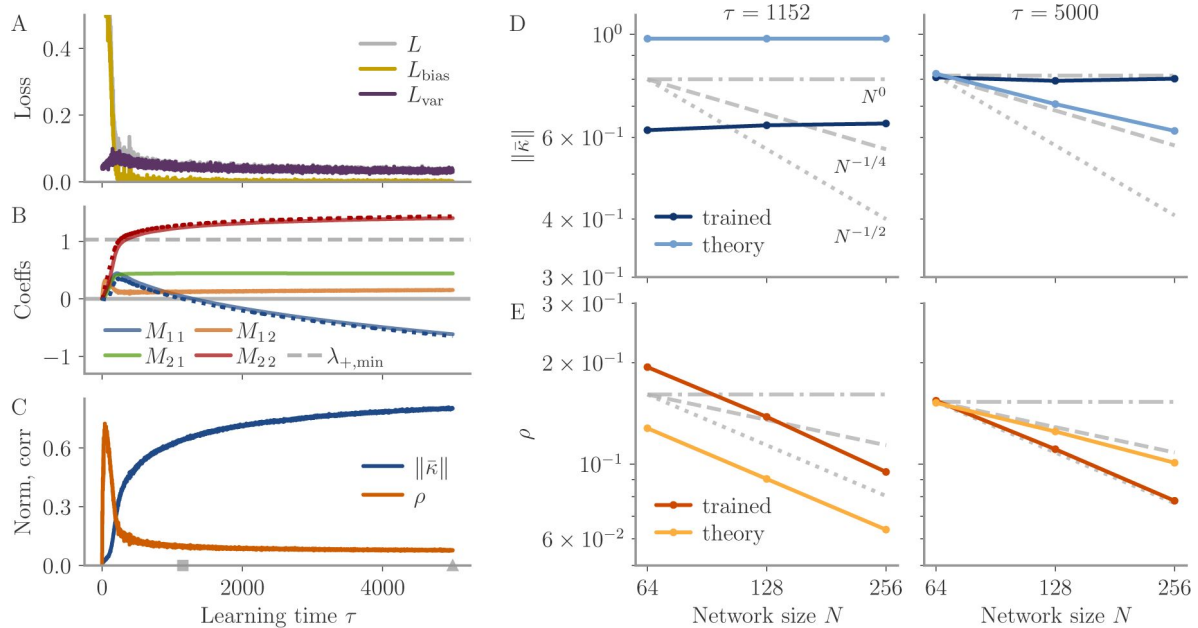


Fig. 26

Training history leads to order-one fixed point norm. We trained RNNs on the example fixed point task. Similar to [Fig. 13](#), but with smaller noise, $\sigma_{\text{noise}} = 0.2$, and with learning rate increased by 2 and number of epochs by 2.5. **A-C**: Learning dynamics with gradient descent for one network with $N = 256$ neurons. The first 400 epochs are dominated by L_{bias} , and $M_{11} \approx \lambda_-$ becomes *positive*. The negative feedback loop, $\lambda_- < 0$ only forms later in learning. The matrix M does not become symmetric during learning. **D-F**: Fixed point norm and correlation for different N evaluated when $\lambda_- = 0$ (left) and at the end of learning (right). The time points are indicated by a square and triangle in **C**, respectively. At $\lambda_- = 0$, simulation and theory agree for the scaling: $\|\tilde{\kappa}\| = O(1)$ and $\rho = O(1/\sqrt{N})$. At the end of training, the theory predicts a decreasing fixed point norm, but the simulated networks inherit the order-one norm from the training history.

References

- [1] Arieli Amos, et al. (1996) **Dynamics of ongoing activity: explanation of the large variability in evoked cortical responses** *Science* **273**:1868–1871
- [2] Atallah Bassam V, Scanziani Massimo (2009) **Instantaneous modulation of gamma oscillation frequency by balancing excitation with inhibition** *Neuron* **62**:566–577
- [3] Barak Omri (2017) **Recurrent neural networks as versatile tools of neuroscience research** *Current Opinion in Neurobiology* **46**:1–6
- [4] Beiran Manuel, et al. (2021) **Shaping dynamics with multiple populations in low-rank recurrent networks** *Neural computation* **33**:1572–1615
- [5] Blanc Guy, et al. (2020) **Implicit regularization for deep neural networks driven by an ornstein-uhlenbeck like process** *Conference on learning theory* :483–513
- [6] Bondanelli Giulio, Ostojic Srdjan (2020) **Coding with transient trajectories in recurrent neural networks** *PLoS computational biology* **16**
- [7] Buonomano Dean V, Maass Wolfgang (2009) **State-dependent computations: spatiotemporal processing in cortical networks** *Nature Reviews Neuroscience* **10**:113–125
- [8] Chettih Selmaan N, Harvey Christopher D (2019) **Single-neuron perturbations reveal feature-specific competition in V1** *Nature* **567**:334–340
- [9] Chizat Lenaic, Oyallon Edouard, Bach Francis (2019) **On lazy training in differentiable programming** *Advances in Neural Information Processing Systems* :2937–2947
- [10] Churchland Mark M, et al. (2010) **Cortical preparatory activity: representation of movement or first cog in a dynamical machine?** *Neuron* **68**:387–400
- [11] Churchland Mark M, et al. (2012) **Neural population dynamics during reaching** *Nature* **487**:51–56
- [12] Degenhart Alan D, et al. (2020) **Stabilization of a brain-computer interface via the alignment of lowdimensional spaces of neural activity** *Nature biomedical engineering* **4**:672–685
- [13] Dubreuil Alexis, et al. (2021) **The role of population structure in computations through neural dynamics** *bioRxiv* :2020–7
- [14] Finkelstein Arseny, et al. (2021) **Attractor dynamics gate cortical information flow during decisionmaking** *Nature Neuroscience* **24**:843–850
- [15] Flesch Timo, et al. (2022) **Orthogonal representations for robust context-dependent task performance in brains and neural networks** *Neuron* **110**:1258–1270
- [16] Galgali Aniruddh R, Sahani Maneesh, Mante Valerio (2023) **Residual dynamics resolves recurrent contributions to neural computation** *Nature Neuroscience* **26**:326–338

- [17] Gallego Juan A, et al. (2018) **Cortical population activity within a preserved neural manifold underlies multiple motor behaviors** *Nature communications* **9**:1–13
- [18] Gallego Juan A, et al. (2020) **Long-term stability of cortical population dynamics underlying consistent behavior** *Nature neuroscience* **23**:260–270
- [19] Gallego Juan A, et al. (2017) **Neural manifolds for the control of movement** *Neuron* **94**:978–984
- [20] Gao Peiran, et al. (2017) **A theory of multineuronal dimensionality, dynamics and measurement** *bioRxiv* doi: 10.1101/214262v2
- [21] Geiger Mario, et al. (2020) **Disentangling feature and lazy training in deep neural networks** *Journal of Statistical Mechanics: Theory and Experiment* **2020**
- [22] Golub Matthew D, et al. (2018) **Learning by neural reassociation** *Nature neuroscience* **21**:607–616
- [23] Hafting Torkel, et al. (2005) **Microstructure of a spatial map in the entorhinal cortex** *Nature* **436**:801–806
- [24] Hennequin Guillaume, Vogels Tim P, Gerstner Wulfram (2014) **Optimal control of transient dynamics in balanced networks supports generation of complex movements** *Neuron* **82**:1394–1406
- [25] Hennig Jay A, et al. (2018) **Constraints on neural redundancy** *Elife* **7**
- [26] Hubel David H, Wiesel Torsten N (1962) **Receptive fields, binocular interaction and functional architecture in the cat's visual cortex** *The Journal of physiology* **160**
- [27] Jacot Arthur, Gabriel Franck, Hongler Clément (2018) **Neural tangent kernel: Convergence and generalization in neural networks** *Advances in neural information processing systems* :8571–8580
- [28] Jacot Arthur, et al. (2022) **Saddle-to-Saddle Dynamics in Deep Linear Networks: Small Initialization Training, Symmetry, and Sparsity** *arXiv*:2106.15933
- [29] Kadmon Jonathan, Timcheck Jonathan, Ganguli Surya (2020) **Predictive coding in balanced neural networks with noise, chaos and delays** *Advances in Neural Information Processing Systems* **33**:16677–16688
- [30] Kao Ta-Chu, Sadabadi Mahdiah S, Hennequin Guillaume (2021) **Optimal anticipatory control as a theory of motor preparation: A thalamo-cortical circuit model** *Neuron* **109**:1567–1581
- [31] Kaufman Matthew T, et al. (2014) **Cortical activity in the null space: permitting preparation without movement** *Nature neuroscience* **17**:440–448
- [32] Kim Christopher M, et al. (2023) **Distributing task-related neural activity across a cortical network through task-independent connections** *Nature Communications* **14**
- [33] Kingma Diederik P, Ba Jimmy (2014) **Adam: A method for stochastic optimization** *arXiv*: 1412.6980

- [34] Kloeden Peter E., Platen Eckhard (1992) **Numerical Solution of Stochastic Differential Equations**
- [35] Li Zhiyuan, Wang Tianhao, Arora Sanjeev (2021) **What Happens after SGD Reaches Zero Loss? -A Mathematical Framework** *arXiv:2110.06914*
- [36] Liu Chaoyue, Zhu Libin, Belkin Mikhail (2020) **On the linearity of large non-linear models: when and why the tangent kernel is constant** *Advances in Neural Information Processing Systems* **33**
- [37] Liu Yuhua Helena, et al. (2023) **How connectivity structure shapes rich and lazy learning in neural circuits** *arXiv:2310.08513*
- [38] Logiacco Laureline, Abbott LF, Escola Sean (2021) **Thalamic control of cortical dynamics in a model of flexible motor sequencing** *Cell reports* **35**
- [39] Maheswaranathan Niru, et al. (2019) **Universality and individuality in neural dynamics across large populations of recurrent networks** *Advances in Neural Information Processing Systems* :15603–15615
- [40] Makin Joseph G, et al. (2018) **Superior arm-movement decoding from cortex with a new, unsupervised-learning algorithm** *Journal of neural engineering* **15**
- [41] Mante Valerio, et al. (2013) **Context-dependent computation by recurrent dynamics in prefrontal cortex** *Nature* **503**
- [42] Mastrogiuseppe Francesca, Ostojic Srdjan (2018) **Linking connectivity, dynamics, and computations in low-rank recurrent neural networks** *Neuron* **99**:609–623
- [43] Mei Song, Montanari Andrea, Nguyen Phan-Minh (2018) **A mean field view of the landscape of two-layer neural networks** *Proceedings of the National Academy of Sciences* **115**
- [44] Musall Simon, et al. (2019) **Single-trial neural dynamics are dominated by richly varied movements** *Nature neuroscience* **22**:1677–1686
- [45] O'Keefe John, Dostrovsky Jonathan (1971) **The hippocampus as a spatial map: preliminary evidence from unit activity in the freely-moving rat** *Brain research*
- [46] O'Shea Daniel J, et al. (2022) **Direct neural perturbations reveal a dynamical mechanism for robust computation** *bioRxiv*
- [47] Oby Emily R, et al. (2019) **New neural activity patterns emerge with long-term learning** *Proceedings of the National Academy of Sciences* **116**:15210–15215
- [48] Okun Michael, Lampl Ilan (2008) **Instantaneous correlation of excitation and inhibition during ongoing and sensory-evoked activities** *Nature neuroscience* **11**:535–537
- [49] Pagan Marino, et al. (2022) **A new theoretical framework jointly explains behavioral and neural variability across subjects performing flexible decision-making** *bioRxiv*
- [50] Paszke Adam, et al. (2017) **Automatic differentiation in PyTorch** *none*
- [51] Pedregosa F., et al. (2011) **Scikit-learn: Machine Learning in Python** *Journal of Machine Learning Research* **12**:2825–2830

- [52] Pei Felix, et al. (2021) **Neural Latents Benchmark '21: Evaluating latent variable models of neural population activity** *Advances in Neural Information Processing Systems*
- [53] Preciado Victor M, Rahimian M Amin (2016) **Controllability gramian spectra of random networks** *2016 American Control Conference (ACC)* :3874–3879
- [54] Rajeswaran Pavithra, et al. (2024) **Assistive sensory-motor perturbations influence learned neural representations** *bioRxiv*
- [55] Ratzon Aviv, Derdikman Dori, Barak Omri (2024) **Representational drift as a result of implicit regularization** *Elife* **12**
- [56] Rigotti Mattia, et al. (2013) **The importance of mixed selectivity in complex cognitive tasks** *Nature* **497**
- [57] Rivkind Alexander, Barak Omri (2017) **Local dynamics in trained recurrent neural networks** *Physical Review Letters* **118**
- [58] Rokni Uri, et al. (2007) **Motor learning with unstable neural representations** *Neuron* **54**:653–666
- [59] Romo Ranulfo, et al. (1999) **Neuronal correlates of parametric working memory in the prefrontal cortex** *Nature* **399**
- [60] Russell Lloyd E, et al. (2022) **All-optical interrogation of neural circuits in behaving mice** *Nature Protocols* **17**:1579–1620
- [61] Russo Abigail A, et al. (2018) **Motor cortex embeds muscle-like commands in an untangled population response** *Neuron* **97**:953–966
- [62] Russo Abigail A, et al. (2020) **Neural trajectories in the supplementary motor area and motor cortex exhibit distinct geometries, compatible with different classes of computation** *Neuron* **107**:745–758
- [63] Sadtler Patrick T, et al. (2014) **Neural constraints on learning** *Nature* **512**:423–426
- [64] Saxena Shreya, Cunningham John P (2019) **Towards the neural population doctrine** *Current opinion in neurobiology* **55**:103–111
- [65] Saxena Shreya, et al. (2022) **Motor cortex activity across movement speeds is predicted by network-level strategies for generating muscle activity** *Elife* **11**
- [66] Schroeder Karen E, et al. (2022) **Cortical control of virtual self-motion using task-specific subspaces** *Journal of Neuroscience* **42**:220–239
- [67] Schuecker Jannis, Goedeke Sven, Helias Moritz (2018) **Optimal sequence memory in driven random networks** *Physical Review X* **8**
- [68] Schuessler Friedrich, et al. (2020) **Dynamics of random recurrent networks with correlated low-rank structure** *Physical Review Research* **2**
- [69] Schuessler Friedrich, et al., Larochelle H., et al. (2020) **The interplay between randomness and structure during learning in RNNs** *Advances in Neural Information Processing Systems* :13352–13362

- [70] Sompolinsky Haim, Crisanti Andrea, Sommers Hans-Jurgen (1988) **Chaos in random neural networks** *Physical Review Letters* **61**
- [71] Stringer Carsen, et al. (2019) **High-dimensional geometry of population responses in visual cortex** *Nature* **571**:361–365
- [72] Stringer Carsen, et al. (2019) **Spontaneous behaviors drive multidimensional, brainwide activity** *Science* **364**
- [73] Susman Lee, et al. (2021) **Quality of internal representation shapes learning performance in feedback neural networks** *Physical Review Research* **3**
- [74] Sussillo David (2014) **Neural circuits as computational dynamical systems** *Current Opinion in Neurobiology* **25**:156–163
- [75] Sussillo David, Abbott Larry F (2009) **Generating coherent patterns of activity from chaotic neural networks** *Neuron* **63**:544–557
- [76] Sussillo David, Barak Omri (2013) **Opening the black box: low-dimensional dynamics in highdimensional recurrent neural networks** *Neural computation* **25**:626–649
- [77] Sussillo David, et al. (2015) **A neural network that finds a naturalistic solution for the production of muscle activity** *Nature neuroscience* **18**
- [78] Talluri Bharath C, et al. (2022) **Activity in primate visual cortex is minimally driven by spontaneous movements** *bioRxiv* :2022–9
- [79] Turner Elia, Dabholkar Kabir, Barak Omri (2021) **Charting and navigating the space of solutions for recurrent neural networks** *Advances in Neural Information Processing Systems* **34**
- [80] Vyas Saurabh, et al. (2020) **Computation through neural population dynamics** *Annual Review of Neuroscience* **43**:249–275
- [81] Wang Ziyue Aiden, et al. (2023) **Not everything, not everywhere, not all at once: a study of brain-wide encoding of movement** *bioRxiv* :2023–6
- [82] Willett Francis R, et al. (2021) **High-performance brain-to-text communication via handwriting** *Nature* **593**:249–254
- [83] Williams Alex, et al. (2021) **Generalized Shape Metrics on Neural Representations** *Advances in Neural Information Processing Systems* **34**
- [84] Yan Kun, Cheng Gengdong, Wang Bo Ping (2016) **Adjoint methods of sensitivity analysis for Lyapunov equation** *Structural and Multidisciplinary Optimization* **53**:225–237
- [85] Yang Greg, Hu Edward J (2020) **Feature Learning in Infinite-Width Neural Networks** *arXiv: 2011.14522*
- [86] Yang Ning, Tang Chao, Tu Yuhai (2023) **Stochastic gradient descent introduces an effective landscape-dependent regularization favoring flat solutions** *Physical Review Letters* **130**

Editors

Reviewing Editor

Tatyana Sharpee

Salk Institute for Biological Studies, La Jolla, United States of America

Senior Editor

Michael Frank

Brown University, Providence, United States of America

Reviewer #1 (Public review):

Summary:

In this work, authors utilize recurrent neural networks (RNNs) to explore the question of when and how neural dynamics and the network's output are related from a geometrical point of view. The authors found that RNNs operate between two extremes: an 'aligned' regime in which the weights and the largest PCs are strongly correlated and an 'oblique' regime where the output weights and the largest PCs are poorly correlated. Large output weights led to oblique dynamics, and small output weights to aligned dynamics. This feature impacts whether networks are robust to perturbation along output directions. Results were linked to experimental data by showing that these different regimes can be identified in neural recordings from several experiments.

Strengths:

Diverse set of relevant tasks
Similarity measure well chosen
Explored various hyperparameter settings

Weaknesses:

One of the major connections to found BCI data with neural variance aligned to the outputs. Maybe I was confused about something, but doesn't this have to be the case based on the design of the experiment? The outputs of the BCI are chosen to align with the largest principal components of the data.

Proposed experiments maybe have already been done (New neural activity patterns emerge with long-term learning, Oby et al. 2019). My understanding of these results is that activity moved to be aligned as the manifold changed, but more analyses could be done to more fully understand the relationship between those experiments and this work.

Analysis of networks was thorough, but connections to neural data were weak. I am thoroughly convinced of the reported effect of large or small output weights in networks. I also think this framing could aid in future studies of interactions between brain regions.

This is an interesting framing to consider the relationship between upstream activity and downstream outputs. As more labs record from several brain regions simultaneously, this work will provide an important theoretical framework for thinking about the relative geometries of neural representations between brain regions.

It will be interesting to compare the relationship between geometries of representations and neural dynamics across connected different brain areas that are closer to the periphery vs. more central.

Exciting to think about the versatility of the oblique regime for shared representations and network dynamics across different computations.

Versatility of oblique regime could lead to differences between subjects in neural data.

<https://doi.org/10.7554/eLife.93060.2.sa2>

Reviewer #2 (Public review):

Summary:

This paper tackles the problem of understanding when the dynamics of neural population activity do and do not align with some target output, such as an arm movement. The authors develop a theoretical framework based on RNNs showing that an alignment of neural dynamics to an output can be simply controlled by the magnitude of the read-out weight vector while the RNN is being trained: small magnitude vectors result in aligned dynamics, where low-dimensional neural activity recapitulates the target; large magnitude vectors result in "oblique" dynamics, where encoding is spread across many dimensions. The paper further explores how the aligned and oblique regimes differ, in particular that the oblique regime allows degenerate solutions for the same target output.

Strengths:

- A really interesting new idea that different dynamics of neural circuits can arise simply from the initial magnitude of the output weight vector: once written out (Eq 3) it becomes obvious, which I take as the mark of a genuinely insightful idea
- The offered framework potentially unifies a collection of separate experimental results and ideas, largely from studies of motor cortex in primate: the idea that much of the ongoing dynamics do not encode movement parameters; the existence of the "null space" of preparatory activity; and that ongoing dynamics of motor cortex can rotate in the same direction even when the arm movement is rotating in opposite directions.
- The main text is well written, with a wide-ranging set of key results synthesised and illustrated well and concisely
- Shows the occurrence of the aligned and oblique regimes generalises across a range of simulated behavioural tasks
- A deep analytical investigation of when the regimes occur and how they evolve over training
- Shows where the oblique regime may be advantageous: allows multiple solutions to the same problem; and differs in sensitivity to perturbation and noise
- An insightful corollary result that noise in training is needed to obtain the oblique regime
- Tests whether the aligned and oblique regimes can be seen in neural recordings from primate cortex in a range of motor control tasks
- The revised text offers greater clarity and precision about when the aligned and oblique regimes occur and in the interpretation of the analyses of neural data

Weaknesses:

- The depth of analytical treatment in the Methods is impressive; however, the paper and the Methods analyses are largely independent, with the numerous results in the latter not being

mentioned in the Results or Discussion. It in effect operates as two papers.

<https://doi.org/10.7554/eLife.93060.2.sa1>

Author response:

The following is the authors' response to the original reviews.

Public Reviews:

Reviewer #1 (Public Review):

Summary:

In this work, the authors utilize recurrent neural networks (RNNs) to explore the question of when and how neural dynamics and the network's output are related from a geometrical point of view. The authors found that RNNs operate between two extremes: an 'aligned' regime in which the weights and the largest PCs are strongly correlated and an 'oblique' regime where the output weights and the largest PCs are poorly correlated. Large output weights led to oblique dynamics, and small output weights to aligned dynamics. This feature impacts whether networks are robust to perturbation along output directions. Results were linked to experimental data by showing that these different regimes can be identified in neural recordings from several experiments.

Strengths:

A diverse set of relevant tasks.

A well-chosen similarity measure.

Exploration of various hyperparameter settings.

Weaknesses:

One of the major connections found BCI data with neural variance aligned to the outputs.

Maybe I was confused about something, but doesn't this have to be the case based on the design of the experiment? The outputs of the BCI are chosen to align with the largest principal components of the data.

The reviewer is correct. We indeed expected the BCI experiments to yield aligned dynamics. Our goal was to use this as a comparison for other, non-BCI recordings in which the correlation is smaller, i.e. dynamics closer to the oblique regime. We adjusted our wording accordingly and added a small discussion at the end of the experimental results, Section 2.6.

Proposed experiments may have already been done (new neural activity patterns emerge with long-term learning, Oby et al. 2019). My understanding of these results is that activity moved to be aligned as the manifold changed, but more analyses could be done to more fully understand the relationship between those experiments and this work.

The on- vs. off-manifold experiments are indeed very close to our work. On-manifold initializations, as stated above, are expected to yield aligned solutions. Off-manifold initializations allow, in principle, for both aligned and oblique solutions and are thus closer to our RNN simulations. If, during learning, the top PCs (dominant activity) rotate such that they align with the pre-defined output weights, then the system has reached an aligned solution. If the top PCs hardly change, and yet the behavior is still good, this is an oblique solution. There

is some indication of an intermediate result (Figure 4C in Oby et al.), but the existing analysis there did not fully characterize these properties. Furthermore, our work suggests that systematically manipulating the norm of readout weights in off-manifold experiments can yield new insights. We thus view these as relevant results but suggest both further analysis and experiments. We rewrote the corresponding section in the discussion to include these points.

Analysis of networks was thorough, but connections to neural data were weak. I am thoroughly convinced of the reported effect of large or small output weights in networks. I also think this framing could aid in future studies of interactions between brain regions.

This is an interesting framing to consider the relationship between upstream activity and downstream outputs. As more labs record from several brain regions simultaneously, this work will provide an important theoretical framework for thinking about the relative geometries of neural representations between brain regions.

It will be interesting to compare the relationship between geometries of representations and neural dynamics across connected different brain areas that are closer to the periphery vs. more central.

It is exciting to think about the versatility of the oblique regime for shared representations and network dynamics across different computations.

The versatility of the oblique regime could lead to differences between subjects in neural data.

Thank you for the suggestions. Indeed, this is precisely why relative measures of the regime are valuable, even in the absence of absolute thresholds for regimes. We included your suggestions in the discussion.

Reviewer #2 (Public Review):

Summary:

This paper tackles the problem of understanding when the dynamics of neural population activity do and do not align with some target output, such as an arm movement. The authors develop a theoretical framework based on RNNs showing that an alignment of neural dynamics to output can be simply controlled by the magnitude of the read-out weight vector while the RNN is being trained. Small magnitude vectors result in aligned dynamics, where low-dimensional neural activity recapitulates the target; large magnitude vectors result in "oblique" dynamics, where encoding is spread across many dimensions. The paper further explores how the aligned and oblique regimes differ, in particular, that the oblique regime allows degenerate solutions for the same target output.

Strengths:

- A really interesting new idea that different dynamics of neural circuits can arise simply from the initial magnitude of the output weight vector: once written out (Eq 3) it becomes obvious, which I take as the mark of a genuinely insightful idea.*
- The offered framework potentially unifies a collection of separate experimental results and ideas, largely from studies of the motor cortex in primates: the idea that much of the ongoing dynamics do not encode movement parameters; the existence of the "null space" of preparatory activity; and that ongoing dynamics of the motor cortex can rotate in the same direction even when the arm movement is rotating in opposite directions.*

- The main text is well written, with a wide-ranging set of key results synthesised and illustrated well and concisely.
- The study shows that the occurrence of the aligned and oblique regimes generalises across a range of simulated behavioural tasks.
- A deep analytical investigation of when the regimes occur and how they evolve over training.
- The study shows where the oblique regime may be advantageous: allows multiple solutions to the same problem; and differs in sensitivity to perturbation and noise.
- An insightful corollary result that noise in training is needed to obtain the oblique regime.
- Tests whether the aligned and oblique regimes can be seen in neural recordings from primate cortex in a range of motor control tasks.

Weaknesses:

- The magnitude of the output weights is initially discussed as being fixed, and as far as I can tell all analytical results (sections 4.6-4.9) also assume this. But in all trained models that make up the bulk of the results (Figures 3-6) all three weight vectors/matrices (input, recurrent, and output) are trained by gradient descent. It would be good to see an explanation or results offered in the main text as to why the training always ends up in the same mapping (small->aligned; large->oblique) when it could, for example, optimise the output weights instead, which is the usual target (e.g. Sussillo & Abbott 2009 Neuron).

We understand the reviewer's surprise. We chose a typical setting (training all weights of an RNN with Adam) to show that we don't have to fine-tune the setting (e.g. by fixing the output weights) to see the two regimes. However, other scenarios in which the output weights do change are possible, depending on the algorithm and details in the way the network is parameterized. Understanding why some settings lead to our scenario (no change in scale) and others don't is not a simple question. A short explanation here, nonetheless:

- Small changes to the internal weights are sufficient to solve the tasks.
- Different versions of gradient descent and different ways of parametrizing the network lead to different results in which parts of the weights get trained. This goes in particular for how weight scales are introduced, e.g. [Jacot et al. 2018 Neurips], [Geiger et al. 2020 Journal of Statistical Mechanics], or [Yang, Hu 2020, arXiv, Feature learning in infinite-width networks]. One insight from these works is that plain gradient descent (GD) with small output weights leads to learning only at the output (and often divergence or unsuccessful learning). For this reason, plain GD (or stochastic GD) is not suitable for small output weights (the aligned regime). Other variants of GD, such as Adam or RMSprop, don't have this problem because they shift the emphasis of learning to the hidden layers (here the recurrent weights). This is due to the normalization of the gradients.
- FORCE learning [Sussillo & Abbott 2009] is somewhat special in that the output weights are simultaneously also used as feedback weights. That is, not only the output weights but also an additional low-rank feedback loop through these output weights is trained. As a side note: By construction, such a learning algorithm thus links the output directly to the internal dynamics, so that one would only expect aligned solutions – and the output weights remain correspondingly small in these algorithms [Mastrogiuseppe, Ostojic, 2019, Neural Comp].

- In our setting, the output is not fed back to the network, so training the output alone would usually not suffice. Indeed, optimizing just the output weights is similar to what happens in the lazy training regime. These solutions, however, are not robust to noise, and we show that adding noise during the training does away with these solutions.

To address this issue in the manuscript, we added the following sentence to section 2.2: “While explaining this observation is beyond the scope of this work, we note that (1) changing the internal weights suffices to solve the task, and that (2) the extent to which the output weights change during learning depends on the algorithm and specific parametrization [21, 27, 85].”

- It is unclear what it means for neural activity to be "aligned" for target outputs that are not continuous time-series, such as the 1D or 2D oscillations used to illustrate most points here.

Two of the modeled tasks have binary outputs; one has a 3-element binary vector.

For any dynamics and output, we compare the alignment between the vector of output weights and the main PCs (the leading component of the dynamics). In the extreme of binary internal dynamics, i.e., two points $\{x_1, x_2\}$, there would only be one leading PC (the line connecting the two points, i.e. the choice decoder).

- It is unclear what criteria are used to assign the analysed neural data to the oblique or aligned regimes of dynamics.

Such an assignment is indeed difficult to achieve. The RNN models we showed were at the extremes of the two regimes, and these regimes are well characterized in the case of large networks (as described in the methods section). For the neural data, we find different levels of alignment for different experiments. These differences may not be strong enough to assign different regimes. Instead, our measures (correlation and relative fitting dimension) allow us to order the datasets. Here, the BCI data is more aligned than non-BCI data – perhaps unsurprisingly, given the experimental design of the prior and the previous findings for the rotation task [Russo et al, 2018]. We changed the manuscript accordingly, now focusing on the relative measure of alignment, even in the absence of absolute thresholds. We are curious whether future studies with more data, different tasks, or other brain regions might reveal stronger differentiation towards either extreme.

Recommendations for the authors:

Reviewer #1 (Recommendations For The Authors):

There's so much interesting content in the supplement - it seemed like a whole other paper! It is interesting to read about the dynamics over the course of learning. Maybe you want to put this somewhere else so that more people read it?

We are glad the reviewer appreciated this content. We think developing these analysis methods is essential for a more complete understanding of the oblique regime and how it arises, and that it should therefore be part of the current paper.

Nice schematic in Figure 1.

There were some statements in the text highlighting co-rotation in the top 2 PCs for oblique networks. Figure 4a looks like aligned networks might also co-rotate in a particular subspace that is not highlighted. I could be wrong, but the authors should

look into this and correct it if so. If both aligned and oblique networks have co-rotation within the top 5 or so PCs, some text should be updated to reflect this.

This is indeed the case, thanks for pointing this out! For one example, there is co-rotation for the aligned network already in the subspace spanned by PCs 1 and 3, see the figure below. We added a sentence indicating that co-rotation can take place at low-variance PCs for the aligned regime and pointed to this figure, which we added to the appendix (Fig. 17).

While these observations are an important addition, we don't think they qualitatively alter our results, particularly the stronger dissociation between output and internal dynamics for oblique than aligned dynamics.

Figure 4 color labels were 'dark' and 'light'. I wasn't sure if this was a typo or if it was designed for colorblind readers? Either way, it wasn't too confusing, but adding more description might be useful.

Fixed to red and yellow.

Typo "Aligned networks have a ratio much large than one"

Typo "just started to be explored" Typo "hence allowing to test"

Fixed all typos.

Reviewer #2 (Recommendations For The Authors):

- Explain/discuss in the main text why the initial output weights reliably result in the required internal RNN dynamics (small->aligned; large->oblique) after training. The magnitude of the output weights is initially discussed as being fixed, and as far as I can tell all analytical results (sections 4.6-4.9) also assume this. But in all trained models that make up the bulk of the results (Figures 3-6) all three weight vectors/matrices (input, recurrent, and output) are trained by gradient descent. It would be good to see an explanation or results offered in the main text as to why the training always ends up in the same mapping (small->aligned; large->oblique) when it could, for example, just optimise the output weights instead.

See the answer to a similar comment by Reviewer #1 above.

- Page 6: explain the 5 tasks.

We added a link to methods where the tasks are described.

- Page 6/Fig 3 & Methods: explain assumptions used to compute a reconstruction R^2 between RNN PCs and a binary or vector target output.

We added a new methods section, 4.4, where we explain the fitting process in Fig. 3. For all tasks, the target output was a time series with P specified target values in N_out dimensions. We thus always applied regression and did not differentiate between binary and non-binary tasks.

- Page 8: methods and predictions are muddled up: paragraph ending "along different directions" should be followed by paragraph starting "Our intuition...". The intervening paragraph ("We apply perturbations...") should start after the first sentence of the paragraph "To test this,...".

Right, these sentences were muddled up indeed. We put them in the correct order.

- Page 10: *what are the implications of the differences in noise alignment between the aligned and oblique regimes?*

The noise suppression in the oblique regime is a slow learning process that gradually renders the solution more stable. With a large readout, learning separates into two phases. An early phase, in which a “lazy” solution is learned quickly. This solution is not robust to noise. In a second, slower phase, learning gradually leads to a more robust solution: the oblique solution. The main text emphasizes the result of this process (noise suppression). In the methods, we closely follow this process. This process is possibly related to other slow learning process fine-tuning solutions, e.g., [Blanc et al. 2020, Li et al. 2021, Yang et al. 2023]. Furthermore, it would be interesting to see whether such fine-tuning happens in animals [Ratzon et al. 2024]. We added corresponding sentences to the discussion.

- Neural data analysis:

(i) Page 11 & Fig 7: *the assignment of "aligned" or "oblique" to each neural dataset is based on the ratio of D_{fit}/D_x . But in all cases this ratio is less than 1, indicating fewer dimensions are needed for reconstruction than for explaining variance. Given the example in Figure 2 suggests this is an aligned regime, why assign any of them as "oblique"?*

We weakened the wording in the corresponding section, and now only state that BCI data leans more towards aligned, non-BCI data more towards oblique. This is consistent with the intuition that BCI is by construction aligned (decoder along largest PCs) and non-BCI data already showed signs of oblique dynamics (co-rotating leading PCs in the cycling task, Russo et al. 2018).

We agree that Fig 2 (and Fig 3) could suggest distinguishing the regimes at a threshold $D_{\text{fit}}/D_x = 1$, although we hadn't considered such a formal criterion.

(ii) Figure 23 and main text page 11: *discuss which outputs for NLB and BCI datasets were used in Figure 7 & and main text; the NLB results vary widely by output type - discuss in the main text; D_{fit} for NLB-maze-accuracy is missing from panel D; as the criterion is D_{fit}/D_x , plot this too.*

We now discuss which outputs were used in Fig. 7 in its caption: the velocity of the task-relevant entity (hand/finger/cursor). This was done to have one quantity across studies. We added a sentence to the main text, p. 11, which points to Fig 22 (which used to be Fig 23) and states that results are qualitatively similar for other decoded outputs, despite some fluctuations in numerical values and decodability.

Regarding Fig 22: D_{fit} for NLB-maze-accuracy was beyond the manually set y-limit (for visibility of the other data points). We also extended the figure to include D_{fit}/D_x . We also discovered a small bug in the analysis code which required us to rerun the analysis and reproduce the plots. This also changed some of the numbers in the main text.

- Discussion:

"They do not explain why it [the "irrelevant activity"] is necessary", implies that the following sentence(s) will explain this, but do not. Instead, they go on to say:

"Here, we showed that merely ensuring stability of neural dynamics can lead to the oblique regime": this does not explain why it is necessary, merely that it exists; and it is

| *unclear what results "stability of neural dynamics" is referring to.*

We agree this was not a very clear formulation. We replaced these last three sentences with the following:

“Our study systematically explains this phenomenon: generating task-related output in the presence of large, task-unrelated dynamics requires large readout weights. Conversely, in the presence of large output weights, resistance to noise or perturbations requires large, potentially task-unrelated neural dynamics (the oblique regime).”

| *- The need for all 27 figures was unclear, especially as some seemed not to be referenced or were referenced out of order. Please check and clarify.*

Fig 16 (Details for network dynamics in cycling tasks) and Fig 21 (loss over learning time for the different tasks) were not referenced, and are now removed.

We also reordered the figures in the appendix so that they would appear in the order they are referenced. Note that we added another figure (now Fig. 17) following a question from Reviewer #1.

<https://doi.org/10.7554/eLife.93060.2.sa0>